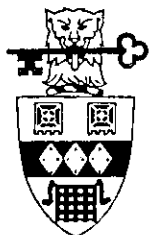


VECTOR

All the *APL* that's fit to Print!

- 16-page Educational Supplement
- APL90: Abstracts and Registration
- Fully Revised Product Guide
- APL*PLUS/PC Version 9 Reviewed
- More Problems for APL Buffs
- Error Trapping in APL2



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.6 No.4 April 1990

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1989/90 Committee

Chairman:	Peter Donnelly 0420-87024	Dyadic Systems Ltd., Park House, The High Street, ALTON, Hants GU34 1EN
Secretary:	Graham Parkhouse 0483-571281 (x2379)	Dept. of Mech. Eng., University of Surrey, GUILDFORD GU2 5XH.
Treasurer:	John Sullivan 081 462-3628	24 Mounthurst Road, Hayes, Kent BR2 7QN
Journal Editor:	Adrian Smith 04393-385	Brook House, Gilling East, YORK YO6 4JJ
Activities:	Dr Peter Branson 0737-242950	Oaklands Cottage, Wray Common, Reigate, Surrey RH2 0LE
Education:	Dr Alan Sykes 0792-295296	Dept of Management Science University College of Swansea, Singleton Park, SWANSEA SA2 8PP
Publicity:	Jonathan Martin 081 562-5697	(S499) British Airways, PO Box 10, Heathrow Airport, HOUNSLOW, Middlesex TW6 2JA
Technical:	David Eastwood 071-922 8866	MicroAPL Ltd South Bank Technopark, 90 London Road, LONDON SE1 6LN
Recruitment:	Jill Moss 0225-462602	APL People Ltd, The Old Malthouse Clarence St., BATH, Avon BA1 5NS
Projects:	John Searle 081 948-6737	13A Mount Ararat Road, Richmond, Surrey TW10 6PQ
Administration:	Allison Chatterton 0992-552489	9, Oak Grove, HERTFORD, SG13 8AT

Journal Working Group

Jonathan Barman	048839-575
Anthony Camacho	0727-60130
Cathy Dargue	0895-31313
Adrian Smith	04393-385
David Ziemann	071-436 9481

Typeset using MS Word 5.0 and GoScript on HP Deskjet+

Printed in England by Short-Run Press Ltd, Exeter

CONTENTS

EDITORIAL: a Network Survival Kit	Adrian Smith	Page 3
APL NEWS		
Quick Reference Diary		5
APL Course Diary		7
British APL Association News	Graham Parkhouse	9
News from Sustaining Members	Alison Chatterton	11
The Education VECTOR	Alan Sykes	17
APL90: Invitation and Abstracts		33
REVIEWS		
APL Product Guide	Alison Chatterton	44
APL*PLUS/PC Release 9	Adrian Smith	54
APL2.5 for the Microwriter Agenda	Adrian Smith	58
RECENT MEETINGS		
Introduction	Adrian Smith	60
Arthur Whitney on A	Ed Shaw	62
Workshop on Defined Operators	John Scholes et al	64
A Hypertext Interface to the Oracle	Adrian Smith	71
GENERAL ARTICLES		
A Design Framework for APL Systems	Dave Piper	78
TECHNICAL SECTION		
Hackers Corner: the Falkoff Dual Keyboard	Adrian Smith	92
Technical Correspondence	Dave Piper et al	95
A New Development Environment in Dyalog APL	John Scholes	101
Error Trapping in APL2/PC	Norman Thomson	105
Comments on APL-2741	Adin Falkoff	116
Problems for APL Buffs (II)	Zdenek Jizba	117
A Note on Comparison Tolerance	R.G. Williams & R.A. Green	123
Index to Vol 6		125
Goodbye, and thanks for all the Fish!		126
Index to Advertisers		127

Development Programmers/Consultants

Saladin is a fast-growing UK company providing decision support systems to the oil and petrochemicals industries, particularly the trading and planning sectors. Our business is international with offices in London and the US and client sites around the world.

Our software and communications systems are APL based, mainly using APL*Plus/PC.

We offer a range of opportunities for people with APL skills and currently seek —

- Development Programmers with a flair for creative and innovative system building
- Technical Consultants able to work closely with customers.

Saladin was set up in 1986 by a group of oil consultants and systems experts. We released our first software systems in early 1987 and have grown successfully since then as customers recognise the high value of the products and support that we offer. Our working environment is stimulating and fast-moving and technical staff work closely with their sales and consulting colleagues.

Salaries and benefits are excellent and will certainly attract experienced applicants.

Please contact Nigel Harris, our Technical Manager, for an informal discussion on 01-874 8247 or write to him with a CV to —

Saladin Computer Systems Ltd, 100 Garratt Lane,
Wandsworth, London SW18 4DJ.

Saladin
COMPUTER
SYSTEMS
LIMITED

Editorial: A Network Survival Kit

by Adrian Smith

Did you ever think how comfy your mainframe is? At the Association's March meeting, we had an excellent outline of Sharp's LOGOS environment, where you can develop code in a co-ordinated way, and have it automatically copied into all the affected applications at the press of a key. LOGOS protects you by retaining old versions of everything you change, and it encourages a professional attitude to documentation, change-journals and so on.

I don't pretend that I have anything approaching this at Rowntrees, but at least when I change a mainframe utility function, it gets automatically copied into any workspace which uses it. We also put all our users through a common startup workspace, so we can easily DROP old CONTINUE workspaces, check on frequency of usage, warn people about storage limits, and so on. At Christmas, everyone gets the trotting Reindeer ... yes, you can do animations even in GDDM!!

Now contrast this with a bunch of APLers developing applications on PCs. Obviously, the PC comes equipped with similar utility workspaces, but what happens when I need to fix a bug, or ship an enhancement? Do I trot round with a floppy disk, wait till the PC is free, and simply copy the .AWS or .ASF files over? Fine for development aids (as long as I remember to do it), but what about all the runtime workspaces out there? I know who I gave them to, but PC applications have a habit of spreading themselves!

In summary: once you give a PC user a disk, you relinquish all control over the contents. You no longer have any jurisdiction over the application code, or any restraints on where it could go next. It may well end up outside the company altogether; people often take work home with them ... I doubt if they are too careful about returning it when they leave!

In principle, the solution is fairly obvious: you need a network! If no-one ever needs to have your application physically present on their local disk, most of the management problems go away. Of course people can still take work home, but the vast bulk of systems remain where you can get at them. Of course, there are all sorts of design issues to be sorted out before you should even contemplate supporting live applications in this kind of environment. Well, we are going live at 6 factories across the UK at the moment, and I haven't really started to think about it yet!

The most obvious constraint is speed. On my PS/2 I have the usual complement of A;B;C;D: ... however (depending on the health of a rather ad hoc length of Ethernet cable) I can also boast of G: (actually a Vax in computer room); H: (another Vax somewhere in the factory); and I: (50 miles away at Castleford). G: and H: are about as fast as my PS/2 model-70 hard disk; I: is at the end of a 9600 baud phone connection! To)LOAD a workspace over this sort of link can take over 5 minutes.

Another problem concerns the behaviour of 'runtime' APL code. On the mainframe, we still get occasional crashes from ancient systems written in the days when no-one considered the possibility of there being more than 12 PFkeys. Typically these have statements like $\rightarrow(\text{Next, Help, End, } \dots) [1+PF]$... which is a pretty good guarantee of an *INDEX ERROR* when someone accidentally catches PF24. Most users have got into the habit of $PF+0 \rightarrow LC$, and don't even bother to ring up! Under APL*PLUS II (runtime) a trivial error like this will simply dump them in DOS, losing half a morning's work.

Some of the questions you need to think about:

- can you really afford to ship 'runtime' code? We are not talking *packages* here, we are talking fast moving applications which need to change as the business need changes. I suspect the answer is *no*, and that (at the very least) you should give the user emergency access to $\uparrow$ somewhere in the system. In doing so, you violate your APL*PLUS runtime licence ... oh bother!
- how are you going to distribute utility code? I suspect that what is needed is something a bit like LOGOS which will build and maintain 'patch' files; and a corresponding component in each application which will detect these and 'upgrade' itself. To make such a 'Trojan Horse' approach work, you need to make sure that you get the bare minimum of 'listening code' into *every* application *now*. You can design the horse later ... but if no-one is there to wheel it into the city, it will avail you nothing.
- what about multiple developers working on the same application? On the mainframe, we have a system of project libraries, and can use access controls to ensure no-one does anything silly. On the PC, it is dreadfully easy to trample all over each other's code!

In short, sooner or later, you may well get bundled unceremoniously off your mainframe, and if you are in a position to start sorting out some of these issues now, then you should. In my experience, it is always better to panic sooner rather than later, and a sound line of retreat never did any general any harm at all.

Quick Reference Diary

Date	Venue	Event
12 January 1990	London	BAA Meeting
16 February	London	BAA Meeting
23 March	London	BAA Meeting
20 April	London	BAA Meeting
8th June	London	BAA Annual General Meeting
13th - 17th August 1990	Denmark	APL90: APL for the Future
28 September	London	BAA Meeting
19 October	York	BAA Field Trip to Rowntree
23 November	London	BAA Meeting

British APL Association meetings are currently held at the Royal Over-Seas League, Park Place, near Green Park Tube station. Meetings begin at 2.00pm and close at 5.30pm. We are exploring alternative venues, as the committee feels it would be very helpful to have a Barco projector available for all meetings.

The dates from September are provisional, as we may need to re-book if we change venue.



for 386 PCs under MS-DOS

Dyalog APL for 386 PCs under MS-DOS is a complete implementation of Dyadic's popular APL for Unix systems, AND fully exploits the power of a DOS 386-PC.

JUST LOOK AT THE FEATURES . . .

- *An exciting new Screen Manager in which the contents of the screen are 'shared' with a nested system variable □SM (Screen Map). Facilities include:*

Pop-up windows by localisation of □SM
Scrolling (vertical, horizontal, linked)
GENUINE numeric fields
Date fields
NESTED forms

- Fully integrated window-based Session Manager, Editor and Debugging Environment.
- Compatible with the Draft ISO standard for APL
- Huge Workspaces (16Mb and MORE!)
- Virtual Memory Support (using on-chip demand paging)
- Full 32-bit processing speed
- *APL2 features including:*
 - Nested Arrays
 - Defined Operators
 - Selective Specification
- *Commercial features including:*
 - FMT
 - Component Files
 - Error Handling
 - "AP124-like" Screen Manager
- External Variables
- Interface to GSS*CGI for fast device-independent graphics
- C Interface

. . . THE BENEFITS SPEAK FOR THEMSELVES

Requirements: 386/486-based PC or PS/2, 640K DOS Memory plus 2Mb RAM, EGA or VGA (colour STRONGLY recommended), 80387 coprocessor STRONGLY recommended, DOS 3.3 or later.

For further information contact:
Dyadic Systems Limited
Park House, The High Street, Alton, Hampshire, GU34 1EN
Telephone: (0420) 87024 Fax: (0420) 89886 Telex: 85881

APL Training Courses for 1990

compiled by Adrian Smith

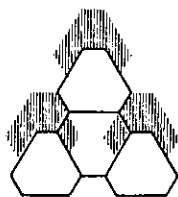
Dates shown cover the period from April to December 1990; the list was last updated on March 28th 1990. For confirmation of dates and/or further details please contact the vendor directly.

Level	Company	Days	Dates
Beginners	Cocking & Drury	3	24 April, 5 June, 3 July, 11 Sept
	MicroAPL	1	10 May, 14 June, 2 Aug, 4 Oct
APL*PLUS/PC	MicroAPL	1	29 Mar, 31 May, 19 July, 13 Sept, 1 Nov
Intermediate	MicroAPL	1	17 May, 5 July, 9 Aug, 11 Oct
APL*PLUS II conversion	MicroAPL	1	5 Apr, 7 Jun 23 Aug, 8 Nov
Advanced (Nested Arrays)	MicroAPL	1	24 May, 12 July, 6 Sept, 18 Oct
	Cocking & Drury	2	15 May, 17 July, 25 Sept
System Design	Cocking & Drury	4	17 Sept
Statgraphics	Cocking & Drury	2	19 June, 2 Oct

Mercia Software Ltd have asked us to note that they no longer offer a regular program of public courses in APL and related subjects.

Dates for Future Issues of VECTOR

	Vol.7 No.1	Vol.7 No.2	Vol.7 No.3
Copy date	1st June 90	7th Sept 90	1st Dec 90
Ad booking	16th June 90	21st Sept 90	8th Dec 90
Ad Copy	30th June 90	30th Sept 90	23rd Dec 90
Distribution	July 90	October 90	January 91



APL PEOPLE

JOBS!

JOBS!

JOBS!

As THE PREMIER APL employment agency, more APL vacancies are filled through us than any other agency.

Established over eight years ago, our understanding of the APL market-place is unrivalled. We always have a good selection of vacancies available throughout the UK.

To find out what we can do for you, contact

Jill Moss on 0225 462602 during office hours
or 0225 333618 eves & w/ends

or write to her at

The Old Malthouse, Clarence St., Bath BA1 5NS

British APL Association News

from Graham Parkhouse

How should we present APL at our meetings? From the earliest days of APL it has been traditional to learn APL hands-on, so it has been a pleasure at the first two meetings of this year to have hands-on demonstrations in front of a big screen. Tracing execution with the active functions displayed in windows is a delightful concept that needs to be experienced rather than read or heard about, and when I watched this at the January meeting I could not help feeling excited about computing.

Watching it all happening under the fingers of an intrepid speaker is an exhilarating experience from which APL can be caught as well as taught, so the committee are taking steps to have a projector regularly at the monthly meetings. The extra cost of having one is quite substantial but we need to maintain a very high standard at our meetings to justify the audience's time. We want to increase the numbers of people attending and to this end we shall be publicising the meetings in Education Vector which is sent out to the 1600 people on the I-APL mailing list. Average attendance at the last few meetings has been about 40.

Lucky for those who live in London! On 19 October Adrian Smith will be hosting a regional meeting of the Association at Rowntrees in York.

We are having difficulty knowing what sort of conference to organise. We have offered to host APL92 and provisionally booked conference facilities at York University, but SIGAPL have failed to respond formally to our offer. Unofficially we hear they are unenthusiastic about the idea. We have had to cancel our booking at York but decided at the March committee meeting we should run a conference in 1992 anyway without calling it APL92.

The APL Statistics Library proceeds apace: the 'bottom shelf' workspace contains 70 functions and by the end of April there will be a preliminary version ready for interested people to try. The initial version is written for APL*PLUS/PC. Anybody interested in helping with testing should contact David Eastwood, whose phone number is inside the back cover of this Vector.

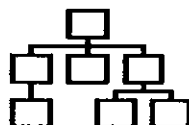
29 March 1990

A single source for APL



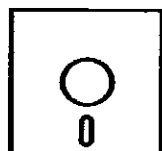
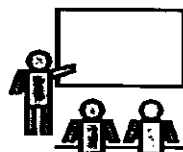
Advice

Design



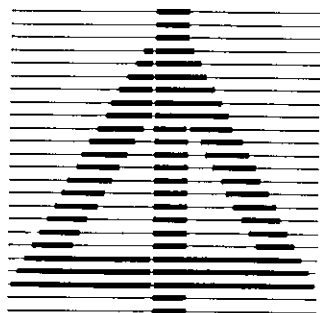
Application
Development

Education



APL software

Support



COCKING & DRURY LTD.

THE APL PROFESSIONALS

180 Tottenham Court Road

London W1P 9LE

Tel: 01 - 436 9481

(From May 1990, our STD code will be 071)

News from Sustaining Members

Compiled by Alison Chatterton

APL Impetus Ltd

George Macleod has joined APL Impetus as a Director.

For the past nine years George has been responsible for running Boeing Computer Services - Europe. In earlier days at Boeing, George and Cedric Heddle started the development of an APL based product which ran on a series of large IBM mainframes as part of the Boeing Time Sharing service. This is the product that Cedric Heddle ported to the IBM PC and transformed into IMPETUS and which represents the major activity of APL Impetus Limited.

George has a special interest in business graphics and this will result in the next major release of IMPETUS having a significant graphics facility totally integrated into the rest of the package.

IMPETUS is an Application Generator, mostly written in APL*PLUS PC, with assembler where necessary for improved performance. It addresses the type of problem that can be formulated as the relationship between a number, perhaps a large number, of numeric tables that can have complex internal and external relationships. In essence IMPETUS decomposes complex problems into smaller problems that are amenable to solution and manages the interaction of the sub-problems. The data is usually hierarchial and multi-dimensional and extensive report formatting facilities are provided. The system is also self documenting.

IMPETUS addresses a wide spectrum of business problems, ranging from those which require considerable conceptual skill to understand at one end of the spectrum, to relatively straight forward problems which, despite their essential simplicity, are beyond the ability of other simpler PC based tools.

If you want to know more call Cedric Heddle at 0732 885126 or George MacLeod at 0442 833419.

Intelligent Programs Limited

The company continues to expand, with an increase in all activities. Demand for support and development work on behalf of our clients has grown, and so we have taken on an additional member of staff since Vector 6.3 came out. We have

also taken on more office space, conveniently within a few hundred metres of our present offices.

HMW Computing Limited

HMW Computing Limited continue to develop and market "4XTRA" our front end Foreign Exchange Dealing and Position Keeping System. We are currently in the process of re-packaging the product, which will provide us with easier support and installation features.

4XTRA's arbitrage calculation package was installed to run on PS/2s in the London branch of a large American bank at the beginning of the month. The package paid for itself in the first few hours of trading, by allowing traders to identify trading opportunities in the ECU market, and hence increase their profits.

We are currently looking for more staff and would be pleased to talk to anyone interested in joining us. Please call Stan Wilkinson during trading hours on 01-353-4212 or at home on 01-286-7068.

If you would like to see the 4XTRA system, we are exhibiting at a number of exhibitions and would be pleased to offer invitations or if you prefer a demonstration can be arranged at our offices or on your own premises.

Our employment agency for the APL community, HMW EMPLOYMENT, continues to attract permanent positions with a variety of clients throughout the U.K. Currently we have many vacancies for APL technicians from juniors through to consultants with attractive salaries and interesting work content.

To find out what employment we can offer, please call Val Lusmore on 01-353-4212. If you would prefer to phone out of office hours, Val's home number is 07618-727.

APL People Limited

1990 got off to an excellent start with a bumper crop of attractive vacancies and a record number of people placed in interesting well-paid jobs! Who said APL is dying?! Signs so far this year have been very encouraging - there are more opportunities for APLers than ever before. This expansion in the usage of APL has been particularly marked in companies servicing the financial community, and there have been some exciting jobs on offer recently (with more to come hopefully!)

We are also involved in helping users of APL and AS (IBM's Application System) to implement an interface between the two (as announced recently in a 'Statement of Intent' issued by IBM). Any AS and APL users reading this who would like to know more about this interface, please contact Jill Moss on 0225 462602.

Those of you who attended a talk given for the British APL Association in February by Tim Perry and Ed Goroehowski from our sister company, Applied Production Logic, may be interested to know that we have now sold our PEFAC estimating system to a wholly-owned subsidiary of a Japanese company! Apparently there is nothing to touch it in Japan, and we are now hoping that this will open up a completely new market for us, selling software to the Japanese manufacturing industry!

Dyadic Systems Ltd

Dyadic is pleased to announce a range of APL workstations and multi-user systems based on the exciting new IBM RISC System/6000. This is an extremely powerful Unix-based system which was announced by IBM on 15th. February, and is scheduled for general delivery in June. The company can now reveal that it was invited by IBM to port Dyalog APL to a pre-production machine in January, and has successfully produced a Beta-test version of the product. Dyadic is still not allowed to publish benchmark results; suffice it to say that they were consistent with an amazing 27.5 MIPS and 7.4 Megaflops claimed by IBM for the entry-level system. Dyadic plans to complete the port in April in good time for the general availability of the hardware in June.

As an APL machine, Dyadic regards the RS/6000 as unrivalled for price/performance, and expects even greater demand than for its precursor, the IBM 6150. Prices for various configurations can be found in Vector's APL Product Guide, but start at only £11,736 for a 27.5 MIPS workstation complete with Dyalog APL.

Having completed Version 6.0 Release 2 of Dyalog APL for DOS/386, Dyadic is turning its attention to porting this version to Unix platforms. The "window-based" development environment takes full advantage of the ability of a DOS program to directly map video output. To achieve the same result on an ASCII terminal or Unix workstation, which forbids such behaviour, is a non-trivial task. Nevertheless, Dyadic is making excellent progress, and expects to deliver Version 6.0 on Unix Systems by early this summer.

In parallel, Dyadic is continuing to enhance the Dyalog APL development environment. Among other developments, the company is investigating facilities to display and edit variables in windows, and the ability to watch variables change as you trace through the execution of a function. It is intended that these new facilities be incorporated in Release 3 in time for APL90.

Cocking and Drury Ltd

Cocking & Drury have been chosen by STSC Inc as one of only three dealers worldwide to test market the new developers version of the APL*PLUS II System for the 80386. The Developers version offers unlimited use of Run Time on payment of a one-off license fee.

STSC have also announced a release date for version 3 of the APL*PLUS II system - June 15 1990. Cocking & Drury are currently beta testing this new version.

The enhancements in version 3 are likely to include a $\square$ NA Fortran interface; a Lotus import/export tool derived from the PC spreadsheet manager; mouse support; the User Command Processor; a Forms Input tool and the ability to put the Session Manager into memory above 1 Meg - this allows more memory capacity for GSS drivers, network software and TSR programs.

Cocking & Drury's pricing policy on the APL*PLUS II system remains the same - the initial purchase price includes one year's support and free upgrades. So anyone buying now automatically gets a free upgrade to version 3 as soon as it becomes available.

Cocking & Drury's first APL*PLUS Nested Array course takes place on 15/16 May - this is a new two-day course. The APL Systems Design course which Cocking & Drury has been running for several years is to be rewritten and will be broken up into a series of individual units which can be taken separately or as a full course in future.

During the last 9 months Cocking & Drury has been taking part in a Business Growth Training project funded by the training agency. This has allowed the company to devote a higher level of resources to staff training, and to make personal development a key part of employees' career development. The Training Agency funds have enabled training in Sales, Marketing, Management and Project Management skills and in new technical areas including SQL (Oracle and Ingres), C and Focus.

On a less happy note we received notification from STSC recently that support for version 8 of APL*PLUS/PC is discontinued from the end of March 1990. Cocking & Drury will continue to offer support on version 8 for a short time beyond that, though there may be particular problems requiring STSC's technical knowledge which will not be available. The only way to be sure of continuing support is to upgrade to version 9.

MicroAPL Ltd

MicroAPL has begun 1990 with a number of software upgrades. Users of APL.68000 for Mirage-based computers will have received note of the current upgrade to version 7.32, together with release notes for some new or upgraded support workspaces. We will shortly be releasing a minor upgrade to the Macintosh version of APL.68000 to accommodate some of the features of the Mac Portable.

We have also now released a new non-APL software product - Micro-TCP/IP. This is an implementation of the TCP/IP networking suite for Mirage-based computers. Micro-TCP/IP is written in 680x0 Assembler and includes all the standard support utilities. Although the product is likely to be of interest only to our Mirage customer base, it does highlight MicroAPL's extensive skills in the area of general systems programming.

Our strong support for STSC products continues and we have been shipping version 9 of APL*PLUS/PC, version 2 of APL*PLUS II for the 80386 and version 4 of Statgraphics for some time now.

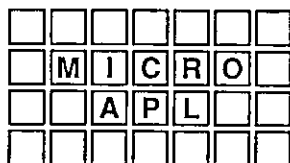
MicroAPL's training courses have been heavily booked at the beginning of 1990 and we are planning to extend the range of topics offered to include APL programming techniques and standards. We have also given a number of in-house courses tailored to our customers' requirements, and this remains a very popular option. It can actually be cheaper than going to a scheduled course if more than three people are planning to attend the same course.

For more details, contact David Eastwood at MicroAPL.

MicroAPL for ...

APL and networking consultancy

MicroAPL has 10 year's experience with small-computer APL systems. We have pioneered the usage of networked microcomputers and can plan, specify, install and maintain PC networks of all sizes. We have experience with both Ethernet and Token-ring networks and their mainframe gateways. Our programming skills cover all areas of APL and systems programming. MicroAPL can also train and supervise your APL staff on your behalf. If you would like to review your systems requirements, contact David Eastwood at:



MicroAPL Ltd.
South Bank Technopark
LONDON, SE1 6LN
Telephone: 01 922 8866

MicroAPL for ...

APL on the widest range of micros

STSC's APL*PLUS Release 8

£425

STSC's APL*PLUS II PC

£1395

APL.68000

APL.68000 for the Mac, ST, Amiga

£86.91

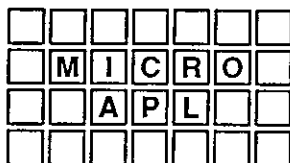
APL.68000 for the Mac II, SE30

£260.82

APL.68000 runs on many multi-user computers, including:

Sun, NCR, Wicat, HP200/300, Spectrum, Aurora, Stride £1200 to £2000

All prices quoted exclude VAT



MicroAPL Ltd.
South Bank Technopark
LONDON, SE1 6LN
Telephone: 01 922 8866

Access and Barclaycard
Orders accepted

THE EDUCATION VECTOR

April 1990

Editor Alan Sykes

This Education Vector has been reprinted from VECTOR Vol.6 No.4. VECTOR is the Quarterly Journal of the British APL Association; for more information about the British APL Association, contact the Secretary: Graham Parkhouse, Dept. of Mech. Eng., University of Surrey, GUILDFORD.

Contents

Editorial	Alan Sykes	18
Communications		19
Doing Useful Things with Strings	Alan Sykes	24
A Selection of Affordable APL Systems		30
Questionnaire	Alan Sykes	31

Editor:
Dr Alan Sykes,
Dept of Management Science and Statistics,
University College of Swansea,
Singleton Park, SWANSEA SA2 8PP

Editorial

Welcome to another edition of Education Vector! Although it only appears quarterly, it has a nasty knack of coinciding with some important academic deadline, so that I end up producing copy in great haste. (I sit typing this editorial when I should be packing my bag to attend the International Conference on Software Engineering.) I console myself with the plain fact that most people these days seem to operate a juggling act with so many balls on the go that dropping one seems inevitable. Despite these sentiments, I think there is something of interest for all readers of Education Vector and I am constantly heartened by the letters I get from far and wide, some of which have found their way into this edition. Please do keep those letters coming.

In my professional capacity as an academic statistician in the University of Swansea, a significant percentage of my time is spent using APL in my own research, teaching statistics using APL, and to some extent teaching APL itself. Our students use it in their first year to do their statistics practical, and to submit assignments for marking by the computer. In their second year, they may take as an option, a course in statistical computing using APL. Third year courses then assume that students can proceed to do whatever computing is necessary to complete their statistical assignments. A growing number of students opt to learn APL, even though many of them have been inculcated into the art of FORTRAN programming in their first year.

It comes as some surprise to them when we mention the possibility of pursuing a career in APL. At a recent recruitment drive held in the college our students were introduced to the wide range of job opportunities that exist in firms such as British Airways, banks such as National Westminster, Insurance firms (e.g. Commercial Union), Operations Research departments (e.g. Rowntrees of York). A significant feature of such job opportunities is that rapid promotion is possible with early development of careers into APL consultancy and management roles.

I mention this as it seems to me a significant potential spin-off of using APL in education. There are significant career prospects for bright students with a broad spectrum of abilities in mathematics, statistics, economics, business studies, and management science plus APL programming expertise because such people are often able to produce innovative solutions to important problems quickly and efficiently. In the last two years, three of our students have started their careers as consultant programmers in APL. It will be interesting to monitor their progress and the extent to which their chosen career path is followed by other graduates.

Communications

Thanks to Dr Dietrich Trenkler from the University of Osnabruck in West Germany. Dr Trenkler teaches courses in APL and has written the following functions to aid the understanding of just how the APL interpreter performs calculations such as ENCODE, DECODE, REDUCTION and SCAN. He supplies the following examples and the necessary code.

```
'÷' REDUCTION 18
÷/1 2 3 4 5 6 7 8 ↔ 0.2734375
1÷(2÷(3÷(4÷(5÷(6÷(7÷8))))))
```

```
'÷' SCAN 18
÷\1 2 3 4 5 6 7 8 ↔ 1 0.5 1.5 0.375 0.3125 2.1875 0.2734375
1 ↔ 1
0.5 ↔ 1 ÷ 2
1.5 ↔ 1÷(2 ÷ 3)
0.375 ↔ 1÷(2÷(3 ÷ 4))
1.875 ↔ 1÷(2÷(3÷(4 ÷ 5)))
0.312 ↔ 1÷(2÷(3÷(4÷5÷ 6))))
2.187 ↔ 1÷(2÷(3÷(4÷(5÷(6÷ 7)))))
0.273 ↔ 1÷(2÷(3÷(4÷(5÷(6÷(7÷8))))))
```

```
16 16 16 ENCODE 500
16 16 16 T 500 ↔ 1 15 4
(1×16×16)+(15×16)+4
```

```
0 16 ENCODE 500
0 16 T 500 ↔ 31 4
(31×16)+4
```

```
0 24 60 60 ENCODE 94863
0 24 60 60 T 94863 ↔ 1 2 21 3
(1×24×60×60)+(2×60×60)+(21×60)+3
```

```
0 24 60 60 DECODE 1 2 21 3
0 24 60 60 1 1 2 21 3 ↔ 94863
0 1 2 21 3
24 1 24 0 0
60 0 26 1560 0
60 0 0 1581 94860
0 0 0 0 94863
```

This is just Horner's rule describing how the result is computed. The first row (column) excluding the first 0 are the arguments of $\perp$. Read row entries as 'multiplied by' and column entries as 'added to'. Note that most zeros in this scheme serve only as a means to build up a matrix.

The functions are necessarily complicated and I am very grateful to Dr Trenker for supplying them.

```

V F REDUCTION V;R;A
[1]  R, ' ↔ ', ∇±R+F, ' / ', ∇V
[2]  A+(-2+R+ρV)ρ')'
[3]  R+(, (' ', [1](((-2+R), 2)ρF, '('), [1] 1 2 ρ' ', F),
    0 1 +∇(R, 1)ρV), A
[4]  (' '≠R)/R
V

```

```

V F SCAN V;I;T;R
[1]  F, '\', (∇V), ' ↔ ', ∇±F, '\', ∇V ◊ ' '
[2]  (5+∇T), ' ↔ ', ∇T+V[-1+I+2]
[3]  M1:R+(-2+T+ρI+V)ρ')'
[4]  T+(, (' ', [1](((-2+T), 2)ρF, '('), [1] 1 2 ρ' ', F),
    0 1 +∇(T, 1)ρV), R
[5]  (5+∇±T), ' ↔ ', 2+T
[6]  →M1×1(ρV)≥I+I+1
V

```

```

V A ENCODE B;T;I;C;N;R
[1]  (∇A), ' τ ', (∇B), ' ↔ ', ∇N+AτB
[2]  I+ρN ◊ R+' ' ◊ A+1+A
[3]  LOOP:→FINISH×10=I+I-1
[4]  R+R, '(' , (-1+ , (∇N[1], [1](I, 1)ρA), '×'), ')+ '
[5]  N+1+N ◊ A+1+A
[6]  →LOOP
[7]  FINISH:( ' '≠R)/R+R, ∇N
V

```

```

V R+A DECODE B;T;I;C
[1]  (∇A), ' ⊥ ', (∇B), ' ↔ ', ∇A⊥B
[2]  A+1φ(ρB)ρA ◊ T+0 ◊ R+(0, ρB)ρI+1
[3]  M1:C+((I-1)ρ0), T+T+B[I] ◊ R+R, [1]C, (T+T×A[I]), ((ρB)-I+1)ρ0
[4]  →M1×1(ρB)>I+I+1
[5]  R+(0, A), B, [1]R, [1](-ρB)+T+-1+B
V

```

Mr C.E.Williams from Loweswater wrote to Anthony Camacho concerning some problems he has had with I-APL. First Mr Williams makes an appeal for help with understanding APL, pointing out that it takes a long time for a letter to permeate through Education Vector back to the author, making rapid progress difficult. The point is well taken and if any reader wishes faster relief from the inevitable niggling problems, then I for one would be more than happy to receive a telephone call - which I would do my best to answer on the spot. So here is the telephone number - 0792 201776. Any other offers?

Mr William's particular problem concerned using DECODE to evaluate the values of a polynomial. I have checked the stated problem and I was able to replicate it with my version of I-APL - namely that DECODE will not accept a matrix left argument - for example $(2 \ 1 \ 4 \ 3) \cdot 2 \ 10$ should yield the vector $18 \ 16$, corresponding to the separate calculations $4 \cdot 2 \ 10$ and $3 \cdot 2 \ 10$. This is rather unfortunate, and I hope it has not discouraged Mr Williams from using APL, particularly for use with electrical circuit theory - I wonder have you read Bob Spence's book on Electrical Circuit Theory using APL? (Bob Spence is Professor of Electrical Engineering at Imperial College and his book is a superb example of how APL can be used as an educational tool to aid understanding.)

Mr Williams also makes comments about the conflicting messages within the pages of VECTOR concerning styles of programming. (Should we or should we not use parentheses and the diamond separator?) I would agree that it is confusing to the beginner to have such contradictory advice. Nevertheless it is probably impossible to avoid it! The only golden rule is to pursue such practices as make it easier for you to produce correct code that is readable to others and to yourself at a later date.

My final letter, from Norman Thomson, (my predecessor as Education Officer) shows only too clearly that submitted code will always be open to criticism by others. And that is how it should be. Please, please do not stop sending me your contributions in whatever style of programming you think appropriate!

The Card Collecting Problem

by Norman Thomson

Whilst wishing to express general pleasure at Claude Henrion's contribution from Switzerland (VECTOR 6.3) I feel compelled to express in somewhat stronger terms than Alan's footnote the view that this sort of code is *NOT* the way to promote APL in education, and indeed provides ammunition for its critics.

Look again at the offending line:

```
R←+/v\p∇f<\(1C)•.=?(N,11×C)ρC
```

In the first place the program cannot be guaranteed to work if chance delivers a run of exceptional length (*editor's comment - I reckon that the probability is in excess of .9999 for all C - any offers for a formal proof using APL of course?*). In the second, reading it from left to right as one should with all good APL code, requires a string of six functions to be comprehended before any data is reached. It is rather like encountering a mathematical expression like ...

```
log(2+cos(exp-arctan(...)))
```

... for which little feeling for what the expression does remains by the time the argument is reached.

By contrast, a direct definition approach offers an excellent solution. Let *DRAW* describe the process of obtaining a full set of cards, given one starts with the card sequence ω . At each step, test whether ω contains a full set (1α) - if so stop and return the number of cards $(\rho\omega)$, if not obtain a new card, add it to the sequence and repeat. This leads to the function:

```
DRAW: DRAW (1α),ω:∧/(1α)∈ω :ρω
```

```
6 DRAW 10
```

13

Simulation consists of repeating this process a prescribed number of times $(1+\alpha)$ - for good measure we will additionally calculate the average on completion:

```
SIM: SIM((-1+α)DRAW 10),ω:(1+α)≤ρω:ω,(+/ω)÷ρω
```

```
5 6 SIM 10
13 19 8 12 11 12.6
```

If complete compatibility with Claude's function is required, define:

```
CARDS: (α,ω)SIM 10
```

We thus have a solution which relates to the problem in an obvious way, which doesn't require the best bit of a page to explain it, which works in all cases, and which incidentally runs 7-8 times faster in I-APL than Claude's APL trickery (the sort of thing I have to confess I have all too often wallowed in myself).

I believe that the issue raised here is important in discussing APL in education, and please Claude do continue to make your valued contributions to VECTOR - this letter is in no sense an attack!

Yours sincerely

Norman Thomson

Doing Useful Things With Strings

by Alan Sykes

Introduction

Just in case you think I have a one-track mind, my article this edition concentrates on the manipulation of strings. APL deals with arrays either of numbers or of characters. Many of the APL primitives apply equally well to arrays of characters as arrays of numbers, excluding the arithmetic ones of course. But is it useful to consider manipulating arrays? In the last edition of VECTOR my fellow statistician Tony O'Hagan demonstrated very cleverly one such application - to generate program code for the computation of a wide range of similar problems that could be described by a formula (string).

My contribution is not so esoteric, but nevertheless useful in helping newcomers to APL to understand some of the things that can be done with strings. One such situation is that of importing data from a file external to APL.

Manipulating an ASCII File

Even if my concern is still with data analysis, frequently the data of interest may come from a computer file external to APL. All APL interpreters give the user the ability to read in an ASCII file, though they tend to have different ways of doing it! Let's suppose that we have managed to get such a (small) file into APL as a text matrix. What must we do to convert it into numbers?

```
      ASF
-1.2  2.1  3.4
 1.2  3.1  2.4
 1.3 -3.4  5.6
 4.7  3.5  1.2
```

```
      pASF
4 16
```

As you can see, *ASF* is clearly not what it looks - a 4 by 3 array of numbers but a 4 by 16 character matrix. Also note that it includes the usual 'middle of the line' minus sign to indicate those numbers that are negative. So we have to convert such signs to the APL high-minus sign, and then 'execute' i.e. convert to numbers

using the APL execute symbol $\leftarrow$. Since this will only work on a vector, the matrix must be converted into a vector using $''$. In doing so we make sure that there is at least one space on the end of each line, so that the end of one line does not merge into the beginning of the subsequent line.

Let's do this in easy stages. First convert those signs.

```
A←,ASF,' '
```

This catenates one column of spaces onto the right of *ASF*, and then converts to a vector.

```
A[(A=' ')/⊥ρA]←' '
```

The vector $A=' '$ is a boolean vector indicating where the ' ' signs are in *A*, which is used to reduce the integers 1 2 3 4 to give the positions in *A* where they occur.

```
A←⊥A
```

This is all that is required to convert the string *A* into the vector of numbers portrayed by *A*. This can then be dimensioned to give a numeric matrix.

```
A←4 3ρA
```

Thus we may usefully write this as a utility function called *ASCTONUM*:

```
▽ RES← DIM ASCTONUM ASF
[1]  ASF←,ASF,' '
[2]  ASF[(ASF=' ')/⊥ρASF]←' '
[3]  RES←DIMρ⊥ASF
▽
```

Boxing a Set of String Vectors

It is very often useful to have a set of names in APL as a character matrix. If the names are not all of the same length, then we need to be able to pad out the names to the longest length using blanks - an operation usually referred to as making a *BOX*.

The key to the solution of this problem is knowing how to replicate values of a vector. This is demonstrated by the following examples.

```

      1 2 3/9 8 7
9 8 8 7 7 7

      1 0 2 0 1 1/'A*P*LE'
APPLE

```

In each case, a vector of non-negative integers of the same length as the source vector is used to say how many elements of each are to be used. So how do we use this facility for boxing?

Let's suppose that we are faced with the string of names ALAN,ALBERT and ADA. We will separate the names using the character '/' ...

```
X←'ALAN/ALBERT/ADA/'
```

Note that we have used the separation string as the last character - any other character that is not in the names could be used. First we find out just where the separators occur, and replace them by blanks:

```

      I+(X=⌊1+X)/⌊pX      (Remember ⌊1+X is the last element of X)
      I
5 12 16

```

```

      X[I]←' '
      X
ALAN ALBERT ADA

```

Now we find out how long the names are (vector J), the maximum length (M) and hence the number of spaces required to pad each out to the longest (K):

```

      J←~1+I-0,~1+I
      J
4 6 3

      M←⌈/J
      M
6

      K←M-J
      K
2 0 3

```

If we now make a vector, most of whose elements equal 1, except those elements corresponding to spaces (given by I), which we make equal to the elements of K , then this vector may be used to replicate X :

```

      REP←(ρX)ρ1
      REP[I]←K
      REP/X
ALAN__ALBERTADA___      (The underscores indicate spaces.)

```

Hence we have ...

```

      3 6ρREP/X
ALAN ALBERT ADA

```

We pull all these elements into a function called *BOX* as follows:

```

▽ RES←BOX X;I;J;K;M;REP
[1]  ⌘ Find out where the separators are
[2]  I←(X=~1+X)/⌈ρX←,X
[3]  ⌘ Replace them with spaces
[4]  X[I]←' '
[5]  ⌘ Calculate the length of each name J
[6]  ⌘ and the maximum length M
[7]  ⌘ and the number of spaces required to pad out
[8]  J←~1+I-0,~1+I
[9]  K←(M←⌈/J)-J
[10] ⌘ Construct a vector for using with 'replicate'
[11] REP←(ρX)ρ1
[12] REP[I]←K
[13] ⌘ Replicate and convert to a matrix
[14] RES←({ρI),M)ρREP/X
▽

```

Right-Justification

Whether you like your text right-justified or not is a matter of taste. Conventional word-processors normally incorporate a 'reformat' procedure which takes the jagged text and neatly puts it into lines of a given length. How might one tackle this in APL? The problem is a similar one to the previous example involving placing extra spaces adjacent to existing spaces so that on re-shaping, the resulting matrix has no gaps at the end of the line, and words are not split. As newspaper readers will know, this can lead to serious problems for small columns (such as splitting 'legend' to become 'leg-end'). My attempt at right-justification will not attempt to be sophisticated enough to cope with small column widths in general, though we use a small amount of text as an illustration.

X←'EDUCATION VECTOR ARTICLES ARE DESIGNED FOR BEGINNERS IN APL'

is our text vector, and suppose the line length required is 30. As before, we find out where the blanks occur (*J*), and the position of the last blank before the line length (*K1*). In addition, we find the last non-blank before *K1*: *K2*.

```

LL←30
I←X=' '
J←I/∖ρI
J
10 17 26 30 39 43 53 56
K1←J[+/J≤LL]
K2←1+K1-(ϕ(K1+X)≠' ' )∖1

```

This last line requires some explanation. The vector *K1+X* consists of the first *K1* elements of *X*, and hence *(K1+X)≠' '* returns a boolean vector corresponding to all the non-blank elements. The character *ϕ* reverses the order of the elements of its vector, and *V∖1* finds the first occurrence of 1 in vector *V*, hence *K2* is as required.

So we can now calculate the number of extra spaces required - namely *NS+LL-K2*. We then have to decide how to apportion the extra spaces *NS* amongst the number of blanks *NJ*. A whole range of different strategies are possible - I decided to construct a vector of length *NJ* which consists of integers as near equal as possible, with the higher values at the beginning. The values used are therefore equal to *M* and *M+1* where *M←⌊NS÷NJ*, and the number of values of *M+1* is given by *N+NS-M×NJ*. To right-justify the first line, all we need to do

now is to construct a vector *REP* to replicate the elements of the vector *X* the required number of times.

So what about the rest of the vector? If we allow our function to be recursive, then at this stage we can just apply the function to the rest. To do this, we must check initially the vector is greater than the length of line *LL* - if it isn't then there's nothing to do except make sure that the text is a matrix so that lines may be appended to the top.

The ideas expressed here are incorporated into the function *RJ* with left argument equal to *LL* and right argument the text vector *X* to be right-justified:

```

V R+LL RJ X;I;J;K1;K2;REP;F;NJ;M;NS;N
[1]  A Check for length being greater than one line
[2]    +(LL<ρX)/ct
[3]    R+(1,LL)ρLL+X ◊ +0
[4]    ct:I+X=' '
[5]    A K1 is the position of the last blank in the first line
[6]    A K2 is the position of the last non-blank char before K1
[7]    J+I/ρI ◊ K1+J[+/J≤LL]
[8]    K2←1+K1-(ϕ{K1+X}≠' ')\1
[9]    A Reduce I to len K2 and find posns J and no NJ of blks
[10]   I←K2+I ◊ NJ+ρJ+I/ρI
[11]   A Calc the number of blanks F to be added at each J
[12]   NS←LL-K2 ◊ M+⌊NS÷NJ ◊ N+NS-M×NJ
[13]   F←(N,NJ-N)/ 1 0 +M
[14]   A Make vector for replicating blanks in 1st line to LL
[15]   REP+(ρI)ρ1 ◊ REP[J]+REP[J]+F
[16]   A Repl current line and appd to same treatment of rest
[17]   R←(REP/K2+X),[1]LL RJ K1+X
V

```

25 RJ X

... as an example

EDUCATION VECTOR
ARTICLES ARE DESIGNED
FOR BEGINNERS IN APL

Of course, there's much more to a proper facility to right-justify text. Carriage returns in the text indicate that the contents of that line should not be right-justified, and in addition, unnecessary spaces in the text should be removed before right-justification is applied. This suggests 'an easy exercise for the reader' (as one of my Maths professors used to say). How can unnecessary blanks be removed from a text vector? If you feel like experimenting, please write to Education Vector with your results!

A Selection of Affordable APL Systems

This section is an extract from the complete Vector APL Product Guide. It lists APL systems and literature which are either public domain (if you can find them) or commercially available at reasonable prices.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
I-APL Ltd	I-APL/PC or RML Nimbus	4.50	ISO conforming interpreter. Supplied only with manual.
	APL/BBC	4.50	As above
	I-APL/Archimedes	4.50	As above
IBM	Try-APL2	free	Non-commercial interpreter, with 20-page instruction sheet and sample APL2 workspaces.
MicroAPL	QL/APL (keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL (APL chars)	87	VSAPL compatible APL for QL with many extra features.
	APL.68000 Amiga	87	Full APL interpreters with support for windows, mouse, graphics etc.
	APL.68000 for Atari ST	87	
I.P. Sharp	Sharp APL/PC V2	40	shareware for IBM PC/XT

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL-385	Try-APL2	5.00	To cover copying and postage, if you can't find someone to give it to you!
I-APL Ltd	An APL Tutorial	2.50	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2nd Edn)	5.50	226pp by Helzer
	APL in Social Studies	2.50	36pp by Traberman
	I-APL Instruction Manual	2.50	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	14.50	185pp by Thomson

** Please add one pound packing charge per order **

Renaissance Data Booksellers The widest range of APL books available anywhere.

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
I-APL Ltd	Anthony Camacho	2 Bienenheim Road, St. Albans, AL1 4NR. Phone 0727-60130 for queries, order forms, bulk orders
	J C Business Services	56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU N.B. PAID ORDERS ONLY
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 01-922 8856
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A.. Tel: (212)864-3078
I.P.S.A. Ltd.	Mike Butler	7th Floor B Block, Coventry Point, Market Way, Coventry CV1 1EA Tel: 0203 256562

Education Vector Questionnaire

At present Education Vector is mailed as a separate item to well over 1000 people who are currently on the I-APL mailing list. It also forms a section in the quarterly journal of the British APL Association called VECTOR. This journal contains information and articles on all aspects of APL, and is mailed to all members of the Association for an annual membership fee of £10.

The British APL Association is committed in principle to Education Vector and its aim of supporting newcomers to APL in schools, colleges and also in industry. Nevertheless it has to consider ways of meeting the substantial cost of postage and package to an increasing number of readers. Please help us by answering the short questionnaire below and returning the completed form.

Education Vector Questionnaire

1. Would you like to continue receiving copies of Education Vector?

Yes/No

2. Would you like to become a member of the British APL Association for an annual fee of £10 (£5 only for students) and receive Education Vector as part of the quarterly journal of the Association?

Yes/No

3. If you do not wish to become a member, would you be prepared to contribute to the cost of postage and package of Education Vector, by paying an annual subscription of £2?

Yes/No

4. Is the present form of Education Vector Good / Helpful / Unhelpful / Unsatisfactory? (Constructive comments and criticisms welcome!)

Please forward your completed questionnaire to the Editor, at: Education Vector, Department of Management Science and Statistics, University College Swansea SA2 8PP



Renaissance Data Systems
Enlightenment Thru Information Processing
ALL APL BOOKS IN PRINT
Catalog Excerpts June 1, 1989



If we do not carry it (in English, at least), we dare you to find it elsewhere!
Specializing in books on:
APL AS A TOOL OF THOUGHT!

ACCOUNTING STRUCTURED IN APL , Iljiri. 1984, 491p.	\$10.00
The basic principles of accounting with numerous APL expressions to model them.	
ABSTRACT ALGEBRA - A COMPUTATIONAL APPROACH , Simms. 1984, 491p.	\$55.95
An introductory course in the basic concepts of algebra and fundamental algebraic algorithms. A methodology for investigating computational questions. Includes lemmas, theorems, and corollaries.	
COMPUTING IN STATISTICAL SCIENCE THRU APL , Anscombe. 1981, 426p.	\$39.00
Lots of statistics and APL. His workspaces are available from Yale University. A classic.	
MATHEMATICAL EXPERIMENTS ON THE COMPUTER , Grenander. 1982, 525p.	\$59.50
Case studies using APL in a number of fields, including statistics, linear algebra, geometry, asymptotics, neural networks, invariant curves. Detailed description and analysis of APL functions in all topics discussed.	
PROBABILITY IN APL , Alvord. 1984, 142p.	\$19.50
A delightful, yet serious development of 31 functions for fun and learning with roll, deal, binomial distributions, combinations, permutations, geometric distributions, and the World Series. Friendly examples. No previous APL required.	
Diskette for above containing functions covered in book.	\$22.00
APL - THE LANGUAGE AND ITS ACTUARIAL APPLICATIONS , de Kerf, Goovaerts, Stiers. 1987, 223p.	\$89.00
Introduction to APL. Loss reserves, credibility, probability, numerical analysis, forecasting, with APL functions.	
COMPUTER GRAPHICS FOR DESIGNERS AND ARTISTS , Kerlow and Rosebush. 1986, 298p.	\$44.95
A well organized, beautifully done reference book. Step-by-step explanations of terms and techniques from bits and bytes to 3-dimensional imaging. Many explanatory illustrations, most in color. No APL, but, says Rosebush, APL thinking and data structures are implicit throughout.	
APL2 AT A GLANCE , Brown, Pakin, Polivka. 1988, 444 p.	\$29.00
Solid, unimposing, introduction to APL2. Clear illustrations, modern exercises in each chapter. Gets you started.	
THE APL IDIOM LIST , Perlis and Rugaber. 1977, 50p.	\$ 6.50
Funded by Mobil. Written at Yale. Very terse, but includes nuggets of gold.	
APL - ADVANCED TECHNIQUES AND UTILITIES , Berquist. 1986, 450p.	\$44.95
Good discussion of alternative approaches to a wide range of programming tasks - efficiency, searching, dates, workspace documentation, file design, boolean techniques, financial utilities. Many idioms.	
Diskette for above.	\$15.00
Contains all functions described in text. Formatted for APL*PLUS/PC, uploadable to Sharp APL, VS/APL, APL2.	
APL AS A TOOL OF THOUGHT, PROFESSIONAL DEVELOPMENT SEMINARS, NY/SIGAPL	
They cover a wide range of topics in business and education. 1987 - logic, insurance, statistics, A. I., accounting, fractals, linked lists. Other years: Teachers toolbox, computer science, biology, graphics, engineering, data bases, and more.	
1984, 1985, 1986, 1987. 120 to 150 pages each (1983, 47p. \$10.00)	each \$15.00
All five as a package.	\$55.00

Name: _____	Date: _____	Please make check or money order payable in \$U. S. and send with this form to: Renaissance Data Systems P. O. Box 20023 Park West Finance Station New York, New York 10025-1510 U.S.A. (212)864-3078	
Street: _____			
City: _____	Prov./State: _____		
Country: _____	Postal Code: _____		
Telephone Day: _____	Evening: _____		
Title _____	Price _____	Quantity _____	Total _____
Title _____	Price _____	Quantity _____	Total _____
Title _____	Price _____	Quantity _____	Total _____
Subtotal: _____			
Postage and handling - U.S. and Canada (\$2.50 + \$2.50 each item over \$25.00): _____			
- International (surface mail add 15%, airmail add 50%): _____			
NYC Residents please add 8 1/4% sales tax, NY State, 4%: _____			
Please send complete CATALOG of 64 Titles: ☐ Total: _____			

APL90

The International Conference on APL
13 - 17 August 1990
Falkoner Center, Copenhagen, Denmark

Welcome to APL90

Some 17 years after the first APL conference was held in Denmark, we cordially welcome both old-timers and new hands to APL90. We can truly say that, in line with tradition, APL90 will be an exciting opportunity for exchange of both ideas and experience within the world of APL. As something new, however, the complete Proceedings, containing all the papers and abstracts, will be received prior to the conference by those who register before the early dead-line. This will allow participants to prepare for the more than 50 oral and poster presentations, as well as the tutorials, birds-of-a-feather sessions, software exchanges, and vendor forums and exhibitions. The Program Committee is convinced that everyone will be kept busy!

The main theme of the conference is 'For the Future' and many of the presentations and activities will indicate potential directions for new developments. Both as a result of an 'internal drive' to improve APL, but also - and obviously - inspired by and competing with developments in other programming environments. Plotting a course towards the future, however, often involves mapping the present and a number of the presentations will document results already obtained through the use of APL.

Some of you may have noticed a small change of plans. The week before this invitation went to the printer, our first choice of conference centre closed down, and although they are threatening to reopen "soon" we felt it was safest to move the conference to Copenhagen - to the Falkoner Center only 3 km. from the centre of Copenhagen. We believe that the new venue will prove conducive not only to the pursuit of the aim of the conference, but also allow this to take place in a friendly and relaxed atmosphere. Through the choice of presentations and other activities we are convinced that everyone will leave a little wiser, with plenty of food for thought, inspiration and, hopefully, with lots of good memories.

Gitte Christensen, Per Gjerloev
Conference Chair, Programme Chair

Conference Overview

A very important part of our motivation for taking on the task of organizing APL90 is that we find APL conferences to be of great personal and professional value. There are very few opportunities for APL users and programmers to educate themselves, except by meeting and talking to other APL'ers. The annual APL conference provides this opportunity which we so sorely need. This is where we can catch up on the latest advances in software and hardware, and the place where we can meet people who, like ourselves, use APL to solve problems. It is an event from which we always return full of new knowledge, energy and inspiration.

APL conferences have a tradition which must be respected - but each organizing committee is allowed to try out some new ideas. Our primary goal has been to show that APL is a successful tool, and that it will continue to be so in the future. Fortunately, the submitted papers, together with the invited speakers, have delivered some solid documentation to this effect.

Another important goal which we set ourselves from the start was not to force attendees to choose between keeping up with the technical programme and living a social life which provides some of the best opportunities for gathering information and making new friends and business contacts. All participants registering before June 1st will receive a copy of the proceedings before August 1st. We have also set Wednesday afternoon aside for meetings. This means that you can schedule business meetings, Birds-of-a-feather sessions etc. without having to miss part of the conference programme.

We have chosen to give so-called "Poster Sessions" a more prominent role at APL90. These sessions are particularly appropriate for presentations suited to a dialogue between the author and his audience. We have encouraged Software Exchange contributors to perform short presentations of their works in order that it be easier to get an idea of what is being offered.

Finally, we have invited prominent proponents of each of the two main APL dialects to give advanced tutorials, aimed at educating those who believe in the other dialect. An understanding of the actual differences between these systems is a requirement for a fruitful discussion of the future development of APL. An attempt to kick off such a discussion will be made by Mr. Trenchard More who will be expounding upon the latest improvements to his array theory.

We hope that you will enjoy our version of the APL conference - APL90 - and look forward to seeing you all in August.

Invited Speakers

Mads Oevlisen: People, Corporations and Information.

Mr. Oevlisen is the President of Novo Nordisk A/S, a Danish biotechnology corporation. How does he envisage Management Information and Information Management in the future, in a research-based and market-driven environment?

Hans-Peter Meinzer: Visualization of medical tomography image series.

Image processing is a computationally intensive task. Dr. Meinzer will not only talk about but also show you how data, harvested from medical scanning procedures, can be analyzed and transformed into very useful three-dimensional images with high informative content, thus aiding physicians in diagnosis and therapy. And, of course, how this has been done with APL.

Trenchard More, Jr.: Looking Beyond APL2 and NIAL.

Mr. More is now retired from his former position as research fellow with IBM. He is the father of the array theory which laid the foundation for nested arrays in APL2. A later version of the array theory was used as a basis for the Nial language. For some years, Mr. More was a visiting professor at the Technical University of Copenhagen, where students of Ole Franksen and Peter Falster have done pioneering work using Nial. More recently, Mr. More has had further insights, particularly with respect to the handling of empty arrays, leading to new revisions of array theory. His address will deal with the consequences of these new insights - the differences between APL2 and Nial, and some ideas about what should be next.

Allan Huang: Massively parallel computing.

Mr. Huang is working as chief scientist with AT&T, Bell Laboratories. The most recent results of his work include the prototype of the world's first optical computer, which runs between 100 and 1,000 times faster than the supercomputers of today. The optical computer is a massively parallel machine, and to help make efficient use of the new architecture, Mr. Huang has developed a concept which he calls 'computational origami'. This is a process by which a large problem with a vast amount of data can be ordered so that it can be computed efficiently by a parallel computer.

Baird Smith: The Soft world.

Mr. Smith of IBM gives a multimedia presentation showing the impact of computers on everyday life. The presentation provides insight into just how deeply our lives are influenced by the use of modern information technology, as well as indicating some of the directions future developments may take.

Mark Dempsey: APL software engineering and CASE, where is the common ground?

For many years, Mr. Dempsey worked on software development tools at I.P. Sharp Associates. He was one of the main architects behind LOGOS - IPISA's APL development environment. Now working for Affinity Systems, Mr. Dempsey will give us a survey of the world of software engineering and CASE.

Poster Sessions:

Jan Ahlqvist
Tony O'Hagan
Don Erickson
Frank Evans

Sven Gunnar Loenn

D. Smellie
Adrian Smith
Howard J. Smith, jr.
Howard J. Smith, jr.
Steve Dunwell

SRS Service System
APL Statistics Library Project
Technical Support Program for APL-related questions
A Structured Approach to Analysis and Design of
Complex Systems
DEMOS - a PC-system for population projections for
small areas
Structured Expert System Design
Designing an APL Typeface
What's Ahead for 2000 A.D.
Some Uses of Truncated Boolean Vectors in Analysis
An APL monography

APL90 Papers

The following is a list of abstracts from the 47 accepted papers for oral presentation at APL90. The papers have been organized into 8 streams of related subjects.

STREAM 1: APL FOR THE FUTURE

These presentations deal with the properties of APL which should secure it a firm position in future data processing, but also with practices necessary to reach this goal.

Kromberg, Morten and Gfeller, Martin: An Application Development Platform. While APL was a highly productive system for personal computing and application development when it was introduced, it is lacking nowadays in the areas of programming-in-the-large and user interface construction. The design and implementation of a new Application Development Platform (ADP) is described. The ADP frees the programmer from user interface concerns, and frees the application from device specific code.

Promisel, Steven and Merrill, James: Managing a Diamond Jewelry Manufacturing Business Using APL. We discuss the development of several interconnected computer systems designed and built for a manufacturer of finished jewelry. Our approach will be two-fold. We describe how the company uses APL-based computer systems to analyze pricing issues and handle the manufacturing process. We show how the system has been developed by describing what techniques have employed and what tools have been created and used. The purpose of this paper is to show how an APL application development system can be used to build continually evolving

interactive computer systems in a combined sales and manufacturing environment.

Recker, Ursula and Rys, Michael: A Preferable Look - APL in Window-based Environments. Currently, most APL systems are line and workspace oriented. Some concepts will be shown on how a modern style APL dialect named AIDA could fit into a real window-based, event driven environment. To achieve this goal, a future user interface in an experimental environment is sketched by presenting the language environment, the editor and the debugging facility.

Rudd, Jack: Toward a Common Prototyping Language. The paper describes a proposal to the U.S. Defense Advanced Research Projects Agency (DARPA) for the development of a very-high-level Common Prototyping Language. The proposal suggests using a subset of APL2 together with an APL2-to-ADA Translator. Although the proposal did not win the first phase of the contract, DARPA returned positive feedback and invited the authors to continue their work and compete in the next phase of contract activity. Together with excerpts from the proposal to DARPA, the paper describes the respects in which the proposal fell short of DARPA's requirements, and how an APL-based solution might yet be brought into conformance with these requirements.

Schutz, Charles A.: Writing Applications for Uniform Operation in a Mainframe or PC: A Metric Conversion Program. A metric

conversion program was developed to convert 350 measurements units between inch-pound (or USA customary) and metric systems for engineering design and documentation. The program follows the primary national metric standard with its conversion factors and special rules for arithmetic, rounding, accuracy, formatting, and terminology. The portability of APL2 under VM/CMS and PC DOS was exploited in moving the mainframe program to the PC. Cover-functions to isolate environmental differences are described for file operations, pop-up windows, printing, stack and host system commands.

Souls, Donald: Stability in a Sea of Change. Question: As a developer, how do you continue to productively develop business applications when the hardware/software environment changes twice each year? We describe here a solution for the APL business applications developer to maintain a productive environment under these circumstances, by isolating the user interface in the application from the underlying workstation hardware. This is parallel to IBM's SAA strategy of a common user interface, only at the developer's, not the user's level.

STREAM 2: THE LANGUAGE AND ITS THEORY

In this group are found the presentations dealing with the theoretical aspects of APL or, in a sense, its very nature.

Benkard, Philip J.: Nonce functions. Data, functions, and operators exist in diverse current APL systems in primitive, derived, and defined forms. Data, in addition, occur in a literal form. This paper introduces a form of function literal, subsequently extended to include operators. Lexical scope is restricted to the minimum to achieve perspicuousness and syntactic predictability. The short expressions which result form convenient, explicit representations of ordinary structural operators and combinations thereof. The trade-offs between extra key strokes and symbol clutter are discussed.

Girardot, Jean Jacques: The A+ Programming Language, A Different APL. This paper introduces A+, which is an Array Oriented programming language derived from APL, and explains the motivation for creating such a language. After discussing some problems of current APL system, like the lack of control structures, the usage of dynamic binding, and the difficulty of handling functions we demonstrate how these problems are solved in A+. The most important feature of A+ is that it uses lexical binding rather than dynamic binding as APL does, which solves some old problems of APL while making it a safer language, at the same time offering a new kind of programming style.

Girardot, Jean Jacques: Arrays and References. Generalized arrays in APL have remained a controversial subject. Although major APL vendors have long settled their positions, it seems legitimate to reopen the old debate since the creation of an extended standard for APL is now being undertaken. An analysis of both nested and boxed array systems, in the light of the new developments in APL, consisting of the introduction of a new data-type, shows the possibility of and the interest in having both systems with their own specificities in a single implementation.

Hui, Roger et al.: APL1? The definition of APL in the dictionary can be significantly simplified and enhanced by ignoring some of the inhibitions imposed by conformity with earlier APL, by using the phrasal forms hook and fork, and by more fully exploiting operators. The paper describes a version of APL simplified and enhanced in this fashion. The authors have also explored techniques for obviating special symbols, as a result APL1? will run on any terminal that provides ASCII characters. Finally, the paper describes the unusual style of C

programming used to implement APL1?, which is available as freeware on several machines.

STREAM 3: SUPERCOMPUTING

In the future, the demand for numerically intensive computing will increase markedly. The presentations in this group address some of the relevant issues.

Barnes, Robert: ACORN: APL to C on Real Numbers. A crude APL to C compiler was produced while investigating improved tools for solving numerically intensive problems on supercomputers. Although intended only as a proof of the feasibility of compiling APL for numerically intensive environments, ACORN has shown that straightforward compiled APL should be able to compete with hand-optimized FORTRAN in a number of practical applications. The advanced semantics of APL give it a performance edge over more primitive languages, because APL preserves the parallel or array nature of algorithms, allowing them to be mapped onto vector and array-oriented machines in a more direct and obvious fashion than scalar-oriented languages.

Ching, Wai-Mee: Automatic Parallelization of APL-style Programs. APL-style programs use high level primitives on arrays instead of DO-loops whenever possible. For such programs, the average size of a basic block is much larger than those in their FORTRAN counterparts. Hence, it is sufficiently profitable and relative easy to concentrate parallelization of APL-style programs on basic blocks only. The compiler exploits functional parallelism on data independent sub-expressions and data parallelism of array primitives on array elements. The compiler front end does a local data dependency analysis and emits synchronization flags at function nodes. The back end does partitioning of (assembly code) array loops. A set of low-level synchronization primitives on MVS has also been under development. This will enable running of compiled applications in parallel mode on IBM 3090 multi-processors to access the effectiveness of various scheduling methods on a shared memory model.

Sauermaier, Juergen: A Parallel APL Machine. The suitability of APL as a programming language for parallel computers seems to be obvious. Nevertheless, integral concepts of parallel APL machines are rare. At Saarbrücken University, we developed a 32-processor prototype of a parallel APL machine along with a high performance network designed for up to 256 processors. Besides building the prototype, we specified in detail in which way the parallel APL machine is operating. In this paper, we present the basic principles of the parallel APL machine together with the efficiency of the machine for the most important APL functions.

Geyer-Schulz, Andreas et al.: An APL-Simulator of Non-Von Neumann Computer Architectures. APL's powerful primitives and compactness make it an ideal tool for simulating hardware functions in order to gain insight into the functionality, the performance, and the programming of an algorithm without the need of the painstaking process of actually building the target machine. We have developed an APL2 software simulator for Non-Von Neumann computers. The basic data structure of the system is an array whose elements model a number of RAM's containing control registers, data and program code. The user can "define" the machine by specifying the communication network and primitives, the instruction set and its semantics, and the complexity measure. We demonstrate the use of the program system for studying various schemes for adaptive load sharing on multicomputers.

STREAM 4: APL EDUCATION

These presentations deal with the art of teaching or presenting APL to both beginners and old hands.

Cherlin, Edward: *APL Trivia*. APL derives great expressive power from seemingly trivial features, such as empty arrays, but there is still resistance to the implementation of the elementary trivial functions, such as left [] and right [], which perform no calculations. This paper defines functional triviality, describes trivial functions and operators and their uses, discusses the mathematical basis of their expressive power, and defines new trivial functions and operators. It urges implementation of several of these functions as primitives for reasons of efficiency and expressiveness, and considers their efficient implementation via idiom recognition.

Poliivka, Raymond: *Reading To Write*. Teaching a program language offers many challenges. Foremost, of course, is the question of how to present the material. Part of any presentation are the drills and exercises. Frequently, after the content has been presented, the student is asked to write a program. This is much akin to asking a student, who has just been exposed to the grammar and some vocabulary of a new language, to write an essay in that language. This paper will discuss several alternate approaches in presenting exercises to develop the proficiency of a student in the use of a programming language.

Thomson, Norman D.: *The Art of Staying Within One's Depth*. A barrier to the successful use of APL2 is that many functions by being inherently depth-increasing or depth-decreasing can tempt the programmer to lose control of depth and thereby forfeit the expressive power of APL2. The "split pepper" effect following a deluge of 'each's can be just as damaging as ISO APL chains of left-brackets and quotes. The temptations of a fast interactive system as a substitute for thought are considerable, and should be resisted. Informal perceptions of how APL2 functions and operators work can be more valuable than the formal descriptions given in manuals. This paper consists of 4 sections, each with rules or precepts for the disciplined and controlled use of nestedness.

STREAM 5: MANAGING APL: THE SYSTEM ENVIRONMENT

The friendliness of APL towards both implementors and users will be one of the 'critical success factors' for its use in the future. The presentations in this group all deal with some facet of this issue.

Foster, Garth H. et al.: *An Integrated Programming Environment for APL2/PC*. This paper describes how full screen editing and debugging using a mouse and windows can be added to the APL2/PC programming environment. The use of AP124 and a special purpose auxiliary processor to support windowing are described.

Fox, Ralph Jr.: *Colour APL Beautiful*. This paper discusses the many ways you can use colour display attributes to make your use of the APL*PLUS PC and APL*PLUS II systems more productive and enjoyable. An extensive suite of utility functions will be illustrated and demonstrated. These functions enable you to establish colour schemes for the editing, maintenance, and display of functions and various types of data arrays. Many of the utilities are also applicable to monochrome monitors.

Kondrashev, Andrew V.: *The Family of Soviet APL Systems*. This paper deals with the original implementation of the family of APL systems for diverse mini, micro and personal Soviet computers.

Barker, Guy et al.: *Conscientious Programming Using PMA*. This paper draws attention to the importance of application support and the need for APL software management. It describes the use of APL systems at Philips to assist in providing a controlled application support environment and examines in detail an APL software management system, PMA (Program Maintenance Aid).

Lucas, Jim: *Programming ecology, or the world at large*. APL, like any programming language, interacts with various environments and individuals. This paper examines the interactive demands of each external influence and the means by which APL has tried to meet these demands as a set of mutually interdependent systems, i.e. an ecology. It suggests some general principles for ecological success, illustrating them with examples from both natural biological systems and APL. It then proposes some future evolutionary directions for APL which should not only insure its survival but gain it a dominant position in the programming ecology. Not the least important conclusion is that APL must be responsible for its own success.

Rutiser, William A.: *Object-Oriented Programming of X Window System Graphical User Interfaces*. This paper describes an experimental interface from an APL implementation to the X Window System and an APL object-oriented programming style for creating graphical user interfaces.

Weintraub, David: *APL2OS: Design Considerations for a Nested Array File System*. APL2OS is an External Function for the APL2 system, designed to enable APL2 applications to access operating system files (and information about these files) in a straightforward and efficient way, using the power of APL2 syntax to maximum advantage. The design goals and approaches for APL2OS are discussed, in the context of a summary of its features.

Wheeler, James G.: *Design and Implementation of the Interface to Compiled Languages in APL*PLUS II*. APL*PLUS II is a second-generation 32-bit APL interpreter that runs under the MS DOS operating system. A recent release has introduced two new system functions that make it possible for APL to dynamically load and execute programs written in compiled languages such as C and Fortran. These enhancements presented both design and implementation challenges. On the design side, it was necessary to make it as easy as possible for the APL programmer to integrate non-APL routines into applications. The implementation side required considerable technical creativity and invention, since the MS DOS environment provides no built-in functions for dynamic loading and execution of programs.

Oesterman, Johnny: *Very high quality user interface and fast data filing using a PC*. The user response to any computer system depends very much on the way the user 'sees' and 'feels' the system, or the user interface. In order for APL to remain competitive compared to other PC software products it is imperative that it is easy for the APL programmer/user to exploit the full potential of the PC environment. Since data filing operations are often disproportionately time-consuming on a PC, fast file handling functions are essential. This paper introduces, through the use of a toolbox of compiled utilities, a new way to implement an efficient and attractive user interface combined with fast and flexible handling of fixed length data files.

STREAM 6: APPLICATIONS: HOW TO SOLVE PROBLEMS WITH APL

APL is being used to solve diverse but very real problems some large, some smaller. The presentations in this group

demonstrate solutions to a selection of problems which may, hopefully, provide inspiration to go on and do likewise

Bottoni, Paolo et al.: Definition of image interpretation strategies in APL. Images can be analyzed as digital arrays in which not only numeric, but also topologic, geometric, and shape properties have to be detected. The use of APL functional notation permits the easy definition of interpretation strategies to detect and manipulate these properties: A) in a concise form, avoiding house-keeping details, so that the strategy can be generally understood. B) allowing the decomposition of high-level functions into their components, highlighting substrategies. C) focusing on those activities which are fundamental to each substrategy. These properties facilitate visualization of the general algorithmic ideas underlying a given strategy, while at the same time execution of the generated code allows checking their computational meaning on real data. The concept will be illustrated through the discussion of examples.

Fris, Erik S.: APL2 and MIDI. The Musical Instrument Digital Interface (MIDI) protocol may be effectively modelled in APL2 through the definition of scalar functions. This paper demonstrates various techniques in the modelling and manipulation of MIDI data as well as other musical models for pitch and chords in APL2.

Groner, Leo and Cook, Janice H.: QMOD, an analytical response time model for distributed systems. Network designers are faced with a combinatorial explosion of choices not only among various vendors' workstations, hosts and servers, but also among application distribution strategies, communication media, communication protocols, and network topologies. To study performance tradeoffs among various designs, the authors have developed a generic system throughput and response-time model for distributed systems. We have applied the model to actual customer networks. Since the package is designed with an open set of submodel types, and since APL2 is portable across many architectures and operating systems, we can easily integrate non-APL2 code.

Langlet, Gerard A.: The travelling salesman problem, revisited with APL. This paper introduces a new algorithm for the TSP and the class of n-p complete combinatorial problems, which were first found empirically with the help of APL used as a 'tool of thought'. Then follows a demonstration which shows that simplicity - as well as vector programming - may lead to significant progress in some fields of optimization. Programs are short and efficient. Reprogramming in Pascal or Fortran leads to code which to some extent runs faster than the interpreted APL models. The use of parallel processors would significantly improve the solution of the TSP. APL should also play a role in this new field of computer science.

Laumaa, Timo: Desktop Publishing on mainframe with APL and Ventura Publisher. This paper describes a facility which can provide APL applications with very high quality output, by integrating the mainframe APL environment and a PC desktop publishing system. The concept also shows an example of a VM/CMS server machine running on a PC. The system described in this paper is an unusual example of a distributed Host/PC application - here the mainframe transmits data to be efficiently processed in a PC.

Olsen, Thomas: Multi-Axis NC Postprocessor for Machining Centers. APL2 is applied to postprocessing CL (cutter location) datafiles output from the CAD/CAM systems APT, CATIA and CADAM NC. For multi-axis (XYZ with one or two rotary axes) applications, an APL2 PP (postprocessor) was found more convenient and expedient than commercially-available PPs. Availability of 'source code' plus immediate response to user need outweigh the higher cost of processing with an APL interpreter. The paper briefly reviews the concept

of NC postprocessing and evolution of the subject PP from earlier version marketed by IBM Germany. Implementation notes include use of ADRS to maintain NC machine characteristics and APL2 functions, a FORTRAN interface to CL-data, and WSSIZE minimization.

Pritchard, Tom: IBM System/370 Channel Programming Using APL. This paper describes how disk, tape and other devices attached to an IBM System/370 class machine running the VM operating system can be programmed directly from APL. The techniques rely on a few surprisingly simple APL functions and a simple auxiliary processor.

Teileri, Timo and Olkkola, Toivo: LYYTI - An Integrated Design and Control System. LYYTI is an APL2-based integrated system for design and manufacturing of customer-tailored switchgears, containing some expert system features. System development started in 1981. The LYYTI system uses the relational database DB2 and interfaces with FORTRAN, GDDM and CADAM (via FORTRAN). LYYTI runs on an IBM 3090-180S under TSO and all of the approx. 120 daily users access the same packaged workspace. Porting the system to a PC- PS/2 network is in process.

STREAM 7: APPLICATIONS: FINANCIAL AND INSURANCE

Traditional strongholds for APL applications have been the worlds of finance and demography. The presentations in this group demonstrate that this is still so.

Foots, William G.: APL2 Analysis and design of mortgage-backed securities. This paper demonstrates the ability of APL2 data structures and algorithmic capabilities to transform a description of a contingent claims pricing model into an implemented representation. The usefulness of the model lies in its ability to generate scenario-based analyses of the impact of varying market conditions on the value of the mortgage-backed security.

Iversen, Torben: The use of APL in Management Information and Economics in the Copenhagen Telephone Company, KTAS. KTAS is operating in complex and dynamic surroundings which has emphasized the need for instant, reliable management information. This paper describes the activities which the company have integrated into a management information system and how this has been achieved through the use of APL.

Kaski, Eeva-Liisa: End-user KESSU. The present-day KESSU IV is the tenth generation of a Finnish medium-term macro-economic computerized model. Since the beginning in the early sixties, the model has constantly been extended and optimized. One of the major advances has been the porting of the system from a mainframe to PC environments. Despite the increasing complexity of the model it performs very satisfactorily on PCs and enables economists to evaluate a multitude of 'what-if' scenarios. The system structure is modular and robust, including error recovery and help facilities. The user interface is largely menu-driven, permitting the model to be used by staff without EDP expertise.

Meyer, Ole M.: An insurance simulation model. This paper describes the development of an economic simulation model for Topdanmark, an insurance/financial corporation. The model is interactive, menu-driven and permits the manipulation of all relevant economic determinants (interest rates, inflation, investment portfolio, premium income, outstanding claims etc.). Output is in the form of scenario-based P&L accounts and balance sheets. The model was developed by economist end-users.

Miroshnikov, Alexey I.: Algorithm Alterable Models and APL. This paper deals with the software development of a set of economics systems models: algorithm alterable models (AAM). The special feature of AAM is that research work with the model leads to new knowledge about the modelled system, which necessitates changes to the model itself. After the change, the cycle is repeated. As an example, the paper discusses modelling of a long-term planning process in construction. Corresponding software - DSS - is described for planning. DSS has been implemented in APL 2.0 on a CDC Cyber-172 mainframe in LIAS USSR. Pseudo-parallel executions, combining APL with ALGOL and FORTRAN subsystems, have been developed to improve the dynamic characteristics of the DSS software.

STREAM 8: APPLICATIONS: SCIENTIFIC AND ENGINEERING

APL is being used in the scientific and engineering community as a tool to deal with everyday computing needs, as well as to explore theoretical issues, and the presentations in this group illustrate how.

Alfonso, Manuel: Neural networks in APL. Neural networks are fairly straightforward to program in a matrix oriented language such as APL. The only general improvement that would benefit them would be the implementation of sparse matrices. Small networks can be trained easily using the standard procedures (back propagation, etc).

Apiola, Heikki and Peltola, Pirkka: Integrating APL with symbol manipulation, numerical software and graphics. This presentation deals with the idea of combining symbol manipulation, numerical computation and graphics into a high-level environment for solving problems in scientific computing. The basic building blocks are Fortran subroutine libraries, a symbol manipulation system, and a vector-matrix language. Thus, we have chosen two different high-level host systems which are suitably connected to the Fortran-world (APL and Macsyma). We stress the idea of giving the user building blocks, as well as ideas for extending the system to meet the specific needs of each individual dealing with computational problems in very diverse computing environments.

Capra, Renato: Preliminary Mesh Checking for Structural Analysis. The purpose of this work is to provide tools for checking the nodes and elements of a Finite Element Modeler for Structural Analysis (Nastran). All the checks related to single elements (warping and stretch factor, interior angles) can be performed on the whole model without loops using scalar functions. Operations involving comparisons between elements (such as coincident nodes and coincident elements checking) require a non-trivial approach. A simple algorithm for recursive space-splitting is described with performance considerations for large models. Graphic output obtained using the Graphics Support Library of RTPC and an interface to Interleaf Workstation Publishing System are also provided.

Hawkes, Alan G.: Markov Processes in APL. Using examples from two areas of application, ion channel models and systems reliability, we illustrate the power of APL as an executable notation for the mathematics of finite-state Markov processes. We stress, in particular, the algebra of partitioned

matrices and second-generation features of DIALOG APL which allow a simple natural way of creating certain structured matrices.

Hendriks, Ferdinand and Ching, Wai-Mee: Sparse Matrix Technology Tools in APL. We have implemented sparse matrix technology tools in APL. Such tools have been conspicuously scarce, because APL has not been the language of choice for solving boundary value problems governed by partial differential equations in a way that adds flexibility and, on account of its compactness, maintainability. The main criticism of APL in numerically intensive applications has been execution speed. APL compilation addresses this drawback and shows factors of speed improvement of better than about three.

Lin, Edward Y.H. and Bricker, Dennis L.: Implementing the Recursive APL Code for Dynamic Programming. Despite the importance and wide applicability of Dynamic Programming techniques in optimization, the complexity and uniqueness of various dynamic programming models often makes the comprehension of their solution procedures difficult, particularly for a novice. Although it can be argued that, unlike the Simplex method in linear programming, no general dynamic programming algorithm exists, with the recursive capability of APL, it is feasible to develop a "systematic" solution approach for deterministic dynamic programming. In this paper, we introduce a set of such APL code developed to solve various types of dynamic programming problems with minor modifications on some of its key functions to accommodate the nature of each type of problems.

Pantziarka, Panagiotis: Object Oriented Database Using Frames in Second Generation APL. The appeal of Object Oriented systems and methodologies is that they provide conceptually simple and powerful tools to represent the real world. The essential elements of object orientation include: information hiding, classes, dynamic instantiation of objects, methods and the inheritance of properties by object. Frames are a way to combine both data and procedures to represent knowledge. This paper describes a frame representation system that is used to create a simple object oriented database. The frame structures are described and it is shown that they can be represented as nested arrays. The inheritance mechanisms are detailed and functions developed to handle the frames. Finally, a proposal is made for the further development of frame structured systems in APL.

Springall, Tom and Tillet, Gustav: A Shading Approach to Non-Convex Clipping. The conventional way of clipping a graphic object with a rectangular window or any convex polygon is to cut the object along a line tangent to each side of the polygon. Parts of the object on the exterior side of the line are thrown away. However, this process fails if the clipping window is non-convex, i.e., has one or more corners that point inward, or if the window consists of more than one polygon. One way of dealing with this situation is to break up the window into simple convex polygons. A different approach, presented here, is to use shading logic. The window is seen as an area to be shaded, and the object, as the shading pattern. The method yields not only the clipped object, but also, as a by-product, the reverse clipping, i.e., that part of the object that lies outside the window. This is an advantage in graphics editing where both clippings are often needed at the same time.

The APL90 Conference Exhibition

The APL90 Conference Exhibition will be tightly integrated with the Conference itself, as most of the stands are in the lobby area right outside the lecture rooms. The exhibition will show the latest advances from the major vendors of APL and related software and hardware products.

Vendor forums will also be offered, for detailed expositions, and particularly for new and enhanced product launches. These are half hour presentations by vendors in the lecture rooms. The overall schedule of Vendor Forums is shown in the conference programme on page 12; the detailed schedule will be announced at the conference.

Call For Software Contributions

Since APL86 in Manchester each APL conference has featured a Software Exchange, and APL90 will continue this tradition. We hereby invite you to share your code with the APL community. In an attempt to raise the visibility of the offered software, APL90 invites you to prepare a short presentation (10-15 minutes) of your masterpiece. In addition, we would like a short (1-2 page) introduction to the software.

We hope to have a complete collection of software from the previous exchanges as well as the new submissions received for APL90. All contributions are welcome. We will accept submissions in the following forms:

- 1) Workspaces/Files for any APL interpreter running under MS-DOS.
- 2) APL68000 or APL*PLUS workspaces for the Apple Macintosh.
- 3) DOS files in WSIS (Workspace Interchange Standard) form.

Please send your name, address, and a brief description of your proposed submission to:

Morten Kromberg,
APL90 Software Exchange, c/o Insight Systems ApS
119 Nordre Strandvej, 3150 Hellebaek, Denmark

To help us plan, please also indicate the hardware, operating system and interpreter on which your contribution will run. Also indicate whether you are willing to make a short presentation, and any requirements you have in this respect. This preliminary information should be received no later than May 15th, 1990. Contributions in machine-readable form should be received no later than June 30th, 1990.

Registration Procedure

Please do not attempt to register until you have received a printed registration form. Invitations containing registration forms can be obtained from the congress bureau or from members of the programme and organizing committees.

The registration form should be returned to the Conference Bureau, DIS Congress Service Copenhagen A/S, at your earliest convenience duly filled in and preferably typewritten, accompanied by the fee and other payments in Danish Kroner (DKK).

Deadline For Early Registration: June 1, 1990.

Registration Fees: Dkk

Participant, member of DD/ACM/SIGAPL, before June 1	3,650.-
Participant, member of DD/ACM/SIGAPL, after June 1	4,150.-
Participant, not member of DD/ACM/SIGAPL, before June 1 ..	4,300.-
Participant, not member of DD/ACM/SIGAPL, after June 1 ...	4,800.-
Student participant (please see below)	1,000.-
Accompanying person	1,150.-

Students: Only full-time students will be accepted as participants on student fee, and sufficient verification must be sent together with the registration form. The verification could be either a copy of your student card (1990) or a letter from the head of your faculty.

The conference fee includes: The conference fee for participants includes full conference documentation, coffee/tea during the conference Monday to Friday, the Get-Together on Monday, the Banquet on Thursday, Tivoli pass Wednesday and Thursday and Bus pass Monday to Friday. Registration before June 1 also includes mailing of the Conference Proceedings to the participant before the conference.

The conference fee for students includes only the conference programme, Proceedings, the Get-Together on Monday and Bus pass Monday to Friday.

The conference fee for accompanying persons includes the Louisiana Museum Tour on Tuesday (incl. lunch), the Get-Together on Monday, the Banquet on Thursday, Tivoli pass Wednesday and Thursday and Bus pass Monday to Friday.

Accommodation

The Conference Bureau offers participants of the APL90 Conference hotel rooms at a special discount conference price. Rooms will be booked according to your indication on the Registration Form - in as far as possible. Participants are advised to reserve rooms as early as possible, August being the peak season.

Hotels are available in the following price categories:

Category	Single	Double
A	1,320	1,650
B	725-895	850-1,130
C	550-785	
D	320-350	400-450

Extra bedding for children is available at most Hotels on request.

Hotel reservations will be made only when the Conference bureau has received your deposit (indicated on the Registration Form). The deposit will be refunded in full as a deduction from the bill, which must be settled on checking out of the hotel. The deposit also serves as a guarantee for the first night in case of non-arrival.

As a confirmation of your hotel booking you will receive a hotel voucher with full hotel name and address. The voucher must be presented to the hotel upon arrival. When you receive the voucher, please check that it shows the right arrival and departure dates.

Conference Bureau:

DIS Conference Service Copenhagen A/S
Linde Alle 48
DK-2720 Vanløse
Denmark

Tel: 45 31 71 22 44
Fax: 45 31 71 60 88

Fully Revised APL Product Guide

Compiled by Alison Chatterton

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

Please note that the guide has been fully revised for Vector 6.4. All vendors have been personally contacted, and any who failed to reply have had their entries removed.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- * Complete APL Systems (Hardware & Software)
- * APL Interpreters
- * APL Visual Display Units
- * APL character set printers
- * APL-based packages
- * APL Consultancy
- * Other services
- * Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions to the APL Product Guide should be sent to Alison Chatterton, at the address on the inside back cover.

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	IBM PCs & compatibles	poa	includes PC, mono/colour monitor, APL interpreter, operating system software, plus optional printers, graphics boards, additional memory etc.
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
MicroAPL	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
	Spectrum	7,000+	Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
M.T.I.C.	Atari 1040ST	799	1 Mb Mono/Colour System, includes 1 Mb disc drive & mains transformer built into Console.
	386sx AT	2600+	Entry level 32-bit compact AT with DIALOG APL. 2MB ram, VGA, 40MB hard disk, 1.44MB floppy disk, 102-key keyboard, monitor, mouse, MS-DOS 4.01, WORKS and WINDOWS-386. Many expansion options available.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES(£)	DETAILS
I.P. Sharp	SHARP APL	poa	International Network application systems and public databases.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	APL*Plus/PC Release 9	535	STSC's APL for IBM PC, PC/AT and PS2. Upgrades from earlier Releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1600	Incorporates mainframe features & performance in a version of APL for the PC
	Run-time	poa	
	Dyalog APL	1000-10,000	2nd generation APL for Unix systems

	APL2/PC	375	IBM's APL 2 for the PC.
Cooking/Drury	APL*PLUS/PC Rel 9	535	STSC's full featured APL for IBM 's and compatibles - Version 9 offers the potential for larger workspaces using extended/expanded memory; full screen editing of numeric arrays; user command processor; point & edit etc. Upgrades to version 9 are available from Version 8 and earlier releases.
	APL*PLUS PC Run-Time	poa	Closed version of the interpreter for developers, prevents user exposure to APL
	APL*PLUS II System	1600	High powered APL interpreter for the 80386 chip. Price includes one years maintenance and free upgrades - volume discounts VERSION 3 DUE JUNE 1990.
	APL*PLUS II Developer System	poa	All the features of the APL*PLUS II System plus unlimited free run-times to enable developers to distribute their applications.
	APL*PLUS UNIX	poa	STSC's 2nd generation APL for all major Unix computers and workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers running under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL
Dyadic	Dyalog APL	for DOS/386	995 Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and dSM Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL	for Unix Systems	995-12,000 Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
I-APL Ltd	I-APL/PC or RML Nimbus	4.50	ISO conforming interpreter. Supplied only with manual. (see 'Other Products' for accompanying books)
	I-APL/BBC	4.50	As above
	I-APL/Archimedes	4.50	As above
IBM UK	IBM PC APL2	348	APL2 for the IBM PC. Program 5799-PGG/1, PRPQ number RJ-0411. From all IBM dealers.
MicroAPL	APL.68000	2,000	Full implementation with component files, error trapping etc. for SPECTRUM, HP300, SUN, NCR, NIXDORF, BULL etc.
	QL/APL (keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL (APL chars)	87	VSAPL compatible APL for QL with many extra features.
	APL.68000 Apple Macintosh	87	
	APL.68000 Apple Mac II	260	
	APL.68000 Amiga	87	Full APL interpreters with support for windows, mouse, graphics etc.
	APL.68000 for Atari ST	87	
	APL*PLUS/PC - REL 9	450	
	APL*PLUS II	1,395	
I.P. Sharp	APL*PLUS/PC	480	STSC APL
	Upgrade V7 to V8	100	
	Upgrade any to V8	200	
	Sharp APL/PCX V1	1,650	For IBM XT/AT 370
	Sharp APL/PC V2	40	shareware for IBM PC/XT
	Sharp APL	poa	for IBM mainframes

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	IBM & compatibles	poa	IBM and 'budget' APL VDUs - monochrome/colour/graphics.
Dyadic	IBM 3151	599	Monochrome APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
	IBM 6154	1,228	Colour APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
MicroAPL	Insight VDT-1	795	Inexpensive APL VDU
	Insight GDT-1	1,450	With monochrome graphics
	Concept201	1,295	APL VDU with 8 page memory
	Concept 201G	1,650	Graphics VDU
Shandell	HDS3200/10 APL	965	15" screen, 8 page memory, windows, 80/132 columns, full overstrike. Multi-host multi-session support. ANSI X3.64, DEC VT100, VT220, Tektronix 4010/4014 1024 x 390 resolution.
	HDS3200/25 APL	1065	As above plus switchable 25 or 50 line screen, 75 Hz refresh. Resolution 1024 x 780 in graphics mode.
	HDS3200/35 APL	1265	As HDS3200/25 plus local pan & zoom.
	HDS3200/5C APL	1395	14" colour monitor, 75 Hz refresh, 8 or 18 colours from palette of 256. Display memory 96 lines of 80 or 132 columns. APL processing & keyboard with full overstrike. Windows, multi-host, multi-session support. ANSI X3.64, DEC VT220, VT100, VT52 emulation.

APL PRINTERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Epson series	200	Inexpensive dot-matrix and NLQ printers
	Quen-data & Qume etc	500	Daisy-wheel printers
Dyadic	Various	poa	Range of APL printers available.
MicroAPL	Datasouth DS180+	1,195	See Datatrade entry
	Philips GP480	2,137	Matrix printer with letter & draft quality and APL (480 cps).
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL-385	APL-385	50(PC), 125(mf)	Including ...
	FSM-385		Screen development
	DRAW-385		Screen design
	DB-385		Relational W.S.
	GEN-385		Miscellaneous Utilities
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	PANEL	poa	Fullscreen management system
	PFS	poa	Program File System - APL Systems development aid
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.

	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
APL IMPETUS	Impetus	poa	Hierarchical Planning System
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad- functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	The First APL compiler!
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Converts non-APL files to APL
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Information Centre spreadsheet incorporating data exchange between APL, FOCUS, IPPS, SAS, APL/DI, ADRSII, Lotus123, Visicalc, Multiplan & DIF
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(for PC's)	APL*PLUS PC Tools	325	Utilities including: RAM disk, full screen data entry, menu input, report generation, exception handling and games.
	IRMA Module	120	327x IRMA support.
	Fin & Stat. Library	350	Financial & Statistical routines
	SPREADSHEET MGR	195	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII & Lotus
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	PC Utilities including: APLMAC (windows); Unlock (unlocks functions in .AWS); DTEX (text and spreadsheet imp/exp). Mostly available in English or French.
Interprocess	IEDIT	1900-3200	Full screen APL2 editor with immediate APL execution, and a full-screen debugger
(mainframe)	AFM	8200-9800	High performance component and keyed file system (VS APL and APL2)
	Format	1650	A QuadFMT data formatter for VS APL and APL2
	FSM124	1650	AP124 programming for APL applications without GDDM (APL2)
	PowerCode	1300	External functions for APL2
	CALL/AP	3000	for calling non-APL programs (VS APL and APL2)
	UCF	1800	Inter-user data transfer for VM users via IUVC
(PC)	AFM	115	Single user component and keyed files for APL2/PC
Mercia	STATGRAPHICS 4	684	Integrated statistics/graphics system for the PC. Now with macros. Bulk and educational discounts available.
	Upgrade 3 to 4	215	
	Upgrade pre 3 to 4	375	
	MICROSPAN	250	Comprehensive APL tutor

	LOGOL	poa	Logistics management system for PC and 386. Sales Forecasting, Inventory Control, Master Scheduling, Distribution Requirements, Planning etc
	TWIGS		A modular library of tools to teach and explore state-of-the-art materials management concepts.
	T	299	Time series forecasting
	W	99	Warehouse replenishment
	I	199	Inventory Management
	G	99	Grouping requirements into EOQ's
	S	99	Scheduling production/purchasing
		599	All 5 modules above
		4999	All 5 modules site licence
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	APL self instruction for APL*PLUS/PC
	PC TOOLS Vol 1	295	
	STATGRAPHICS Rel 1	495	
	STATGRAPHICS Rel 2	535	
I.P. Sharp	GLOBAL LIMITS	poa	Exposure management for banks
	IPSA/CONNECT	poa	Mainframe to micro link
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V.2	65	Full screen front end to (PSA mailbox
	upgrade to V.2	15	
	NEWSFLASH	poa	Real time message exchange
	VIEWPOINT	poa	4GL - Info centre product
Warwick University	BATS	500	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Consultancy	poa	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL6800, C/Unix, TSO/MVS, VM/CMS, graphics, Operational Research etc. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.
Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry - designing, programming and implementing applications.
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	150-300	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	175-275 275-350 300-450 400-600 450-750	Junior consultant Consultant Senior consultant Principal Consultant Managing consultant
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
Delphi	Consultancy	poa	APL system development on mainframes and micros.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
E & S	Consultancy	poa	System prototyping: all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Ian A. Clark	Consultancy	poa	Computer-based Information Systems implementation where acceptance is critical. APL on PC and Macintosh. Human Factors of HCI; novice ease of use; online assistance; training courses; distance-learning materials.
Intelligent Programs Ltd	Consultancy	175-350	Systems development, enhancements, support.
	Documentation	150-250	Preparation of new manuals, rewriting of existing materials.
	Training	150-250	Training for APL experts through to non-technical system users.
Mercia	Consultancy	poa	APL*PLUS & VSAPL consultancy.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.C.	Consultancy	240-500	Business analysis and APL consultancy
QB On-Line	Consultancy	250	Specialising in Banking, Financial & Planning Systems.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
I.P. Sharp	Consultancy	poa	Consultancy & support service world-wide.
Wickliffe Computer Ltd	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search services available.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	An APL Tutorial	2.50	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2nd Edn)	5.50	228pp by Helzer
	APL in Social Studies	2.50	36pp by Traberman
	I-APL Instruction Manual (2nd Edn)	2.50	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	14.50	165pp by Thomson
** Please add one pound packing charge per order **			
I.P. Sharp	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APL Impetus Ltd	Cedric Heddie	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
APL People	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
APL Software	David Alis Jill Moss	The Old Malthouse, Clarence ST, BATH, BA1 5NS. Tel: 0225-462602
Buckland Management Systems	John Buckland	Westwood, 19 Grange Road, Camberley, Surrey, GU15 2DH Tel: 0276 684327
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. Tel: St. Albans (0727) 60130
Paul Chapman		18, Trevelyan Road, London, SW17 9LN Tel: 01-767 4254
Cocking & Drury Ltd.	Ronilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 01436 9481 Fax: 01-436 0524
Datatrade Ltd.	Tony Checkseid	38 Billing Road, Northampton, NN1 5DQ. Tel: 0604-22289
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 03677-384
Dyadic Systems Ltd.	Peter Donnelly	Park House, The High Street, Alton, Hampshire, GU34 1EN. Tel: 0420-87024
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-24741
H.M.W. Computing Ltd.	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, LONDON EC4Y 0HA Tel: 01-353 4212 Telex: 926604 HAMHSEG Fax: 01-353 3325
HRH Systems	Dick Holt	Box 4496, Silver Spring, Maryland 20904
Ian A. Clark		9 Hill End, Frosterley, Bp. Auckland, Co. Durham DL13 2SX Tel: 038852-7190

I-APL Ltd	Anthony Camacho	2 Blenheim Road, St. Albans, AL1 4NR. Phone 0727-60130 for queries, order forms, bulk orders
	J C Business Services	56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU N.B. PAID ORDERS ONLY
IBM UK Ltd	Nat.Eng. Centre	414 Chiswick High Rd, London W4 5TF Tel: 01-747 0747
Interprocess Systems	Stella Chamberlain	9040 Roswell Road, Suite 690, Atlanta, Georgia 30350-1131 Tel: (404) 992-8400
Marcia Software Ltd.	Gareth Brentnall	Aston Science Park, Love Lane, Birmingham B7 4BJ. Tel: 021-359 5096
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 01-922 8866
M.T.I.C.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH Tel: Sunbury(09327) 60848
Peter Cyriax Systems	Peter Cyriax	213 Goldhurst Terrace, London NW6 3ER Tel: 01-624 7013 (Answerphone) 0960-377953 (Mobile)
Q8 On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB. Tel: 0278-20789
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A.. Tel: (212)864-3078
The Rochester Group	Robert Pullman	50 S.Union St., Rochester NY 14607, U.S.A. Tel:716-454-4360 or 716- 454-4641
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks., HP8 4QH. Tel: 02407-2027. Fax:02407-3118
I.P.S.A. Ltd.	Mike Butler	7th Floor B Block, Coventry Point, Market Way, Coventry CV1 1EA Tel: 0203 256562
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 Tel: (808) 261-7536
Wickdiffe Computer Ltd	Nick Tetler	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel:0946-692588
Warwick Univ.	Dr Andy Pole	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel:0203-523066 x2158

Reviews Section

introduction by Adrian Smith

This Vector includes a brief review of APL*PLUS/PC version-9. Obviously, this concentrates on the new features available in this release; it is assumed that most readers are sufficiently familiar with the basic product that there is no need to mention existing features and facilities.

Readers might like to note the April 1st release date of IBM's latest APL2.5 for the RS/6000 workstation and Microwriter Agenda. At the initial price of £99.99 (excl VAT), this is clearly a breakthrough in APL price/performance, including as it does a 64-bit packager, and a set of AP's to provide full OS/2 Presentation-manager emulation.

We have also received an advance copy of Dyalog APL/386 with the new session manager facilities discussed by John Scholes on page 107. I can confirm the immediate impact made by the new style of session, but I need more time to get to know it in depth. The review will appear in July's Vector.

Finally, we hope also to review Release 6 of the Powertools/PC product, with the Common User Access tools demonstrated at the March meeting of the Association. This gives APL applications the ability to reproduce the effect of MS Windows, complete with mouse support, pop-up graphics and scrolling windows.

APL*PLUS/PC Release 9

reviewed by Adrian Smith

Introduction

I assume that APL*PLUS/PC needs no introduction. It has been the de facto standard PC APL system for the last 5 years, and is only now beginning to be challenged by IBM and Dyadic, with their new 32-bit APLs.

This review is aimed at existing APL*PLUS/PC users, who may be wondering whether they should upgrade to version-9. Consequently it concentrates on the new features in this release, and takes as read the general excellence of this product.

The Infamous Smith Benchmarks

I am going to have to give this up! Every time I benchmark one of STSC's interpreters, it is a little bit bigger, and goes a little bit slower.

```

▽ MAKEOBJ
[1] V←1000ρ 4 5 6 0
[2] M← 100 10 ρ 2.5 6.7 0 0 8 -5.7
[3] S←10000ρ'THE CAT SAT ON THE MAT '
[4] S[999+16]←'ADRIAN'
[5] I←100?1000
[6] B←10000ρV=6
[7] FF← 800 5 ρ'TR HGHJ TYE HJBG UGFHF HJ'
[8] FF[601;]←'EJD..'
[9] 'ALL DONE'
▽

```

I make no claims for the truthfulness of these tests, but at least I can claim a continuity back to Vector 1.4! I feel that the only one worth looking at is *FN 1000*, which is a simple looper, and does almost no other processing. It seems to me to correlate rather well with the perceived 'feel' of the various APL systems I have tested.

APL Expression	Type	Vn.5	Vn.8	Vn.9	Ext.9
$\lfloor \square WA + 1024$	αWA	367K	342K	335K	204K
$T + \square TS \diamond +/V + V \diamond DO \square TS - T \alpha INT \dots$		350	380	390	390
$T + \square TS \diamond +/V[I] \diamond DO \square TS - T \alpha INT \dots$		50	60	60	80
$T + \square TS \diamond V + . \times V \diamond DO \square TS - T \alpha RL \dots$		380	440	460	490
$T + \square TS \diamond +/V * 2 \diamond DO \square TS - T \alpha RL \dots$		3680	3840	3860	3900
$T + \square TS \diamond X + + / M \diamond DO \square TS - T \alpha RL \dots$		660	710	710	720
$T + \square TS \diamond V + . \times , M \diamond DO \square TS - T \alpha RL \dots$		2140	2260	2260	2310
$T + \square TS \diamond +/S = 'T' \diamond DO \square TS - T \alpha CH \dots$		3630	3840	3840	3790
$T + \square TS \diamond 'FAT' \epsilon S \diamond DO \square TS - T \alpha CH \dots$		60	60	60	110
$T + \square TS \diamond X + S \epsilon 'FAT' \diamond DO \square TS - T \alpha CH \dots$		2520	4730	4370	4670
$T + \square TS \diamond X + B \vee B \wedge \sim B \diamond DO \square TS - T \alpha BL \dots$		4840	5540	5540	5550
$T + \square TS \diamond \wedge / B \diamond DO \square TS - T \alpha BL \dots$		110	160	160	210
$T + \square TS \diamond X + \vee \backslash B \diamond DO \square TS - T \alpha BL \dots$		220	280	280	300
$T + \square TS \diamond FN 10 \diamond DO \square TS - T \alpha FN \dots$		330	440	490	510
$T + \square TS \diamond FN 100 \diamond DO \square TS - T \alpha FN \dots$		3100	4060	4230	4280
$T + \square TS \diamond FN 1000 \diamond DO \square TS - T \alpha FN \dots$		30.3s	41.4s	43.5s	43.2s
$T + \square TS \diamond X + FF \wedge . = 'EJD...' \diamond DO \square TS - T \alpha 60$		60	90	90	90
$T + \square TS \diamond X + FF \wedge . = \text{q20 5p} 'EJD...' \diamond DO \square TS - T \alpha 33.6s$		33.6s	37.5s	37.5s	37.5s

```

V FN N;CT;X
[1] CT+0
[2] LP:+(N<CT+CT+1)+EXIT
[3] X+ + / 2 3 p 16
[4] +LP
[5] EXIT: 'DONE'
V

```

The timings on the extended system are rather less reliable, as there seemed to be a much greater variation than with the standard system. This may have been a result of the reduced workspace, or it may be something in the call/return mechanism between the session manager and the interpreter. Either way, I kept APLSM.EXE on my hard disk just long enough to do these checks.

In summary, by upgrading to Version-9 you will lose 7K of free workspace, and you can expect a degradation in speed of between 2% and 5%. In my view, the 'extended system' was a bad mistake by the STSC marketing department, and you would do best to ignore it.

What do you get from Version-9?

UK readers should refer to Cocking & Drury's news on page 15 for the support implications. Those of us who are still mostly on release-5 regard this as a bit of a red herring, indeed I recently had a letter from a contented user of release 3!! However if you work for the kind of company that hates the idea of 'unsupported products' you obviously need at least 1 copy of release-9 to reproduce your problems in!

Other than continued support, some of the things you get for your money are listed in STSC's 'Summary of Changes', which comes with the product. These are the usual mix of minor fixes (relevant only to `□WIN` hackers), and genuine enhancements. Here they are:

- properly typeset manuals. These are a huge improvement; they are much better organised, and much easier to read. I actually found a couple of `□WIN` features I had previously totally overlooked, simply because of the terrible presentation in the old manuals. Important bits like the installation instructions are easy to find, and very clearly set out.
- even bigger numbers in the `e=` startup parameter. In theory, you can shift rather more of the interpreter into upper memory than you can with `e=8192` in version-8. In practice, if you can find 128K of contiguous upper memory (the bit between 640K and 1Mbyte), you are a better man than I am! I suspect this will be of use only to owners of 286 kit with some form of XMS driver; the rest of us will have to be content with the 64K bonus available via `e=8192`.
- numeric editing with `)EDIT` and `□EDIT` in both the standard and extended system. This is a *dead handy* facility, particularly as it is quite a hard thing to write efficiently in APL. Well worth a couple of K of `□WA`.
- simulated `] commands` (as per the mainframe APL*PLUS/1000 system). These are discussed in more detail below.
- odds and ends, e.g. an interrupt to purge the symbol table, upgraded VDI graphics drivers, better terminal emulation, a new *HANDLERS* workspace, and printer support for the Epson LQ printers and HP Deskjet.

Simulated 'User Command Processor'

APL*PLUS/1000 (from release 6 upwards) has had the ability to call 'user-defined' commands such as `]VARS`, using the '`] key`' as an analogue of '`)`' in conventional APLs. The system comes with a number of pre-defined commands, but a major benefit of this is that you can easily define your own, and they then become globally available to all your APL users without the need for everyone to take copies in their own workspaces.

Release-9 simulates this mechanism with a $\Delta UCMD$ function, which pulls in appropriately packaged code from a standard .ASF file, and then runs it with everything automatically localised. An example would be $\Delta UCMD \text{ 'FNS M*'}$ which will list all functions beginning with 'M'. There is a good selection of functions already included in the file, but you will almost certainly want to define new ones of your own. To this end, STSC have provided a set of utility functions, as well as very thorough documentation on the file structure, and the way the user commands operate. In particular, all the cover functions (from $\Delta UCMD$ downwards) are unlocked and well commented; this is a highly commendable move towards a more open architecture.

One of these utility functions does some amazing jiggery-pokery with $\square PFKEY$, to mimic the appearance of a ']' on the screen, while actually executing $\Delta CMD \text{ '[]'}$. Much as I hate this sort of subterfuge, it works reasonably well, as long as you can train yourself to hit F5 whenever you want ']'. A slight snag is that the command is left on the screen in the normal way, and you might get tempted to take the cursor back up to it for another go ... this will of course give you a *SYNTAX ERROR!*

- $] EDIT$... native file editing, but at least you can stop this from putting Ctrl-Z on the end! the APL*PLUS II $] EDIT$ did some really nasty things to my LOGIN.SQL; this is much better!
- $] FNS$... basically this does what $] FNS$ should have done all along; a proper wild-card match, and the ability to list functions from another workspace.
- $] LIB$... a much tidier file listing.
- $] WSLOC$ and $] RENAME$... find and replace names throughout the workspace. Most of us already have something similar, but if you are new to APL you will find both of these invaluable.

These are the obviously useful ones, and they all work equally well in release 8. My only minor gripes are: (a) they are a bit slow on anything other than a 386 machine; (b) please can we have '[' recognition built in to the interpreter.

Summary

Users of slow old 8088 PCs (like me) should ignore all the above; nothing is worth a 40% loss of speed, or 32K of $\square WA$, so stay with release 5. If you have access to reasonably up to date hardware (say a 20MHz 386), the upgrade to version-9 is well worth the trouble, but please leave APLSM.EXE in its wrappings, and stick with the standard system.

IBM's APL2.5 for the RS/6000 and Microwriter Agenda

reviewed by Adrian Smith

Technical Breakthrough

One of the major advantages of generating your APL systems using a high level Intermediate Language (IL) is that you can quickly port your interpreter across a wide variety of operating systems. This technology was first exploited in 1979 with an APL for the IBM System-7 minicomputer, and has subsequently been applied to good effect by Dave Selby and Manuel Alfonseca in shipping APL2/PC in both 16-bit and 32-bit versions.

IBM have exploited the same technique in their April announcement of APL2.5 simultaneously across two widely differing platforms: the top-end RS/6000 workstation (maximum workspace 3.7 Gbyte) and the Microwriter Agenda. It is also fascinating to see the use of Display PostScript exploited for an APL development environment which looks remarkably like Presentation Manager. Again, the device-independence is fully exploited in providing similar facilities on the ultra-high resolution workstation monitor, and the 4x20 character LCD display of the Microwriter.

New Features of APL2.5

As its name implies, a major enhancement of APL2.5 is Array Interpolation. Just as you can join arrays on intermediate axes with Laminate, you can now index arrays and vectors between existing elements:

```

MAT+(112),[1.5] 12p1 2 3      * Standard APL

VEC+1+112

VEC[2 2.5 3]                  * APL2.5
3 3.5 4
```

APL2.5 uses natural cubic splines to interpolate in more complex cases. An obvious use of this facility is to plot smooth curves through a series of points (similar to the GDDM option 'CURVE').

Rather than continue the controversy over the 'best' APL keyboard layout (see the notes on the Falkoff Dual keyboard on page 92), IBM have chosen to bypass the problem completely, and are shipping the MicroWriter keyboard as an essential component of the system. They also provide an optional AP to allow voice recognition of APL expressions.

This is probably the first 'Environmentally Friendly' APL to hit the market. It justifies its distinctive green packaging with an enhanced $\square EA$; when used with a numeric left argument, this gives the user the option of 'Environmentally Aware' execution. For example at 2 $\square EA$ 'expression' any attempt to produce printout will give a *DOMAIN ERROR*, and at $\square EA$ levels above 5, the processor switches to progressively more energy-efficient algorithms, until by 10 $\square EA$ it spends most of its time asleep.

Finally, supporters of I-APL will be delighted to see that not only is Direct Definition supported for functions, but that APL2.5 also has direct definition of Operators:

```

      PLUS:  $\alpha + \omega$ 
2 PLUS (2 4)[1.2]          a Note the need for ( )
4.4

      ELSE:  $\omega : \beta : \alpha$ 
1 -ELSE÷ 5
-5
0 -ELSE÷ 5
0.2

```

Note the extension of the conventional syntax to include β as the 'data' operand, in addition to α and ω as the function operands. Readers might like to compare the simplicity of this new style of operator definition with the same *ELSE* operator in John Scholes's workshop on page 66.

Price and Availability

APL2.5 is shipped on a single 3.5" optical disk, and will be available exclusively from APL-385 from 1st April (UK and Rockall only). The cost is £99.99 (excl VAT), inclusive of the 64-bit packager.

For more details, contact:

I. Phelfrith
 Brook House, Gilling East, York
 Tel: 04393-385

RECENT MEETINGS

This section of VECTOR is intended to document the seminars given at recent meetings of the association; it is of particular value to members who live away from London. It also covers other selected events which may be of general interest to the APL community.

If you would like to speak at one of the regular British APL Association seminars, please ring the Activities Officer (address on inside back cover) who will respond enthusiastically to your offer.

Notes on Recent Meetings

Introduction by Adrian Smith

Many thanks to Ed Shaw for his notes on Arthur Whitney's talk on 'A' at Morgan Stanley in New York. The notes even survived downloading through Anthony Camacho's IPSA mailbox fairly intact.

The January meeting was billed as 'CUA Tools' for APL, followed by a demonstration of the new Dyalog APL session manager, and closing with a Workshop on defined operators. Unfortunately, Per Hultin couldn't make the meeting, so instead of CUA tools, we had a demonstration of some of the existing Powertools technology, as seen in Stan Wilkinson's 4XTRA package.

The demonstration of Dyalog's session manager was most impressive, and readers might like to refer to John Scholes's paper 'A New Development Environment for Dyalog APL' on page 101. John has also provided an extremely thought-provoking session-log of his talk on defined operators.

In February, we enjoyed a talk by Tim Perry on an estimating package for engineering companies called PEFAC. Tim's notes will be in Vector 7.1. I filled in with a demonstration of a little Hypertext interface to Oracle. For some reason, the audience went strangely quiet at the idea of 500 tonnes of chocolate ... I can't think why!

More on 'A' by Arthur Whitney

notes by Ed Shaw, via Anthony Camacho's Mailbox

Way back in January, 1990, Arthur Whitney presented an update on his work on the A language to NY/SIGAPL. The meeting was hosted by Morgan Stanley in one of their corner offices high up in a Rockefeller Center office building in the heart of Manhattan, overlooking a shimmering Avenue of the Americas after an early evening rain. It was a metaphorical venue for an APL that makes you feel on top of the world.

For the most part, Arthur reiterated what he has said at APL89, which was fully reported in the October 1989 issue of Vector. By January A had gone through one more revision and was operating on a machine with even more addressable memory. He also demonstrated A before our very eyes, including a quick pass through a 700 megabyte file containing all prices every five minutes for the last five years of a select group of stocks.

I remember trying to describe what I had seen to my associates in the office the next day. The digits 3716 stuck in my mind but I could not focus on what order of magnitude it was. Finally I calculated 2 to the 32nd power on my trusty super-calculator and realized that it was a workspace of 3.7 gigabytes. The remaining digits didn't matter! For someone who had just recently grabbed 130K of memory above 640K on a PC, it took the breath away, made the heart skip a beat, and caused the hands to develop a slight tremor. Just for the fun of it, Arthur pushed the workspace to the maximum - 4,097,835,008 bytes while I was talking to him recently on the phone. So is Morgan Stanley happy? No. They are anxiously awaiting the arrival of an IBM RISC System/6000 which will have an address space of 2^{52} . I leave that computation to the reader.

It is wonderful talking with Arthur. He is excited about both APL and what he has been able to do with the interpreter. He is extremely pleased with the reception A has been getting at Morgan Stanley. Usage is spreading beyond the Analytical Proprietary Trading Group of which he is a part into other areas of the company. It may well be that A will save APL at Morgan. APL's performance has been increasingly challenged of late.

When asked what contributed to the increase in speed of A over other commercial APL interpreters, Arthur suggested the following factors:

- Many of the primitive functions execute 2 to 5 times faster because they use faster algorithms written in C.

- A allows the user to select a faster APL algorithm to solve the same problem because it has functions/operators not available in (some) other APL's, e.g. the rank operator.
- A uses **tight** code. It is more efficient simply because it was written by one person rather than by many over a period of years.
- A pre-parses to a greater extent than other implementations.
- Some meaningful efficiency derives from the absence of garbage collection (reorganizing memory so that free space is contiguous), perhaps as much as 20%.

How does A manage to avoid garbage collection? The answer lies in the large addressable memory space. When a request for memory is made, available memory is broken down into blocks which are some power of two in size. The new object is allocated a contiguous segment of these blocks. This process continues as more memory is requested. If real memory is no longer available, then unused objects are swapped out and the available pages are swapped in. There you do have object movement, but only what is necessary. In garbage collection as many objects are moved as are needed to clean up the fragmentation. As memory is freed in A, no movement takes place. Those pages are merely flagged as free.

Whether that space is usable depends on whether it is large enough to contain a new request. So there is no free lunch. In effect a trade off is made. Give A a large address space and under normal circumstances (who would call Morgan Stanley's circumstances normal?) there will always be space available. But fragmentation does occur and it is not directly removed. Theoretically, half the space might be fragmented enough so that only small objects can fit, so it is still possible to get a *WS FULL*.

Arthur and his colleagues have successfully processed objects of 400 megabytes in size, requiring intermediate storage of 800 megabytes. It took approximately a half hour to process on the Solbourne (A computer using SUN's SPARC chip. A side note: SUN was founded by people who were involved in the Stanford University Network project in the 1970's.)

One of the implications of A's innovative memory management technique is that it ought to be transferable to other APLs running under an operating system that provides access to the paging mechanism. One wonders if it might indeed be implementable in some form using only a virtual workspace file as in STSC's APL*PLUS/PC.

What is next? The current project for A is to add an interesting user interface using X-windows. And after that? ... Stay tuned. Since the source code is only 30K and the object code is only 100K maybe some way can be found to make A available to the rest of us mortals. One can hope.

Some Defined Operators in Dyalog APL

by John Scholes - Dyadic Systems

The following text is a 'hard copy' of one of the sessions which took place at the British APL Association workshop on 'User Defined Operators'. The functions and operators shown here are certainly not all original, and many are perhaps more interesting than useful. However some of them may provide food for thought.

First, an operator which is useful when writing function expressions such as $+$, $\times$ or $+$, $\times$, $/$ as it shows you what's going on during the evaluation.

```

      ▽ Z←A(FN V)B ⍝ verbosely
[1]  A (⊞OR'FN') B '→' (Z←A FN B)
      ▽

```

$\ominus$ OR is a function in Dyalog APL in the $\ominus$ CR, $\ominus$ VR, ... mould which gives a sketch of the named function.

```

      2 +V 3
2  +  3  →  5
5

```

```

      +V/14
3  +  4  →  7
2  +  7  →  9
1  +  9  → 10
10

```

```

      3 4 (+V).(×V) 10 20
4  ×  20 → 80
3  ×  10 → 30
30 +  80 → 110
110

```

if your APL doesn't have $\ominus$ OR, you can still get a good idea of the execution of the function by replacing the first line with:

```
[1] A '◊' B '→' (Z←A FN B)
```

in which case you would see:

```

      2 +V 3
2  * 3 + 5
5

```

you may also like to make the V operator ambivalent so you can trace the execution of monadic functions.

In the examples which follow, we will use the primitive 'compose' operator ' $\circ$ '. Compose can be used to glue an argument to a dyadic function to produce a new monadic function. For example:

```

*  o  0.5   is the monadic square root function;
1  o  o      is the trig function 'sin'.

```

```

      (* o 0.5) 9
3

```

ON is an operator which applies its (function) operand repeatedly to the argument.

```

      V Z+REP(FN ON)Z
[1]      +REP+0
[2]      Z+FN Z
[3]      REP+REP-1
[4]      +1
      V

```

... so the following function (verbosely) doubles its argument 4 times:

```

      4 xV o 2 ON 1
1 x 2 + 2
2 x 2 + 4
4 x 2 + 8
8 x 2 + 16
16

```

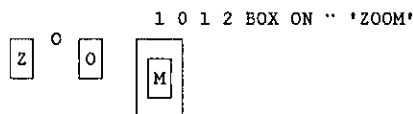
... or using a function which draws a little box around its argument:

```

      2 BOX ON 'ZOOM'

```





Note that *ON* with a boolean left argument gives us conditional function application, a sort of 'if' construct. For example, the following expression draws a box around the matrix only if it is non-null:

```
(0 0#MAT) BOX ON MAT
```

Likewise, the *ELSE* operator conditionally applies either its left or right function operand:

```

      ∇ Z+BOOL(TRUE ELSE FALSE)Z
[1]   +BOOL/T
[2]   Z+FALSE Z
[3]   +0
[4]   T:Z+TRUE Z
      ∇

```

... the following trivial example shows its use:

```

      1 - ELSE ÷ 4
-4
      0 - ELSE ÷ 4
0.25

```

Now we have *ELSE*, we can use it to define *DOWN* which applies its function with *D* each's or *D* levels down from the top of the nested array:

```

      ∇ Z+D(FN DOWN)Z
[1]   Z+(0=D) FN ELSE((D-1)•(FN DOWN")) Z
      ∇

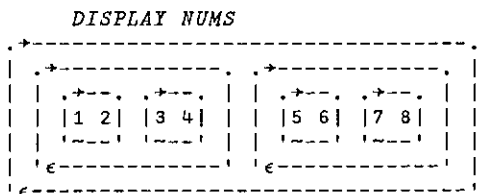
```

which says: If *D* is zero, apply *FN* directly (with zero each's) to *Z*, otherwise apply it at depth *D-1* to each subarray. Note that the string

```
FN ELSE ((D-1)•(FN DOWN"))
```

... forms a single derived function in exactly the same way that $+. \times$ forms the derived function for inner product.

So, if *NUMS* is a Vector-of-Vectors-of-Vectors of numbers.



then ..

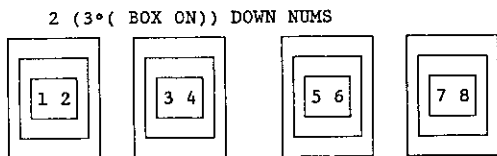
```
1  $\phi$ DOWN NUMS
3 4 1 2 7 8 5 6
```

```
2 +/DOWN NUMS
3 7 11 15
```

```
3 BOX DOWN NUMS
```

Now, combining *DOWN* with *ON* you can say:

Go down 2 levels and there draw 3 little boxes:



Interestingly, this gives the same result as if you said:

```
3 (2*(BOX DOWN)) ON NUMS
```

... or: 3 times in a row, go down 2 levels and there draw a box.

Just as the *ELSE* operator provides a counterpart to procedural languages' if .. then .. else construct, the following rather opaque *OF* operator gives us an equivalent to the *CASE* statement.

```

      ∇ Z+(D)(N OF FN)Z      a Z+N OF FN1 OF FN2 OF FN3 ... Z.
[1] +(3=□NC'N')+2 ∘ N Z+(□EX'N')N Z a LEFT DN TREE WHILE N IS FN.
[2] +(□IO=N')+3 ∘ Z+FN Z      a APPLY FN IF N'TH FROM LEFT OF TREE.
[3] +(□NC'D')+0 ∘ Z+(N-1)Z a PREFIX POSN FROM LEFT IF NOT TOP.
      ∇

```

so we can write ...

```

      2 OF + OF ÷ OF - 4
0.25

```

... apply the 2nd function (÷) to the argument. Similarly:

```

      3 OF + OF ÷ OF - 4
-4

```

an out of range case just returns the argument unchanged.

```

      88 OF + OF ÷ OF - 4
4

```

we can use *OF* to write *LEAVES* which applies its function operand to subarrays of depth *D*. It's a sort of opposite to the *DOWN* operator.

```

      ∇ Z+(D LEAVES FN)Z      a APPLY FN TO LEAVES OF DEPTH D.
[1] Z+(□IO+1+×D-|≠Z) OF (D LEAVES FN") OF FN Z
      ∇

```

This says return:

In the case (D is less than the depth of Z),
 ... the application of D LEAVES FN to each subarray of Z .
 In the case (D is equal to the depth of Z)
 ... the application of FN to Z
 In the default case (D is greater than the depth of Z)
 ... just Z .

```

      0 LEAVES (+/) NUMS
1 2 3 4      5 6 7 8

      1 LEAVES (+/) NUMS
3 7 11 15

      2 LEAVES (+/) NUMS
4 6      12 14

```

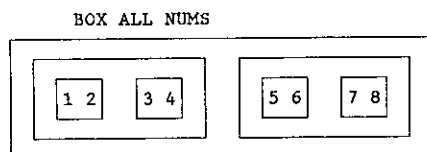
To complement *DOWN* and *LEAVES*, we have *ALL*:

```

      ∇ Z←(FN ALL)Z
[1]  Z←FN (1<|Z) FN ALL" ON Z
      ∇

```

which applies its operand to all levels of the array from a simple (depth 1) array upwards.



```

      +/ ALL NUMS
36

      ϕ ALL NUMS
8 7 6 5      4 3 2 1

```

Finally, we can write an operator to do the 'pervasion' which we normally associate with the scalar primitive functions.

```

      ∇ Z←A(FN PERV)B      a Pervasively.
[1]   Z←(1≡A B) (FN∘B) ELSE (FN PERV"∘B) A
      ∇

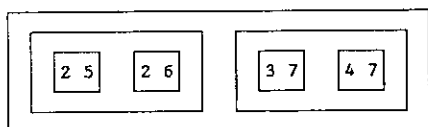
```

... if *A* and *B* are both depth 0, apply *FN* between them, otherwise, apply it pervasively between each subarray ...

```

      BOX ALL 2(3 4) ,V PERV (5 6)7
2 , 5 → 2 5
2 , 6 → 2 6
3 , 7 → 3 7
4 , 7 → 4 7

```



Editor's Note: this text came as a nested vector in a Dyalog APL workspace. I believe I have dug it out correctly, but I gave up on the little boxy bits, which have been cut out from John's original and stuck in!

ORIEL: A Window on the Oracle

by Adrian Smith

Background

ORIEL is an easy to use front-end designed to make data in an Oracle database accessible to shop-floor personnel. The need for such a system arose partly from the special conditions in the chocolate-making department; this is spread over 8 floors of an old building, and it is quite impossible for a team-leader physically to monitor all the machines in his area. The department was also quite unusual in having an extensive degree of computer monitoring and control at the machine level; much of the current plant status was collected on an old PDP-11/44 and used to run shift-end reports.

The departmental manager had for some time been aware that there was a gulf between the machine-level systems (which monitored and adjusted ingredient flow on a second-by-second basis), and the 4-weekly reporting cycle available to him through the Factory Accounting procedures. Typically, a machine feed could drift out of calibration, and the incorrect ingredient usage would not be detected until several weeks after the event!

He realised that most of the data he needed to control the plant at the shift and weekly level was available somewhere in the machine systems, however:

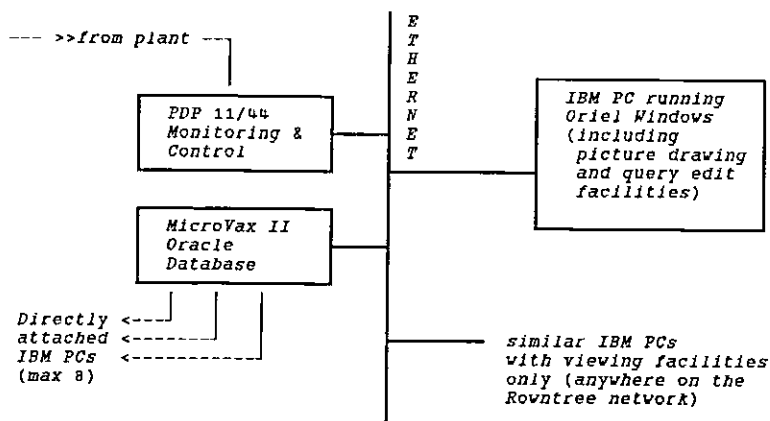
- much of it was only available at a single display, or was printed out and discarded at the shift end. Often the printers were out of paper or ribbon, and the information was lost.
- there was no easy way (other than laboriously by hand) of cross-matching the output from one process with the input to the next.
- many existing readings (e.g. from load-cells on chocolate tanks) were suspect, due to lack of recent calibration.

He also knew that for an information system to be used on the shop-floor, it had to be absolutely straightforward in operation, and that the information shown must be accurate, and as up to date as possible. Another clear requirement was the need for a simple messaging system, which could be used at shift-changeovers, or to communicate between the team-leaders and the departmental management.

General Description

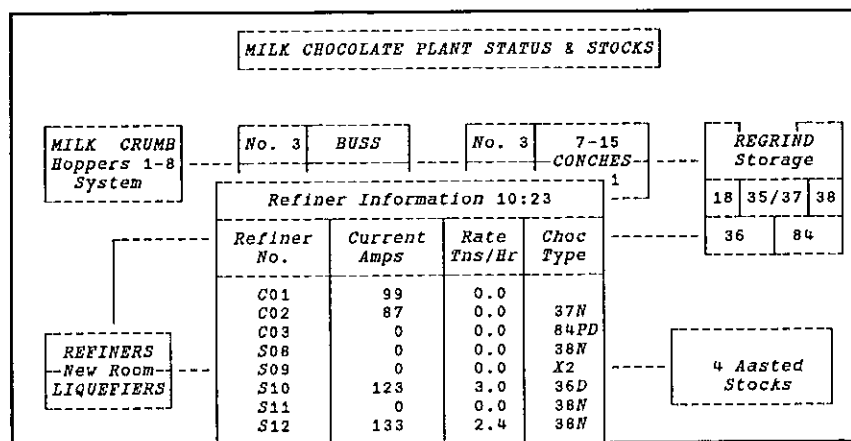
The final system design evolved through a series of prototypes over the course of about 9 months. During this time, the decision was taken to collect the plant information into an Oracle database, running on an independent computer (a DEC MicroVax II), and much of the preliminary database design was done. A program of calibration and checking was also instigated, as it was important that the new system gave accurate results from day one.

The final prototype made heavy use of 'PC' features such as colour, windows and the ability to 'time-out' and refresh screens automatically. For this reason the PC was chosen as the user-interface to the system, in preference to a standard terminal such as a VT-200. The combination of PC and Vax network led to an initial configuration as shown below:



A Simple Schematic of the Network

The user works with a set of plant diagrams, running preset SQL queries by moving the cursor to the appropriate area of the picture and pressing "Enter". If he requires more detail he can usually do "PgDn" to access a more detailed diagram of a specific area of the plant, and hence more detailed queries. There are various options for the expert user, e.g. he can jump directly to the picture he wants, or can set up a 'tour' on a single key, which will run a preset sequence of queries for him.



1ST pict>Plant Menu>Milk Choc ----- 10:27am -

A typical Plant Diagram with some Oracle data displayed

The diagrams and queries are developed using a variety of window-based drawing and editing tools. Typically an SQL query can be assembled and tested very much faster in this environment than in raw Oracle. Queries are saved on the PC until fully debugged, and are then transferred to the MicroVax for production use. In general each query has an associated 'pro-forma' into which the data is fed. This provides a very flexible way of laying out reports (and defining simple graphics).

Examples of use in Chocolate Manufacture

As mentioned in the introduction, one of the key requirements of the Oriol development was to provide ingredients control information to the Team Leaders so that they could take immediate corrective action. A typical display (please note that the recipe figures have been obscured for security reasons) is shown on the following page:

MILK CHOCOLATE PLANT STATUS & STOCKS														
NO.3 BUSS - Ingredients feed rates 10:27 CURRENT RUN ONLY														
Choc Type: 366														
Duration of this run: 22 mins														
Variance (Avge V SPI)														
Ingr Usage Rates (kg/mn) <-negative - positive->														
(KGS)	Current	Average	SPI	5	4	3	2	1	0	1	2	3	4	5
Crumb	82.00	83.38	xx.xx											
LC	5.13	4.91	xx.xx											
Water	2.32	1.77	xx.xx											
Pure	3.49	2.95	xx.xx											
ECB	6.88	7.03	xx.xx											
RERS														
New R	99.82	100.04	xxx.xx											
LIQUEF	N.B. Vanillin is added to all Recipes - No actuals available.													
Target should be to maintain average feed rates at SPI set rates														
(Holding)														

1ST pict>Plant Menu>Milk Choc ----- 10:28am

Typical Ingredients Control Information

3 AND 4 BUSS

36 REGRIND STORAGE	STG 046 0.74 time 10:27	STG 044 4.09 time 10:27	STG 045 2.94 time 10:27
*KEY 'ENTER' TO DISPLAY ALL TANKS	STG 82 4.22 time 10:28	STG 207 2.42 time 10:28	STG 208 3.27 time 10:28
RFC 3	RFS 8	RFS 9	RFS 10

1ST pict>Plant Menu>Milk Choc>Regr 36 ----- 10:30am

Display of Selected Chocolate Storages

In the top picture, the feed rates are shown, along with the average feed rate for the run, and the variance of the average from Recipe (SPI) is displayed in barchart format. I have deliberately chosen a time very early in a run, so the average rates are well away from SPI; typically they all converge within 2% after an hour or so.

Another important use of the system is to allow staff to monitor the stock levels in the intermediate and finished storages. It is vital for them to know exactly where chocolate is when they are faced with a sudden change in requirements (typically resulting from a breakdown) from a user department. The lower picture illustrates part of the Kit Kat storage area.

Note that all the data is shown with its time of collection from the plant; this is important, because if part of the network goes down the system will still show the most recent information available. Typically the times will show that the data is between 2 and 5 minutes old.

Some Technical Details

Oriel runs on an IBM PC (or compatible) with at least 420K of free memory and either an Ethernet connection or a serial port. A colour screen is desirable, with the EGA or VGA type preferred.

Asynchronous Connection to the Vax:

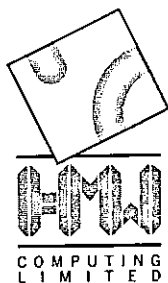
The windows front end is "plug compatible" with a standard DEC VDU (VT 100), i.e. it can be used over a serial connection direct to a Vax, via a terminal server, or even over a telephone line (although this would give dreadfully slow response).

Ethernet Connection:

If a PC already has a network connection (DEPCA or 3Com) Oriel can use this to link directly to an SQL task on the Vax with the PCSA Task-to-Task software. Please note that the free system memory must be sufficient (416K is the absolute minimum) after all the network drivers are loaded.

This is the preferred way of installing Oriel, and this type of connection is required for the analyst who is developing and maintaining a Oriel system.

The implementation of Oriel in the York chocolate-making operation involves roughly 40 diagrams, and 150 Oracle queries. An APL*PLUS/PC run-time license is currently required to use the system; the cost of these is around £100 each. Oriel will be re-implemented on IBM's APL2/PC during 1990. This provides unlimited free runtime copies with no conditions on their use.

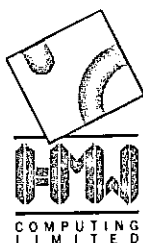


SOLUTIONS FOR THE CITY

We are a specialist firm of consultants with a prestigious list of clients including major City institutions.

For one of these clients we have recently developed a foreign exchange dealing system with the following features:- deal entry, deal enquiry, multi-currency position keeping, basket currency calculators, arbitrage calculators and ticket production.

If you are interested in foreign exchange dealing, treasury operations, portfolio management, commodity broking or similar areas we would be delighted to discuss how we could help. Call Stan Wilkinson, HMW COMPUTING LIMITED Hamilton House 1 Temple Avenue London EC4Y 0HA Telephone 01-353 4212



EXCITING POSITIONS AT THE LEADING EDGE

With top City institutions and leading Blue chip companies amongst our client list we have a lot to offer experienced APL programmers.

Call Stan Wilkinson to discuss your future.

HMW COMPUTING LIMITED
Hamilton House 1 Temple Avenue
London EC4Y 0HA Telephone 01-353 4212

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers

A Design Framework for APL Systems

by Dave Piper

Abstract

This article is based on a presentation made at a meeting of the British APL Association in 1989.

APL is often credited with being the worlds first 'write once, read never' computer language. This idea is disputed by professional users of APL who cite many well structured commercial systems as evidence to back up their position. Many experienced APL developers attribute APL's reputation to its early history. In the early 70s, APL was used by the business community as an end-user language. In the late 70s it was extensively used by IBM in this way; the ADI and ADRS products grew out of this environment.

Both packages addressed widespread business needs, but they were developed without attention to design or quality. They were excellent examples of bad APL, but for many APL users represented the only models available on which design and coding practices could be based. As a result, bad APL programming practices became widespread, and APL's reputation continued to suffer.

Whatever the truth behind the two sides of the argument, the value of systems design is now widely accepted, even in the APL community. The debate has moved from "should we design?" towards "how do we design?". A more cogent form of the question is "how do we design effectively?".

Design Economics

Many design methodologies have been created by various large consultancies across the world. These are structured to reflect the monolithic nature of systems implementation using 'standard' commercial languages and processing techniques. Systems implemented using APL feature a variety of unusual aspects which render these methodologies less productive:

- susceptibility to change requiring an unusual degree of flexibility and responsiveness in the design and implementation
- a wide range of system 'size' requiring a design methodology that can be tailored to fit many different development needs

- parallel processing of data changing the way much of the code is structured

In addition, whatever the view of APL in the world at large, professional APL users seem to be able to deliver function points at a greater rate than users of other languages and techniques. The effects of this higher delivery rate on the design process are important:

- APL projects consume less programmer hours
- less effort can be put into the design process before the marginal cost of design exceeds the marginal benefit in programming

For all these reasons, 'traditional' design methodologies may not be suitable for the design of APL systems.

The Implementation Process

Figure 1 shows a very high level view of the implementation process. The process facilitates the move from unstructured business requirements to the structured implementation of a system to meet these requirements.

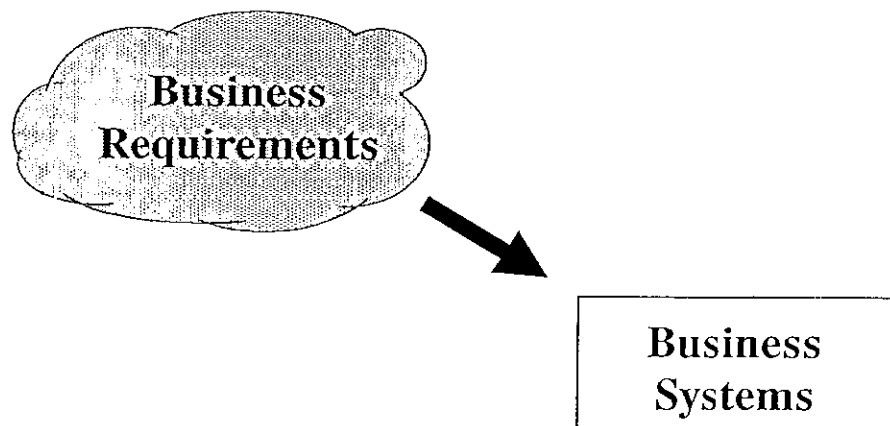


Figure 1: a very high level view of the implementation process.

Typically business requirements can be viewed as an amorphous cloud of ideas; they are:

- unstructured
- contradictory
- undocumented
- and even unknown

In contrast, a system should be characterised by the following:

- structured flows and processing of data
- coherent
- compatible (with other systems)
- documented (please!)
- known

We can take a closer look at the implementation process. A typical structure for the process is shown in figure 2. Design is only a single phase within this process - but a very important phase. The input to the design phase is a statement of systems requirements. This may take the form of a relatively formal, structured expression of the business requirements following some initial business analysis work. Alternatively (and more commonly for APL systems) it simply takes the form of some rough jottings on the back of an envelope. This more limited form is possible with APL because of its power as a means of expressing ideas - APL as a 'Tool of Thought'.

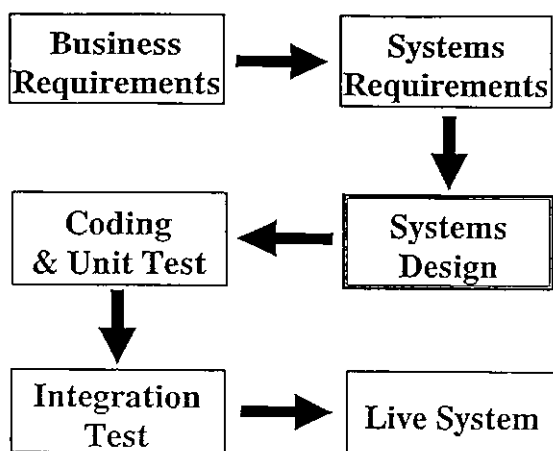


Figure 2: the Structure of the Implementation Process

The output of the design process is a structured design document. The design will vary in scope and complexity to match the size, complexity and implementation requirements of the system. This flexibility in the design process is a key requirement if it is to be a success in more than a limited number of situations.

A second key requirement for the design process is that the results should be communicable. The design documentation forms a bridge between the users with their business requirements and the computer professionals performing the implementation. For the users, the design should be able to answer questions such as "are we getting what we need?". For the systems professional, the design acts as a map to follow during the implementation work. It must, then, be expressed in a standard form to aid comprehension on both sides.

The Design Process

Figure 3 shows a slightly more detailed view of the design process itself. The designer examines the data and functionality required to fulfil the systems requirements.

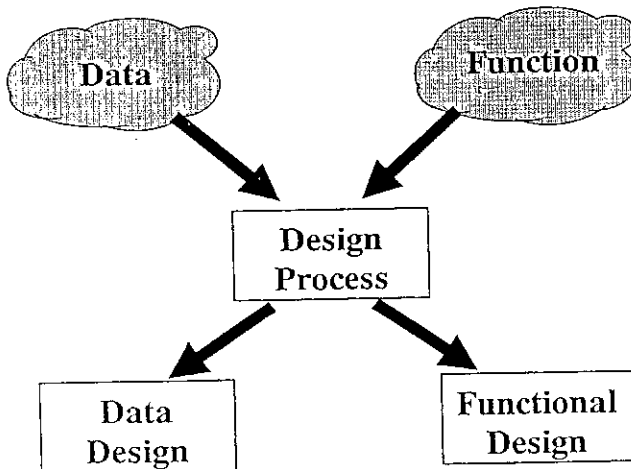


Figure 3: a slightly more detailed view of the design process

As mentioned previously, these may be presented in a wide variety of forms. The output produced by the designer is a structured design for both the data and functional aspects of the system. As a minimum, the design must provide details of:

- data requirements, structures and relationships
- functional requirements

The design may also provide (depending on the complexity, size and criticality of the system):

- details of the user interface
- security and integrity requirements
- quality requirements

and so on....

Figure 4 shows a somewhat different view of the design process, by breaking it into two smaller processes. The first sub-process is one of analysis the main aim of which is to increase the understanding of the system's requirements and aims. This understanding is achieved through the construction of a model of the system. As with all models, this is a simplified view of the system and as such is not implementable. Typically, the model is very independent of the implementation environment.

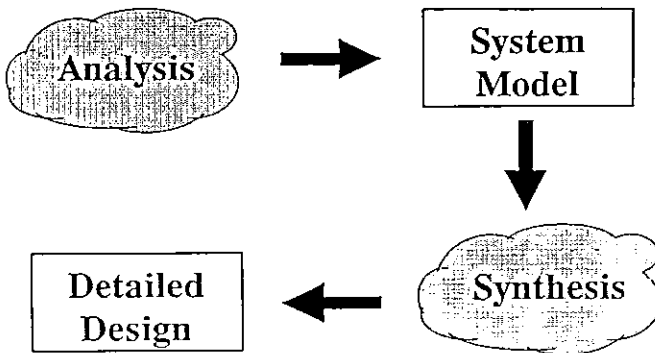


Figure 4: the design process, as two smaller processes

The second sub-process is one of synthesis from the model. The model is modified and expanded in order to increase the level of detail and to take into account the implementation environment. The resulting detailed design is (more-or-less) implementable.

A Design Framework

Both these views of the design process can be combined into a single design framework. The process is broken into the two sub-processes - analysis and synthesis. Each sub-process is made up of two phases, the first concerned with data the second with function. This framework is illustrated in figure 5.

Design Framework

	Data	Function
Analysis		
Synthesis		

Figure 5: a Design Framework

Within the framework, specific analysis techniques have to be placed in each box. There is no fixed recipe by which the design can be compiled; this degree of flexibility allows in-house or standard techniques to be adopted for any or all of the phases. It also allows the effort allocated to each phase of design to reflect the characteristics of the system.

Phases can be omitted from the framework if equivalent activities have been carried out in advance and sufficient information is available to complete subsequent stages. The effort put into each phase can be adjusted to match the

information available, the application and the constraints imposed by the business.

Data Analysis

The prime input to the first phase of the design process is the statement of systems requirements. Other information will be determined by interviewing users etc. Traditional information gathering techniques are the order of the day. More formal techniques such as data normalisation can be used to derive the final data model (figure 6).

Design Techniques

	Data	Function
Analysis	Data Analysis ↓ Data Model	
Synthesis		

Figure 6: Design Techniques

The key aim of data analysis is the production of an abstract data model to aid the understanding both of the systems designer and of the users. If an entity-relationship model is used the data model will consist primarily of:

- Entity/Attribute definitions
- Entity/Relationship diagrams

Together these form a 'data dictionary' which can be used through out the implementation of the system.

The entity relationship model provides the benefits of a simple view of the data. If the modelling is performed accurately, the view will also be correct. Since the model is a simple one, presented in diagrammatic and textual form it fulfils the requirement of being communicable both to users and to other technical staff.

The data model should not be considered directly for implementation. This is true even if a relational database system is being used to store the data. IBM has published an extensive manual (ref GG24-3004) which provides guidelines and heuristics for moving from a data model to an implementable relational database in DB2 (or SQL/DS).

Function Analysis

Once the data model is complete, the systems requirements, interview notes and data model can be used to derive the functional model of the system. This step is performed after data modelling since the characteristics of the data model can have a profound effect on the function model (figure 7).

Design Techniques

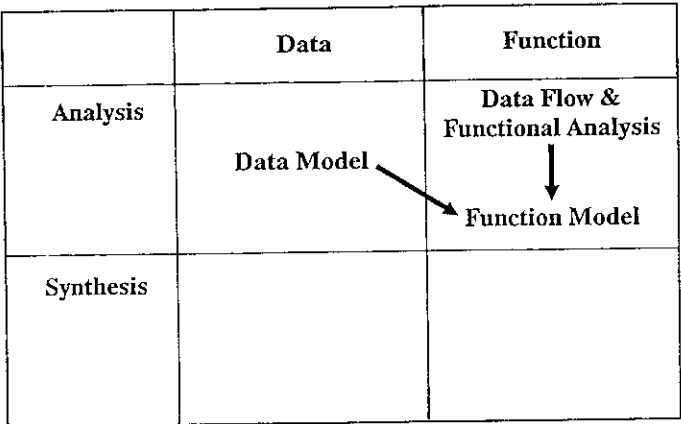


Figure 7: The Data Model can affect the Function Model

The data model is used to identify the commonly used access paths and the most commonly used relationships. Diagraming techniques such as data flow diagrams can be used to illustrate the order of processing and the data used by

each function. Function analysis can show overviews of the dialogues, the data relationships to be used by each part of the system as well as the entry point into the data structures.

As with the data model, the functional model provides a high level view of the system. Note, however, that Diagraming techniques such as data flow diagrams are susceptible to continued refinement. It can be very easy to carry the process too far. This may have a damaging effect on the later design and implementation phases.

Function analysis completes the model of the system. The end of the analysis process presents an ideal opportunity for a review of the application and the design process so far. The users should be able to confirm that the model accurately reflects the requirements for the application.

It is also at the end of the analysis phase that accurate project costs can be estimated from the system model. The model provides a sufficiently detailed view of the system to allow accurate estimates to be made. These may be made by a process of task analysis or by generating function point counts from the model. Function points can be combined with a knowledge of delivery rates to estimate final costs. It may be desirable for comparative reasons to estimate the system using both methods.

Data Design

Data design is the first stage of design synthesis from the system model. The aim of data design is to produce a series of implementable data structures from the system model. The design of data structures depends critically upon the implementation environment. For example the design for an APL component file may be completely different for that produced to store data in a VSAM cluster.

Data design also involves the design of significant data structures that will exist whilst the application is executing. Just as the design of file structures depends crucially on the type of file store, so the design of transient data structures depends crucially on the language being used. Even in the APL world, the presence or absence of features such as nested arrays may require significant alterations to a given design. This is one reason why simple migrations from non-nested implementations of APL to nested array implementations rarely give significant benefits in terms of either performance or maintenance effort.

The dependence of data design on the implementation environment requires the designer to have a considerable level of experience and knowledge of the environment in question. The final data design must take into account both the

data model and the function model in order to be fully effective. The function model is important since it indicates the most important relationships and entry points to the data structures (figure 8).

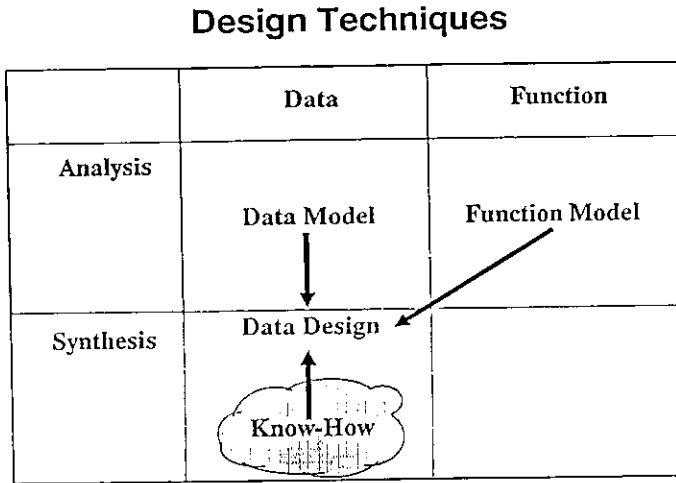


Figure 8: The Final Data Design

Data design is in many ways the most crucial phase for the success of the system; it can have a very significant effect on the flexibility and efficiency of the resulting application. Errors made at this stage may propagate into poor functional design and implementation later in the project. Of course, errors at this stage may be caused by poor analysis or specification work earlier in the project.

Function Design

The last stage of the design process is functional design. This process takes the functional model and refines it into a full, detailed design. The process is based on the functional model but is guided in its direction by the data design (and to a lesser extent the data model).

The function model is used to provide the high level entry points. The design is enriched and guided by a process of functional decomposition. The decomposition is guided by more detailed functional requirements and by the data model (figure 9).

Design Techniques

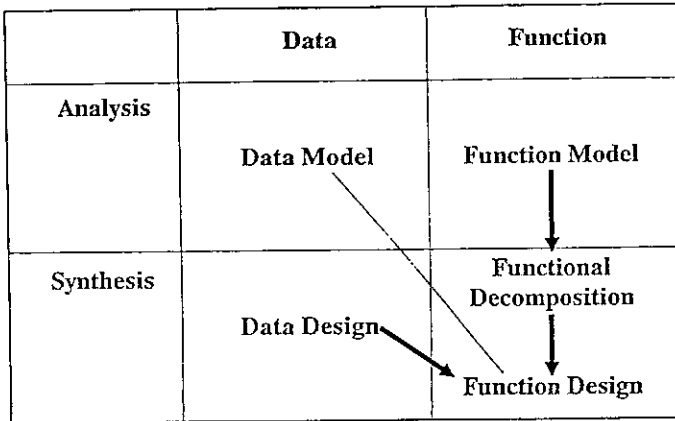


Figure 9: Functional Decomposition

One of the most important aspects of the functional design is the ability to measure its quality. The quality of individual functions can be measured by their coherence (or strength). The quality of the links between functions can be measured by the coupling.

Other, ad-hoc, measures can be applied to the design, these include aspects such as decision splits, the use of 'tramp data' structures etc. Qualitative judgements about design morphology, function position, content and style can also be made.

With the completion of the functional design, the systems design phase is at an end. This presents another useful project review point. Estimates can be reviewed and refined in the light of the detailed design.

Balanced Design

The adoption of design framework such as the one described above allows a balanced approach to be taken toward the design of each application. The balance is determined by taking account of:

- data complexity
- data volumes
- function complexity
- functional scope
- quality requirements

For example, the design process for data complex systems should emphasise the data modelling and design aspects, whereas function complex systems (often in the OR environment) should emphasise function modelling and design. The overall effort involved in design is determined by the system size and overall complexity (figure 10).

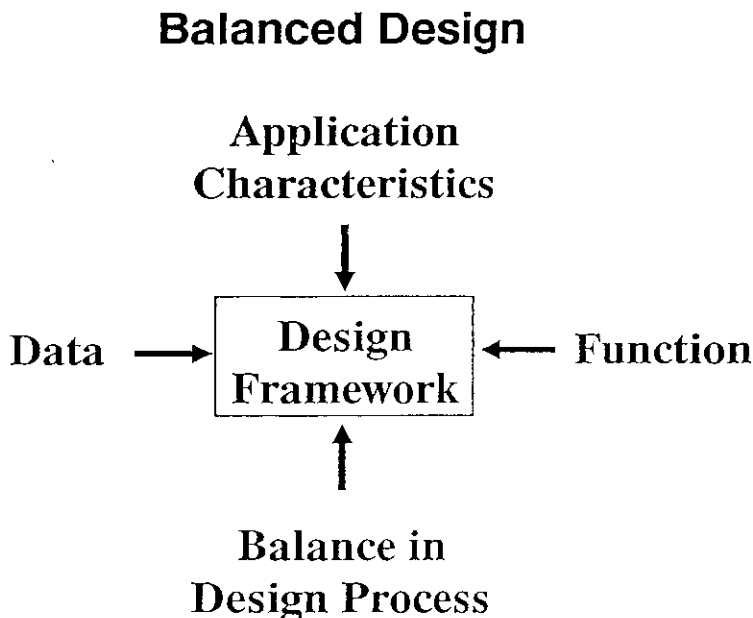


Figure 10: Balancing the Design Effort

Conclusions

This paper has described a formal approach to the design of APL business systems. With practice, these techniques become second nature and much of the formality can be dropped. There is an investment to be made in learning the techniques, but the pay back is large, comes quickly and carries on indefinitely.

The use of a design framework has a number of important advantages for the design of APL applications:

- separation of the data and function components of design
- separation of the analysis and synthesis phases
- sensitivity to characteristics of the application

These advantages allow resources to be allocated to the various phases of a project in an optimal manner. A design framework also allows resources to be allocated during the design phase in a manner that is close (or at least closer) to the ideal.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner: The Falkoff Keyboard	Adrian Smith	92
Technical Correspondence	Peter Branson et al	95
Error Trapping in APL2	Norman Thomson	105
Problems for APL Buffs (II)	Zdenek Jizba	117

Hacker's Corner: The Falkoff Keyboard

introduced by Adrian Smith

I recently came by a fascinating IBM Research Report (due out shortly in QuoteQuad) by the venerable Adin Falkoff. This was entitled 'Dual Keyboards are Better', and was a well-argued case for retaining the dual-mode APL/Text arrangement. Having read his report, I then looked through some of my recent APL*PLUS/PC code, where I have strayed towards *i+ip* rather than *I+ip*; I have no doubt that Adin is absolutely right: the upper-case is much easier to read!

This is in interesting contrast with normal text, for example road-signs which are always in lower-case for better readability. With APL it turns out that there is far less chance of confusion between the capitals and many of the symbols, such as *oip*, to name but a few. The *ITALIC* nature of the APL character set helps to remove any remaining ambiguity.

Next, Adin turns towards the best keyboard layout, given that the 'classic' APL keyboard was heavily constrained by the geometry of the Selectric Golfball, and is in many places thoroughly illogical. His first precept is that if a symbol is shown on a key, then that is where it should go; obvious instances being `[]` and `<>`. This immediately breaks up the nice grouping of `+ - * /`, but has many compensating advantages. In general the overstruck symbols can go on the Alt-base symbol, for example `≤ ≥` are on ALT-`<>` and so on.

I have been running both APL*PLUS/PC and my mainframe keyboard with a sort of ad hoc mixture (I put the `[]` on the `[]` keys, for instance) for years, so it was very little trouble to switch APL*PLUS/PC to use Adin's layout. Of course it required a little adaptation for the UK keyboard (`` stayed with ``), but I have very quickly become accustomed to it, and would like to offer it to Vector readers to try out.

ALL

```
41 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
    26 27 30 31 32 33 34 35 36 37 38 39 40 44 45 46 47 48 49
    50 51 52 53
```

AFKEYS

```
\1234567890-#qwertyuiop[]asdfghjkl;'zxcvbnm,./
|!:"£$/^&*()_+~QWERTYUIOP{}ASDFGHJKL:@ZXCVBNM<>?
\1234567890-#qwertyuiop[]asdfghjkl;'zxcvbnm,./
|!:"£$/^&*()_+~QWERTYUIOP{}ASDFGHJKL:@ZXCVBNM<>?
\1234567890-#qwertyuiop[]asdfghjkl;'zxcvbnm,./
|!:"£$/^&*()_+~QWERTYUIOP{}ASDFGHJKL:@ZXCVBNM<>?
```

This layout is suitable for the UK Enhanced keyboard. If you run the APL*PLUS standard system (users of the extended system or APL*PLUS II can have fun typing in lots of key= lines in their config.APL) you can hit the whole keyboard at one shot with the following little function:

ZZ←ALL KBΔSET AFKEYS

```
∇ R←KEY KBΔSET DEFN;MODE;TAB
[1] R←REPLACE CURRENT KEY DEFINITIONS WITH TABLE SUPPLIED.
[2] R
[3] R <KEY> IS A VECTOR OF KEY NUMBERS
[4] R <DEFN> IS A MATRIX OF CHARS WITH AS MANY COLS AS THERE
[5] R ARE ELEMENTS IN <KEY>.
[6] R
[7] MODE←0,1~1+1+ρDEFN
[8] TAB←2561□PEEK 198 197
[9] R←(1+□AV:DEFN)□POKE TAB+MODE*..+8×KEY-1
[10] R←'Keys redefined .... ',,ρDEFN,[1]' '
∇
```

I suggest you try it for a day or two. I feel that for anyone who programs in languages other than APL, but wants to keep their code as readable as possible, it is a big step forward over the existing APL/text arrangement, or over any of the dual layouts I have seen. Comments please!

The M61 Problem: a Partial Resolution

For readers who own (or are thinking of buying) a PS/2-M61, the keyboard problem highlighted in the last Vector has been partially resolved. Apparently, IBM have changed to a new keyboard controller, which affects the scan codes sent to your application. Of course there is a new keyboard handler to cope with this, so mostly you don't notice a thing.

However, when APL*PLUS/PC attempts to flash the keyboard lights (e.g. if you hit Caps Lock), or if your code attempts a peek to see if Caps Lock is down, it resets the driver to the old version!! This totally screws the keyboard, as reported in Vector 6.3. The fix is:

```
0 □POKE 879      a Cancel keyboard lights
```

... this appears to be OK in the APL*PLUS/PC standard system versions 8/9. Obviously, it needs to be the very first line in the very first function in *INIT0*. It worries me slightly that on a couple of occasions I got the problem in a *CLEAR WS* before I had even touched a key, and on one occasion I got it in DOS having simply done *OFF*. This suggests that there may be at least one more trigger in there somewhere, which will sooner or later catch me out!

As far as I can test, the extended system (APL*PLUS/PC or APL*PLUS II) does not show the problem. Watch this space.

TECHNICAL CORRESPONDENCE

Yet more VCAT ...

From: Peter Branson

6th March 1990

Norman Thomson's response (Vector Vol.6 No.3) to my Education Vector *VCAT* example does raise a couple of interesting points (not to mention Adrian's timing tests!).

Firstly, I don't know what a 'modern didactic view' is, but if we want to use some APL2, why not try a half-and-half approach ($\square IO+1$):

```
R←X VCAT Y
X←(⌊2+1 1,ρX)ρX
Y←(⌊2+1 1,ρY)ρY
R←∘(∘[2] X),∘[2] Y
```

Fairly obviously, this will handle scalars and vectors also. Norman's approach means carrying two names for a simple job, and I really don't like doing that. Working with a plethora of production workspaces, I try to keep to the absolute minimum the number of names that I might need to copy around.

A more serious point was raised in his first paragraph; namely should we introduce beginners (kids and grown-ups alike) to the APL2-type enhancements at an early stage (irrespective of 'free' software)? With a requirement in the near future to start training one or more graduates in APL, I have been pondering this question already, and personally I think not. A full grounding in APL1 is essential if the person is going to be maintaining existing code (as most professional programmers are likely to be doing at some time or other) and he or she must be able to recognise the older idioms too. To bring in the enhancements as well is, I think, a bit too much in the early stages.

The final point is raised by Adrian's timing tests. There are times when good APL1 is needed for efficiency, so a thorough grounding is essential. One small example of this arose when I joined my present company last year. They have a standard utility written using many APL2 features for searching a workspace for occurrences of a character string. Unfortunately, response was poor at the time, and it was rather boring waiting for the result to show. I re-wrote the function with a few APL2 bits, and used a localised function for the individual object searches, leaving only one 'each'. Result: about 25 times faster, and much less boring!

Peter Branson
EDS Ltd.
11 Wellesley Rd.
Croydon CR0 0XX

Is Function Ambivalence a Good Thing?

From: David Piper

March 1990

I am amazed by the extent of the feedback generated by my brief letter concerning an 'ambivalent operator'. The concept was originally simply an exercise for my own intellectual interest and was not (as I hope my response to Claude Henriod in Vector[6;3;93] made clear) a serious attempt at a real solution. Clearly the subject is of interest to many APL users. The suggestion of a potential solution (however great its shortcomings) to the problems of making use of ambivalence begs more serious questions. The most fundamental must be:

"is function ambivalence a good thing?"

To be a 'good thing' we presumably mean a facility must provide useful functionality without sacrificing anything (or at most sacrificing less than we gain). What facilities does functional ambivalence provide? - quite simply the ability to elide the left argument of a defined dyadic function. We use this capability to either:

- supply a default value for the elided argument (as with the subtract vs minus or divide vs reciprocal primitive functions)
- undertake some different but possibly related action (as with the multiply vs signum or add vs conjugate primitive functions).

What, then, is the cost of using function ambivalence? If used to supply default values for the elided argument, the effects are usually not too serious. Criticism

can be made about defining data (i.e. the default value) procedurally rather than as a parameter, but this is rather theoretical. The real weakness can be seen if we examine what happens when the default has to change. The code making use of the default either stops working, or, worse still, continues to work but provides different (presumably invalid) results. Of course, we could always create a brand new function for each different value of default.

When used to implement two separate functions the cost of ambivalence is greater. By implementing two separate functions in a single unit of code we increase the complexity of the code written (back to the start of the discussion; we have to write code to handle the ambivalence). We also reduce the coherence (strength) of the function.

If the two functions are well separated, we may be able to classify the function as having "informational cohesion" (Myers, *Reliable Software through Composite Design* 1975). This is only slightly inferior to the best case "functional" cohesion and no significant maintenance or quality problems will be introduced. On the other hand, if the functions are not well isolated, the best classification we can assign is "logical" cohesion (Page-Jones, *The Practical Guide to Structured Systems Design* 1988) with the maintenance and quality problems such a poorly designed function normally introduces.

Note that for both types of use of ambivalence, the coupling between the function and its superior calling functions is increased. It is likely that the level of coupling will be reduced to that of "control" coupling (see Page-Jones). The argument is not only being used to pass data to the sub-function, information is also being passed by the presence or elision of the argument. Since data and control information are passed with the same piece of data, the coupling is more likely to be classed as "hybrid" Page-Jones makes a good job of explaining the problems associated with this type of coupling.

So, then, we are left to consider cases where it may be valid to use function ambivalence. In "open" systems or utilities, ambivalence can be used to enrich the syntax of the language defined for the user. Even here, it should be used with great care, especially if ambivalence is used to set default parameters.

With regard to the contribution from Phil Last (Vector[6;2;117]), I certainly would not countenance the use of `FIX` to fix functions on the fly. Using this technique, we manage to descend right to the bottom of the quality ladder, to the depths of "pathological" (or content) coupling - one function is being used to modify the contents of another!

The quality impact of using `execute` as advocated by Claude Henriod and Norman Thomson (Vector[6;3;94 97]) is harder to quantify but still smacks of pathological coupling; my general reaction to it (and all uses of `execute`) is YUCK!! I guess, on those occasions when I have (or want) to use ambivalence, I shall have to resort to the test and branch syndrome. At least in APL*PLUS I can use the diamond separator to get the test and the branch on a single function line.

Whether the costs in terms of design quality, discussed above, are greater than the benefits derived from the use of function ambivalence is largely a matter of personal judgement. My own opinion is that except for "open" systems cases, the potential costs far outweigh any short term benefits. I hope this provides some helpful (recursive) feedback and clarifies my viewpoint.

D B Piper
41 Sandown Drive
Rainham
Kent ME8 9DT

Error Trapping or Event Handling?

From: Richard Nabavi

14th March 1990

I have been following with interest the recent VECTOR articles on Error Trapping in its different manifestations within APL*PLUS, Dyalog, IPSA, APL68000 etc. But may I make a plea for greater terminological exactitude in describing what these facilities actually do?

My gripe is a simple one. What I used to think of as *Error Trapping* is being termed *Event Handling*. This term is used by Dyalog, by I.P. Sharp, and even by IBM in the APL2 reference material. Now it seems to me that if your APL code is happily tripping along, interpreting and executing statements according to the will of the sentient being who wrote the function, and suddenly it encounters a *SYNTAX ERROR*, then that is not an *event*, it is a *Bug*. The term *Event*, as used by most of the computing fraternity outside APL, is normally reserved for external events that meet two criteria: (a) they cannot be predicted by following the logic of the code, and (b) they can happen at any time, not just when the system makes a specific operating system call or carries out a particular instruction.

In fact, the only real Event which can normally be trapped in a conventional (non-windowing) APL environment is the user hitting the Attention key; this is indeed a true external event which can happen at any time, and which cannot be

predicted by following the logic of the APL code. It is clearly different in nature from a *VALUE ERROR* or a *RANK ERROR*. (What about *WS FULL*, or *SYMBOL TABLE FULL*? Ed.) However, in modern windowing systems, and on networks or sophisticated multi-user machines, a plethora of other external events can occur, which the APL system might need to know about. Examples of these true Events are:

- the user has selected a menu item from a pull-down menu
- the application's window has been overwritten by another task, and needs to be redrawn
- the network has sent a message containing an update of the \$ to Dmk exchange rate.

Now, as far as I know, these kinds of Events are not really handled by any of the trapping facilities which APL interpreters provide. Even APL68000 on windowing machines like the Macintosh - which does provide Event Handling through a library workspace - does not yet have these facilities built in to the APL interpreter itself, although the operating system interface is largely built around responding to events. So shouldn't we use the term Error Trapping to describe what we have got already, so that confusion can be avoided when we finally announce that APL now has Event Handling built in to it?

In fact, when you do start to think about true Event Handling in APL, you begin to realise that it is a difficult subject, and may require fundamental changes in the way APL systems are designed. But that is another matter; in the meantime, let us not use misleading expressions.

Yours sincerely,

Richard Nabavi
MicroAPL Ltd.

APL and MS Word for HP LaserJet

From: Rex Swain

28 March 1990

For quite some time, I have wanted to be able to use my word processor to mix APL with other fonts. I recently took the time to solve this problem, and have developed programs that teach Microsoft Word to treat STSC's APL font for the HP LaserJet as it would any other font.

Now, I can mix APL symbols such as $\square + \tau p$ with Helvetica and Times Roman and any other fonts I have, on the same line or in separate paragraphs. (*so can I Ed!*). I format the APL portions with the normal Word commands; Word automatically downloads the APL font when necessary. If you have the STSC APL video characters turned on when you invoke Word, you can toggle between Word's text and graphics display modes: in text mode you can see the APL characters on the screen; in graphics mode you can see the full PC character set.

Briefly, my programs modify a specified .PRD file, inserting the APL font definition and translate table; create or add to the corresponding .DAT file; and extract the downloadable APL fonts from STSC's PRINTERS.ASF file. I am willing to distribute these programs on a virtually freeware basis. Anyone who is interested may send me:

- a diskette (360K, or any density 3.5") and a stamped, self-addressed return mailer; or
- for \$10, or \$15 outside the US, (cash only; no checks) I will supply the diskette (specify size), mailer, and postage.

Requirements are:

- an HP LaserJet printer that accepts downloadable fonts (any model except the original 2686A),
- Microsoft Word version 5.0, and APL*PLUS/PC version 8 (or higher, I suppose).

Rex Swain
8 South Street
Washington, CT 06793
USA

Editor's Note: this is a rather faster and cheaper way of doing what I do with MS Word and GoScript. We both exploit the availability of APL character drivers (I use the Hercules RAMFONT approach) in similar ways, and we both rely on the excellently open architecture of MS Word.

*I would still recommend PostScript as the best long-term solution, but if you already have APL*PLUS/PC and an HP LaserJet, I think Rex's offer is a most generous and useful one. Many thanks.*

A New Development Environment for Dyalog APL

by John Scholes - Dyadic Systems

The following is a reprint with minor revisions, of an article which appeared first in APL-CAM the journal of the Belgian APL Users Society and is reproduced here by kind permission of Joseph De Kerf.

Abstract

Much of APL's productivity comes from its development environment. The ability to stop and review the state of execution at any point is a considerable help when producing code. However the procedure available on many APL systems is twenty-five years old and was designed for hard-copy devices. The new Dyalog development environment makes use of memory-mapped screens to give the APL developer a modern 'visible computing' interface.

Introduction

APL is renowned for the speed with which you can produce programs. This speed is due partly to the language itself and partly to the power of APL's development environment.

Much of the power of the language, comes from its ability to let us think at a more abstract level. This means that the language itself frees us from worrying about the more mundane low level tasks present in other programming systems. Tasks such as space allocation and the scalar processing of arrays. However, perhaps not quite so apparent but equally important, is APL's provision of a powerful development environment.

Much of the time spent in producing software is in the refinement of programs. This means in reviewing defined functions; in testing and changing both individual lines of APL and in moving groups of lines inside and between functions.

When APL first appeared in the mid-sixties, no other popular programming system provided development tools nearly as powerful. It gave us:

- Visual fidelity or what is now called WYSIWYG input, albeit only for the current input line.

- The ability to stop the execution at any line of code and examine and change local variables.
- An editor with which we could examine and correct the currently stopped function.
- The ability to restart the function and continue processing.

However, that was a quarter of a century ago and many of the constraints placed on its design have long since been lifted. The original APL ran on a 14.7 cps (~150 baud) hard-copy typewriter. Now we have, in general use, colour screens where the whole screen can be updated 'instantaneously', that is, between successive refreshes.

Dyalog has been able to use this technology to produce a much more powerful development tool. We have not changed the language in any way in doing this and in fact, you may write your code without using the new facilities, although they are always close at hand if you do choose to.

The New Environment

The environment consists of three main parts:

- The SESSION where lines of APL are entered, evaluated and results displayed.
- An EDITOR in which lines are gathered to form functions and operators.
- A TRACER, used to monitor the detailed execution of statements.

An important property is that wherever possible, the environment is 'seamless'. This means that we can move freely among the various parts of the environment without having to change the syntax that we use to manipulate code lines.

An example is the <EXIT> key which is used to close both TRACER and EDIT windows. Another example is with string searching. The search works identically throughout the system so you can search for a string in a function and then tab to the session and continue the search there.

This scheme has a major impact on productivity as it allows you to concentrate on the task at hand without worrying about the superficial problems of which 'mode' you are in.

We must all have suffered for instance, at one time or another, the distraction of wanting just to exit from the current environment and having to lose our train of thought in order to experiment with: quit, bye, x, Control-C,)OFF, exit, Esc ...

Here is a brief overview of the parts of the development environment.

The EDITOR is window-based:

```
)ED FN1 FN2 ...
```

... opens a window for each function. The default windows occupy the top right hand quarter of the screen and overlap with just enough displacement to reveal the name of the function underneath. You can move and resize the windows at will and the <ZOOM> key toggles between this and full-screen mode where each function occupies the whole screen. <TAB> moves between the windows and to the session.

The editor itself is modeless in the sense that operations on the text of the function, such as moving and copying blocks of lines, use special keys away from the main "QWERTY" part of the keyboard. Typing "QWERTY" keys always results in text appearing in the function.

When editing a function, we can, using the <TAG> key, identify a block of lines of code. The tagged block may then be deleted, or moved or copied elsewhere. To copy the block to another function, we <TAB> to the function, cursor to the required line and <COPY> the block. If we <COPY> a block of lines to the session, the lines are executed sequentially. Similarly, a block of lines in the session may be <TAG>'d and copied into a function. An edit window may also be opened for a function by cursoring to its name anywhere on the screen and typing the <EDIT> key. This feature called "point and edit", is a great time saver.

An unusual feature of the editor is that it maintains a complete history of changes. This means that you can undo not just the last change made to the function, but using the <BACK> and <FWD> keys undo and reapply each change made since entry to the editor.

The SESSION allows you to enter lines of APL for immediate execution in the normal way. It maintains a history so that previous lines may be changed and resubmitted. When you type over a line in the session, the line is highlighted and we say it is MARKED. You can mark any number of lines in the session and when you type <ENTER> each marked line is copied to the input line (restoring the session line to its original state) and executed.

The TRACER lets you examine the execution of a line of code. If you terminate a session input line with <TRACE> instead of <ENTER>, each defined function (or operator) in the line is displayed in a window as it is entered. The function is suspended on the first line, and you can choose either to execute the line by typing <ENTER> or trace it by typing <TRACE>. In this way, you can step through each line of the execution of an APL statement. At any point in the trace, you can of course, tab back to the session and execute other lines of code. The overlapping windows during a trace represent the state indicator.

Responding to the trace with the <EDIT> key, changes the current trace window into an edit window. This allows extremely easy access to the top suspended function allowing you to change the function "on the fly" and <EXIT> the edit to resume tracing.

Within the tracer, you can use the <BACK> key to repeat and <FWD> key to skip lines in the function.

The <EXIT> key from within the tracer, removes the top window from the stack and suspends the calling line. In effect, it cuts the state indicator back just one level. This effect, which seems completely natural in this environment, and which is comparatively clumsy in other APL systems, is a powerful example of the correctness of the seamless approach.

The TRACER may also be started retrospectively. That is, if the interpreter encounters an error during execution of your program it suspends execution in the normal way. You can then hit <TRACE> and have the system recreate the trace stack as if you had traced your way to the point of suspension. You can then continue tracing from that point. Typing JLLC in the session during a trace resumes normal non-traced execution of the program.

Conclusion

Trying to give a flavour in words, of a piece of software which owes much of its power to the way it behaves visually is difficult. If you would like to see the development environment in action, you can obtain a free demonstration disk from Dyadic Systems or any Dyalog APL distributor.

Dyadic Systems Ltd,
Park House,
The High St.
Alton, Hants GU34 1EN

Tel 0420 87024
Fax 0420 89886

Error Trapping in APL2

by Norman Thomson

Sadly error/event handling is an area of APL which has escaped standardisation as readers of the four previous articles in this series (see Vectors 6.1 and 6.3) will know. There is no logical reason for this - compare for example ON units in PL/1. Because of the divergences of event-handling philosophy between different products, comparisons of system functions and variables are at best approximate, but with this proviso, and armed with the information in the previous articles a table of rough correspondences is as follows (APL2 refers to the IBM mainframe and PC products):

	STSC	68000	Dyalog	IPSA	APL2
Predetermined Event Action	<code>⌈ELX</code>	<code>⌈ERX</code>	<code>⌈TRAP</code>	<code>⌈trap</code> <code>⌈ec</code>	<code>⌈EA</code> <code>⌈EC</code>
Saved Event Message	<code>⌈DM</code>	<code>⌈ERM</code>	<code>⌈DM/⌈EM</code>	<code>⌈er</code>	<code>⌈EM</code>
Numeric Event Code		<code>⌈LER</code>	<code>⌈EN</code>		<code>⌈ET</code>
Event Simulation	<code>⌈ERROR</code>	<code>⌈ERS</code>	<code>⌈SIGNAL</code>	<code>⌈signal</code>	<code>⌈ES</code>

Note that `⌈ec` (environment condition) in IPSA is quite different from `⌈EC` (execute controlled) in APL2.

Next observe that in APL2 `⌈EM` stands for "Event Message" as opposed to mere "Error Message," thereby indicating that the power of the associated facilities goes far beyond that of error handling. In Vector 6.3 Peter Biddlecombe stresses this point on behalf of the analogous facilities in IPSA. An Event is either an Interrupt or an Error, and Errors can arise either as a result of statement execution or the issue of a system command.

Predetermined Event Action is often used as a mechanism for rerouting. APL2 offers it at the statement level, 68000 at the function level, STSC, Dyalog and IPSA at the global, i.e. workspace level, or at the function level through localisation of `⌈ELX`, `⌈TRAP`, `⌈trap` and `⌈ec`.

In every case the event message is available either as a function in the case of STSC, 68000 and Dyalog, or as a matrix in the case of IPSA and APL2. Dyalog offers it as a nested vector (`⌈DM`) as well as as the result of the function `⌈EM` applied with `⌈EN` as argument. 68000, Dyalog and APL2 offer an explicit coded numeric form, either as a scalar (`⌈EN`), a 2-item vector (`⌈ET`) or a function (`⌈LER`). 68000 additionally gives a line number as the second item of the result of `⌈LER`.

All systems provide the ability to simulate events - Dyalog, IPSA and APL2 all provide the capability of defining user event-types by number.

There is little consistency between products concerning the code numbers given to the various error types. What one might call the "seven deadly sins" of APL, do have a common numbering across 68000 and Dyalog, viz.

- 1 WS FULL
- 2 SYNTAX
- 3 INDEX

4 RANK
5 LENGTH
6 VALUE
11 DOMAIN

(WS FULL is clearly Gluttony, and SYNTAX ERROR Sloth, thereafter the allocation of the remainder becomes more idiosyncratic!) The APL2 system of coding is quite different - viz.:

Resource Errors : 1 1 INTERRUPT
 1 2 SYSTEM ERROR
 1 3 WS FULL
 1 4 thru 1 12 SYSTEM LIMIT of various sorts

SYNTAX ERROR : 2 1 thru 2 5

VALUE ERROR : 3 1 Name with no value
 3 2 Function with no result

Implicit Argument Error : 4 1 thru 4 5 and 4 7 corresponding to
 □PP, □IO, □CT, □FC, □RL, □PR respectively.

Explicit Argument Error : 5 1 VALENCE ERROR
 5 2 RANK ERROR
 5 3 LENGTH ERROR
 5 4 DOMAIN ERROR
 5 5 INDEX ERROR
 5 6 AXIS ERROR

The primary tools for dealing with events in APL2 are □EA and □EC.

Execute Alternate

□EA gives the user the opportunity to try out the right argument as an execution string, and if this fails, withdraw and try something else. It can also be used to eliminate testing and branching, e.g. to assign a default value of 10 to T use

```
T←'10' □EA 'T'
```

Here is an example where a function MEAN is intended to return the mean of a frequency table, which is modelled as a matrix with the first column representing values and the second frequencies:

```
[0] Z←MEAN R
[1] Z←(+/×/R)÷+/R[;2]
```

F

```
1 4
2 6
3 1
```

MEAN F

1.7272727

One way to process a vector right argument is to write an alternative function MEANV and select it if necessary using `⊞EA`:

```
[0] Z←MEANV R
[1] Z←(+/R)÷ρR
```

V←15

'MEANV V' ⊞EA 'MEAN V'

3

Suppose the program's author believes that it is reasonable to anticipate two types of user error, namely those of supplying a vector R instead of a matrix, and of attempting to use the function in an environment with index origin zero. To counter these possibilities and provide a catchall for other user imbecilities he might write a further function MEANA:

```
[0] Z←MEANA R
[1] '' ⊞EA '→0 Z←MEAN R'
[2] '⊞IO←1' ⊞EA '→0 Z←MEANV R'
[3] 'Z←''CANNOT IDENTIFY MISTAKE'' ⊞EA '→0 Z←MEAN R'
```

MEANA V

3

⊞IO←0

MEANA F

1.7272727

MEANA 'ABC'

CANNOT IDENTIFY MISTAKE

An alternative form uses `⊞ET` to go directly to each particular error action, with the standard APL message being generated in the case of a non-identified error (see line 3):

```

[0] Z+MEANB R
[1] ' ' □EA '→0 Z+MEAN R'
[2] →(L1,L2)IF(□ET)≡"(5 2)(5 5)
[3] →0 Z+□EM
[4] L1:→0 Z+MEANV R
[5] L2:Z+MEAN+R(□IO+1)

```

where IF is defined as

```

[0] Z+L IF R
[1] Z+R/L

```

The next example shows the use of □EA and □ET in an operator which returns a 1 or zero depending on whether its argument is in the domain of its operand:

```

[0] Z+(P DOM)R
[1] □ET→0 0
[2] Z+'□ET→0 0' □EA '(P R)≡P R'

```

```

1DOM"3 3.1 'A'
1 0 0

```

DOM can be extended in an obvious way to deal with the case of dyadic P.

The next example demonstrates the availability of □EM in the course of the alternative action:

```

[0] Z+STRONGMSG R
[1] Z+' , [10]6p<+c[2]□EM' □EA R

```

```

STRONGMSG'3+0'
DOMAIN ERROR
DOMAIN ERROR
DOMAIN ERROR
DOMAIN ERROR
DOMAIN ERROR
DOMAIN ERROR

```

Execute Controlled

The result of $\square EC$ is a 3-item vector. For example

```

      (RC ET Z) ←  $\square EC$  ' +O '
      RC
0
      ET
5 4
      Z
DOMAIN ERROR
      +O
      ^
      pZ
3 12

```

RC (Return Code) has one of 6 values, viz:

- 0 Error
- 1 Normal
- 2 Assignment
- 3 No result, e.g. $\square$ ' '
- 4 Branch
- 5 Abort, i.e. $\rightarrow$

Note that unlike the situation with shared variables a zero return code indicates the presence rather than the absence of an error.

ET is the event type (see above for codes),

Z is one of :

- Result (if right argument is valid APL string)
- Empty if right argument has no result
- $\square EM$ otherwise.

Continuing the first example, $\square EC$ can be used to provide a universal mean function which protects the user from APL error messages as follows:

```

[0]  Z←MEANU R;RC;ET
[1]  →0 IF 0≠↑(RC ET Z)←DEC 'MEAN R'
[2]  →0 IF 0≠↑(RC ET Z)←DEC 'MEANV R'
[3]  IO←1
[4]  →0 IF 0≠↑(RC ET Z)←DEC 'MEAN R'
[5]  'CANNOT IDENTIFY MISTAKE'

```

```

IO←1
MEANU 15

```

3

```

IO←0
MEANU F
1.7272727
MEANU 2 3 2ρF
CANNOT IDENTIFY MISTAKE

```

Event Simulation

An event can be simulated by issuing e.g.

```
DES 5 1
```

where the right argument is any 2-item vector of non-negative integers which is immediately assigned to `ET`. The vector may be, but does not need to be one of the error codes listed earlier. If it is not we have a user-defined event. Data may thus be vetted by mapping some anticipated condition, e.g. `L` is not a matrix, into an event type (either error or user-defined), leading to the appearance of code such as the following at the head of functions:

```
DES 5 1 IF 2≠ρρL
```

If the condition fails (i.e. `L` is a matrix), `DES` returns the value 10 and control passes to the next line. The combination of `DES`, `DEA` and `DEC` provide a more general facility than mere rerouting. Here is a simple example comprising a pair of functions in which a valence error (code = 51) is simulated if the ultimate result is less than 10. The alternative action in line 1 of `FN1` is for `FN1` itself to be re-executed.

```

      ⍱CR''FN1' 'FN2'
Z←FN1      Z←FN2 R
Z←'FN1'⍱EA'FN2 ' ,⍱      Z←R
                        +0 IF Z<10
                        ⍱ES 5 1

      FN1
2+2      A no event simulated
4

      W←'ABC'
      FN1
9+9      A valence error simulated
8+8      A ... and again
W        A ... now a genuine error
2+2
4

```

⍱ES also has a dyadic form which allows a tailored event message as the left argument. For example the test for L a matrix given above would more usefully be written

```
'ARG NOT A MATRIX' ⍱ES 5 1 IF 2#ppL
```

⍱ERS in 68000 behaves in a similar way to IPSA's ⍱signal and APL2's ⍱ES. Dyalog's ⍱SIGNAL is similar to monadic ⍱ES. ⍱ERROR in STSC is also monadic but its argument is a character string like the left argument of ⍱ES - the action following ⍱ERROR depends on the setting of ⍱ELX.

Suspension and Control following an Event

Any APL interpreter detecting an error goes through a sequence of

- (a) detection
- (b) reporting
- (c) return of control

So far we have discussed the possibilities for user intervention between (a) and (b). Intervention is also possible between (b) and (c).

In APL2 the rule is that control returns up the calling chain to the last point where a ⍱EA or a ⍱EC is in place. If there is none then it cascades to the topmost function in the chain. This gives great flexibility as in the example below where the user types REP to obtain a selection of (in this case two) report functions REP1 and REP2. In the first of these there is no ⍱EA trap, and so following an error control passes back to the master function. In the second it is trapped at the level of the report function. Deliberate and obvious errors are shown in REP1 and REP2 - in practice errors would be more subtle and occur following chains of function calls, however provided that there are no intervening ⍱EA or ⍱EC's the

resulting action would be identical in terms of flow of control. In both cases APL error messages are concealed from the user.

```
OPT←'REP1' 'REP2'
```

```
[0] REP R
[1] '''NEXT'''⊖EA=OPT[R]
```

```
[0] REP1
[1] 'REPORT1'
```

```
[0] REP2
[1] '''NEXT2'''⊖EA'''REPORT2'
```

```
REP 1
NEXT
```

```
REP 2
NEXT2
```

⊖EA and ⊖EC thus behave in the same fashion as the "Cutback" option of Dyalog's ⊖TRAP or IPSA's ⊖trap.

Distinguishing Simulated Events from Errors

First notice that ⊖EM and ⊖ET are automatically localised within every function so that their current values relate to the function currently at the top of the execution stack. Automatic localisation of ⊖EM and ⊖ET means when simulating events one does not necessarily have to forfeit the power of detecting genuine APL execution errors. In the following development of an earlier example a non-simulated error results in the corresponding values of ⊖EM and ⊖ET being available in the final lines of FN1.

```
⊖CR''FN1' 'FN2'
```

Z←FN1	Z←FN2 R
L1:→TEST' ⊖EA 'FN2 ',⊖	Z←R
→0	→0 IF Z<10
TEST:→L1 IF 5 1≡⊖ET	⊖ES 5 1
⊖EM	

```

W←'ABC'
FN1
8+8      A event 5 1 simulated
W        A event 5 4 genuine error
DOMAIN ERROR
FN2[2]  →0 IF Z<10
          ^   ^
        FN2 W
          ^

```

Variations in APL2/PC

In APL2/PC the `⌈EM` and `⌈ET` resulting from the right argument of `⌈EA` are never placed on the execution stack, and are thus available only during execution of the left argument. If they are to be used in a rerouting sequence initiated by the left argument, they must be assigned to an ordinary variable before the execution of the left argument is complete and before any further errors occur whichever is the soonest. For example if in the function `STRONGMSG` some processing was required to intervene between the `⌈EA` line and the use of `⌈EM` then it would be necessary to assign it in the following fashion:

```

[0]  Z←STRONGMSG R:EM
[1]  Z←'→MSG,OpEM+⌈EM' ⌈EA R
[2]  →0 Z←'CLEAN FINISH'
[3]  MSG:Z←'IT WAS A'([10]6p←↑c[2]EM)

```

```

STRONGMSG'3+0'
IT WAS A      DOMAIN ERROR
              DOMAIN ERROR
              DOMAIN ERROR
              DOMAIN ERROR
              DOMAIN ERROR
              DOMAIN ERROR

```

Using Event Handling to Check Domains

Dave Ziemann emphasised in his article on a Direct Definition Handler in Vector 6.1 that the power of the Event Trapping facilities in all the systems reviewed goes far beyond that of error detection and control. As a further illustration this section shows how `⌈ES` can be used to provide the equivalent of compiled language declarations for function arguments and results. As well as the traditional attributes such as char and integer, APL2 objects have the additional attributes of shape and depth, so that an analogue of an set of internal variable attributes in a compiled language might be

(posint (=2) (<2))

which describes a simple matrix of positive integers. A reasonable standardisation for commenting functions might define domains in the first line and semantics in the second, viz.:

```

       $\forall Z+L$  FN R
[1]  A (boolean (=0)(=0)) + (char(>0)(=1)) FN (real (<2)(N))
[2]  A where ...

```

which would indicate that the left argument is a simple character array, the right argument is a numeric scalar or vector of indefinite depth, and the result is a single bit.

It is a small (?) step to translate a line such as the above into a sequence of $\square$ ES statements, with those appropriate to L and R at the head of the function, and those for Z immediately prior to the point or points of exit. As an example suppose that a function PLUS is designed to add numeric left arguments to character string integer right arguments with the proviso that no item in the result should exceed 10. The basic function is

```

[0]  Z+L PLUS R
[1]  Z+L+R

```

Suitably dressed with domain checks this becomes

```

[0]  Z+L PLUS R
[1]  'L OUTSIDE DOMAIN'  $\square$ ES 9 0 IF  $\sim \wedge /(\text{NUM } L), (2 > \rho \rho L), 2 > \Xi L$ 
[2]  'R OUTSIDE DOMAIN'  $\square$ ES 9 0 IF  $\sim \wedge /(\text{NCHAR } R), ((\rho \rho L) = \rho \rho R), 2 > \Xi R$ 
[3]  Z+L+R
[4]  'Z OUTSIDE DOMAIN'  $\square$ ES 9 0 IF  $\sim \wedge /(\text{NUM } Z), (\wedge / Z \leq 10), (2 > \rho \rho Z), 2 > \Xi Z$ 

[0]  Z+NUM R
[1]  Z+0= $\neq$ 0  $\rho \in R$ 

[0]  Z+NCHAR R
[1]  Z+ $\wedge / R \epsilon$  ' 0123456789 '

```

```
      2 3 PLUS '4 5'
6 8    2 3 PLUS '4 8'
Z OUTSIDE DOMAIN
      2 3 PLUS '4 8'
      ^  ^
      2 PLUS '4 5'
R OUTSIDE DOMAIN
      2 PLUS '4 5'
      ^  ^
      '2 3' PLUS '4 5'
L OUTSIDE DOMAIN
      '2 3' PLUS '4 5'
      ^    ^
```

Summary

The previous writers in this series have commented on the combination of power with simplicity which the comparatively small number of event handling facilities in their various systems offer. I endorse this on behalf of APL2, while adding a note of regret on the unnecessary variety which exists amongst different systems.

APL-2741: the Current Vector Font

A	
O	

Q	α
7	1

M	W
5	w

ρ	ϕ
$\in$	∇

S J G K H Z × W Y . E
T O Q D M F [U N ← I
5 7] 9 B + P R V C A
0 / X L , ~ ° ? [| -
1 2 3 8 4 6 ^ \ ⊃ □ ;
_ (↓ ↑ → √ * ρ ∪ ∩ α
̄̄ < ≠ ≤ ≥ = >) ∨ ⊥
⊂ ÷ ω † : ∈ Γ ° ∇ ! Δ
∓ ∇ ∅ △ φ ∅ ⊗ ∞ ∆ !
; \ / ⊕ ⊗ ∩ □ ⊙ Δ ∈ ≡

a b c d e f g h i j k l { } ☐ ☒ ☓ ☔ ☕ ☖
m n o p q r s t u v w x y z | £ \$ / & # " @
+ - . _ ¢ ~ ◇ □ ▤ ▥ ▦ ≠ ≪ ≫ █ ▬ ▭ ◊ ◌ β

Thanks to Adin Falkoff for the following comments: "The serifs are too large on a number of upper-case characters, especially the bottom of the *F*, the verticals on the *E*, the horizontals on *H* and *W*, and the tops of *P* and *E*; the upper slant on the *K* breaks too sharply where it meets the slant leg; the curl at the bottom of the *J* should have a slightly more bulbous end; the right half of the squiggle on the *Q* is too angular, as is the flourish on the bottom of the *a*; the top of the *ε* is perhaps a little flattened, or the bar is too high; and the slant of the numeral 1 should have a slight concavity, similar to the arrow heads, which are good. The special APL characters are generally good; my only quibble there is that the $\square$ is a little narrow for its height." *Will be fixed for 7.1! Keep 'em coming ... Ed.*

Problems for APL Buffs (II)

by Zdenek V Jizba

Chevron Oilfield Research Corporation (a subsidiary of CHEVRON) has a program called "Adopt-A-School". Under this program, employees volunteer to give short lectures on science to fifth and sixth grade schools in the La Habra area. On learning about this program, I volunteered as an annuitant to prepare talks under the title "computers as tools for thought".

The plan is to use I-APL. Using the direct definition mode, one can easily design a lecture that minimizes teaching hard-to-understand concepts, and maximizes the excitement of learning. For example, to avoid using special symbols, I define all primitive functions by their names:

```
COUNT: ⍵
```

```
ODDS: (2×COUNT ⍵)-1
```

```
REDUCE: ⍺, '/', ⍵
```

```
SCAN: ⍺, '\', ⍵
```

The first two functions are defined and discussed in the class, but the last two are merely illustrated with examples.

```
'+' REDUCE COUNT 100
5050
```

This particular example is used as an occasion to relate the story of Gauss, who as an 11 year old solved such a problem to the amazement of his teacher.

As I was preparing these lectures, I uncovered some other dusty corners of 17th century mathematics. This note takes off from here, and leads into the fascinating subject of integers.

One question asked of the class is "how can one find the square of a number without using multiplication?". After drawing squares on the black board, one can easily justify the following:

```
'+' REDUCE ODDS 5
25
```

In fact the next expression shows this to be true for the first 10 integers:

```
'+' SCAN ODDS 10
1 4 9 16 25 36 49 64 81 100
```

So we can generate a list of consecutive squares from a list of consecutive odd numbers. Since there are odd numbers that also are squares, we have a way to generate triplets such that the sum of the squares of the first two will equal the square of the third.

This can be best shown by defining function *FERMAT*:

```
▽ U←FERMAT W
[1]  generate all quadratic triplets
[2]  U←1+2×W
[3]  U←U,(((U+2)-1)÷2)×.+ 0 1
▽
```

(As an exercise, you might want to explain the formula)

```
F←FERMAT 0,110
```

We can look at this array:

	F	
1	0	1
3	4	5
5	12	13
7	24	25
9	40	41
11	60	61
13	84	85
15	112	113
17	144	145
19	180	181
21	220	221

and note that it includes the triplet {1 0 1} which certainly holds but does not seem to belong. We can verify that the sum of squares of the first two columns equals the square of the third column:

$$1 \quad \wedge / (F[; 3] * 2) = + / F[; 1 \quad 2] * 2$$

The question that immediately comes to mind, is whether this function generates ALL such triplets (we will of course ignore all multiples such as 6 8 10). Might there not be other ways to build yet more distinct triplets? But before we tackle this question let me explain why I named the function *FERMAT*.

In the 17th century, there was a mathematician names Fermat, who specialized in what is now called "Diophantine analysis". In this branch of algebra, only integers are acceptable solutions to problems. Around 1637 Fermat was reading a book written by Diophantus of Alexandria with the title of "Arithmetica". On the margin of one of the pages, Fermat wrote the following:

"To divide a cube into two other cubes, a fourth power, or in general any power whatever into two powers of the same denomination above the second is impossible, and I have assuredly found an admirable proof of this, but the margin is too narrow to contain it." (see p.509 "The World of Mathematics" J R Newman, ed. Simon and Schuster 1956")

This theorem became famous, because all attempts to reproduce his proof since that time have failed. As we will see, one can go a long way towards an insight into the kind of thinking that drove Fermat to his conclusion.

Coming back to our problem, the question is whether there are integers {a,b,c} such that $a < b < c$ other than those generated by function *FERMAT*. We will restrict the search only to those triplets for which the largest common divisor is 1. Clearly we must search sets where $1 < c - a$. It turns out that this question can be resolved not only for powers of two, but arbitrary powers as well.

In equation form we can write (It could be done using APL, but here algebraic notation seems better):

$$(1) \quad (b+k)^N - b^N = a^N$$

When $k=1$, and $N=2$, we have the solution in *FERMAT*. Therefore we want to study the case when $k>1$. First we rewrite (1):

$$(2) \quad (b+k)^2 (b+k)^{N-2} - b^2 b^{N-2} = a^N$$

and note that for $N=2$ the equation is reduced to (1). Expanding the square polynomial, and collecting terms we get:

$$(3) \quad b^2[(b+k)^{N-2} - b^{N-2}] + k(k+2b)(b+k)^{N-2} = a^N$$

Now the expression in brackets is the difference of two powers, and it can be factored as the product of two polynomials:

$$(4) \quad [(b+k)-b] \times P[(b+k), b, (N-3)]$$

Here the polynomial $P[(b+k), b, N-3]$ is shorthand for:

$$(5) \quad P[x, y, M] = x^M + x^{M-1}y + \dots + xy^{M-1} + y^M$$

But since $(b+k)-b=k$, we find that k can be factored out from the left hand side of equation 3. Let us assume that $k > 1$. Then, since $k < a$ (why?), we must have $a = i \cdot j$ where i and j are integers (why?). Assuming that ij , we can now rewrite (1):

$$(6) \quad ((b+k)/i)^N - (b/i)^N = j^N$$

Since $\{a, b, c\}$ have 1 as the greatest common divisor, the ratios $(b \div i)$ and $(c \div i)$ are not whole numbers. Can the sum of the powers of fractional numbers add up to an integer power? Let us assume that the answer is yes and pursue the consequences. We can use (6) to write the function *GFERMAT* to calculate triplets for arbitrary k .

```

      V U+K GFERMAT W;B
[1]  aGenerate all quadratic triplets for given K
[2]  U+((W*2)-K*2)÷2*K
[3]  +(0=p U+(B+U=(U)/U)/0
[4]  W+B/W
[5]  U+W, U+.+0,K
      V

```

When $k=1$ we should get the same result as with *FERMAT*:

```

1  GFERMAT 120
1  0  1
3  4  5
5  12 13
7  24 25
9  40 41
11 60 61
13 84 85
15 112 113
17 144 145
19 180 181

```

Now let us try this function with k=2:

```

2  GFERMAT 120
2  0  2
4  3  5
6  8 10
8  15 17
10 24 26
12 35 37
14 48 50
16 63 65
18 80 82
20 99 101

```

There seem to be three kinds of triplets. First, there is the permutation {3 4 5}. Next we have multiples of triplets generated with k=1 {6 8 10}. Finally there are some new sets {8 15 17}. We can try to remove the first two kinds by adding some more filters to *GFERMAT*:

```

▽ U←K PFERMAT W;B
[1]  ▽ Generate all quadratic triplets for given K
[2]  U←((W*2)-K*2)÷2*K
[3]  →(0=ρU+(B+U=[U]/U)/0
[4]  W←B/W
[5]  U←W, U←0, K
[6]  B←(∧/0=K|U)∨∧/0=2|U
[7]  U←(∼B∨>/U[; 1 2])÷U
▽

```

Now we can try out this function:

```

      2 PFERMAT 120
      8 15 17
     12 35 37
     16 63 65
     20 99 101

```

```

      3 PFERMAT 12000

```

```

      4 PFERMAT 12000

```

Why do you suppose we get results with k=2 only?

Finally, for k=1 and N=2 we have from (1):

```

      ^/(F[;1]*2)=+/F[;2 3]
      1

```

(Why?) When we can also show that "a" must be an odd number. Also note the following:

```

      (-/F[;3 1])÷2
      0 1 4 9 16 25 36 49 64 81 100

```

Can you explain it? At this point I stop. You of course may pursue this and entertain yourself. You may develop additional insights, and remember, you have a powerful ally in APL.

A Note on Comparison Tolerances in APL

by R.G. Williams and R.G. Green

$\square CT$ is a system variable in APL which is applied when any two numbers are compared. Two numeric values are considered to be equal if their absolute difference is less than or equal to the value of $\square CT$ times the larger of the absolute values of the numbers being compared. Hence the applied tolerance varies with the magnitude of the numbers being compared.

For the majority of applications, this system is quite satisfactory, however anomalies can arise which cause a breakdown in the usual rules of mathematics. For example:

```

A ← +1E16
B ← 0
□CT ← +1E15

A = B
0
(1 + A) = 1 + B
1

```

... which is contradictory to the assumption that if A, B, C are real numbers and $A=B$, then $A+C = B+C$.

The primitive functions affected by $\square CT$ are:

```

┌      ceiling
└      floor
>      greater than
<      less than
≥      greater than or equal
≤      less than or equal
=      equal
≠      not equal
|      residue
⊞      matrix inverse, matrix divide
┐      index
∈      belonging to

```

... and in Dyalog APL:

~	excluding
∩	intersection
∪	union

When the equality of numbers to within an absolute tolerance is required, the following expression could be applied:

$$((+|CT) \times |A-B) \leq 1$$

This will overcome the previously mentioned difficulties.

This method is also suitable for comparing any number with zero as, the = primitive in APL will always return zero when comparing any non-zero value A with zero, irrespective of how close to zero A ever becomes.

This APL expression can be adapted easily to provide absolute comparisons with other aforementioned APL primitives.

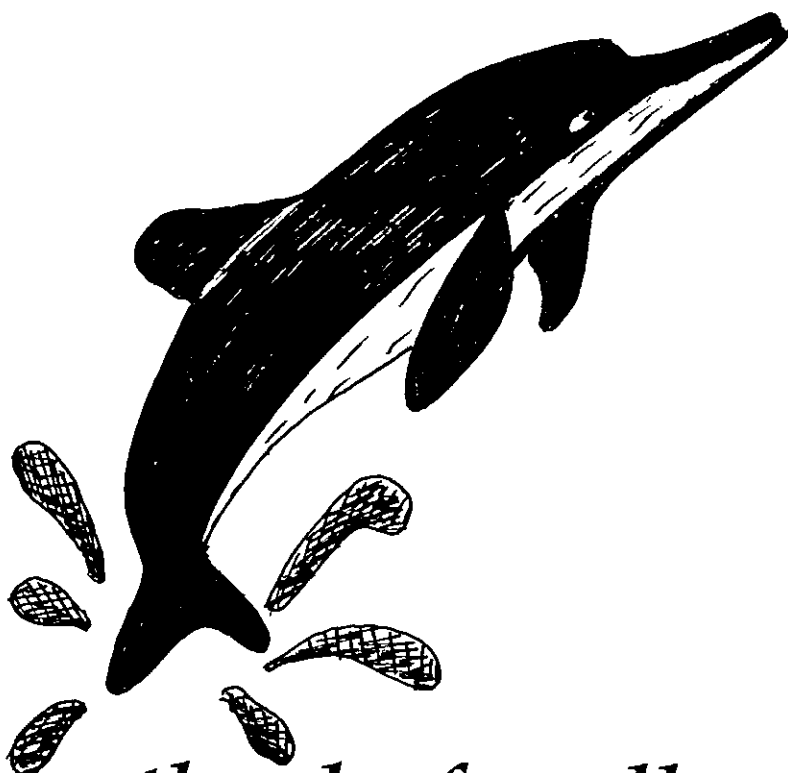
Editor's note: this is one of the few Heffalump traps set by conventional APLs. It recently caught out the 'nutritional analysis' in our recipe modelling system; this was supposed to ignore 'insignificant' traces of things like Nutmeg, but of course these never quite became equal to zero. The OR scientist who discovered this phenomenon made some distinctly unladylike noises when it was suggested that she add 1 to both sides of the equation!

Index to Vol 6

compiled by Adrian Smith

Title	Author(s)	Vol	No	Page
Meetings & Conferences:				
APL89		6	2	59
Conference Roundup	Anthony Camacho	6	2	60
Statistics Tutorial Day	Chris Brady	6	2	65
Notes on the Papers		6	2	70
Tutorial Sessions	Jonathan Martin	6	2	101
ASL at Gregynog	Alan Sykes	6	3	59
BAA at British Airways	Sylvia Camacho	6	3	70
Multicurrency Accounting	Peter Branson	6	3	73
APL for Financial Calculations	D.O. Forfar	6	3	76
Workshop on Defined Operators	John Scholes	6	4	64
A Hypertext Interface to the Oracle	Adrian Smith	6	4	71
General Articles:				
Printing APL Matrices	Jerry M. Brennan	6	1	64
Personal Construct Analysis in APL	Mike Williams	6	1	73
Symbolic Computation and Recursion	Tony O'Hagan	6	3	80
A Design Framework for APL Systems	Dave Piper	6	4	78
Technical Articles:				
Hackers' Corner				
Zapping APL*PLUS version-8	Adrian Smith	6	1	90
256 VGA Colours	Adrian Smith	6	2	114
The Falkoff Dual Keyboard	Adrian Smith	6	4	92
Error Trapping				
APL*PLUS (mainframe & PC)	David Ziemann	6	1	91
APL68000	David Eastwood	6	1	96
Dyalog APL	Pauline Brand	6	1	100
Sharp APL	Peter Biddlecombe	6	3	102
APL2/PC	Norman Thomson	6	4	105
A Direct Definition Handler	David Ziemann	6	1	110
Semantic Class and Arrays of Functions	Dave Piper	6	1	118
APL Printing from APL68000	Iain Hayward	6	1	127
APL Standards Update	David Ziemann	6	2	118
Punctuation in APL	Joseph De Kerf	6	2	123
Problems for APL Buffs (I)	Zdenek V Jizba	6	3	109
.... (II)		6	4	117
The GEM File Selector in APL68000	John Sullivan	6	3	113
A Vector Construction Kit	Adrian Smith	6	3	117
A Note on Quad and QuoteQuad	Joseph De Kerf	6	3	122
A New Devt Environment in Dyalog APL	John Scholes	6	4	101
A Note on Comparison Tolerance	Williams & Green	6	4	123
Reviews				
CIPS APL Toolkit	Jonathan Barman	6	1	51
Using IBM's APL2 on the PC	Anthony Camacho	6	1	57
Impressions of Dyalog APL for MS DOS	Jonathan Martin	6	1	61
IBM's APL2 on the PC				
16-bit	Dick Bowman	6	2	45
32-bit	Nicholas Small	6	2	52
Myriade and AFM file systems	Adrian Smith	6	3	51
APL*PLUS/PC Release 9	Adrian Smith	6	4	54

Goodbye, and



*thanks for all
the fish!*

Index to Advertisers

APL People	8
Cocking and Drury	10
Dyadic Systems Ltd	6
HMW Computing (2xhalf)	76
MicroAPL (2xhalf)	16
Saladin	2
APL Booklist (Renaissance Data Systems)	32

All queries regarding advertising in VECTOR should be made to Alison Chatterton, at the address on the inside back cover.

BAA: Membership Application Form

Membership of the British APL Association is open to anyone interested in APL.
The membership year runs from 1st May to 30th April.

Name: _____
Address Line 1: _____
Address Line 2: _____
Address Line 3: _____
Post or zip code: _____
Country: _____
Telephone Number: _____

Membership category (please circle):89/90

Non-voting student membership (UK only) £5
UK private membership £10
Overseas private membership £18
Airmail supplement (not needed for Europe) £8
Corporate membership £85
Corporate membership overseas £140
Sustaining membership £360

For student applicants:

Name of course: _____
Name and title of supervisor: _____
Signature of supervisor: _____

If you would like to pay by credit card ...

Card no (Access/Visa): _____

Signature (and Expiry Date): _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque "The British APL Association", or you may quote your Access or Visa number. Please send the completed form to:

BAA Administration,
Alison Chatterton, 9 Oak Grove,
HERTFORD SG13 8AT
England

Contributions

All contributions to VECTOR may be sent to the Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (ideally with a photograph of the author and a brief biographical note). Unless otherwise specified, each item will be considered for publication as a personal statement by the author.

Please supply as much material as possible in machine-readable form, ideally on an IBM PC compatible (DSD) diskette. APL code should be kept separate from the text, and can be accepted as camera-ready copy, or as APL*PLUS/PC workspaces or IBM APL transfer format files.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1990

Category	Fee	Vectors	Passes
Non-voting Student	£5	1	1
UK Private	£10	1	1
Overseas Private	£18	1	1
(Supplement for Airmail)	£8		
UK Corporate Membership	£85	10+	5
Overseas Corporate	£140	10+	
Sustaining	£360	50+	5

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Treasurer on the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £125 for half-page or less (There is a £50 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary.

Advertisements should be booked with, and sent to, Alison Chatterton, whose address is given with the Index to Advertisers.

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People
The Old Malthouse
Clarence St, BATH, BA1 5NS
Tel:0225-462602

Cocking & Drury Ltd
180 Tottenham Court Rd
LONDON, W1P 9LE
Tel:071-436 9481

HMW Computing Ltd
Hamilton House,
1 Temple Avenue
LONDON EC4Y 0HA
Tel:071-353 4212
Fax:071-353 3325

Mela Technics Systems Ltd
358 West Barnes Lane
Mottspur Park, Surrey
Tel:081-949-6238
Fax:081-336-1062

I.P. Sharp Associates
1-4 Singer St
LONDON EC2A 4BQ
Tel:071-867 1166
Fax:071-867 9792

Intelligent Programs Ltd
9 Gun Wharf, 130 Wapping High St,
LONDON, E1 9NH
Tel:071-265 1120

Dyadic Systems Ltd
Park House, The High Street
ALTON, Hants
Tel:0420-87024

APL Impetus Ltd
Rusper, Sandy Lane
Ivy Hatch, SEVENOAKS
Kent TN15 0PD
Tel:0732-885126

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel:071-922 8866

Peter Cyriax Systems
213 Goldhurst Terrace
LONDON NW6 3ER
Tel:071-624-7013
Mobile:0860-377963



The British Computer Society, 13 Mansfield Street, London W1M 0BD.